

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế và xây dựng hệ thống quản lý thư viện tích
hợp trợ lý ảo

VŨ THỊ QUỲNH NHƯ
NHU.VTQ215110@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Đỗ Tuấn Anh

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế và xây dựng hệ thống quản lý thư viện tích
hợp trợ lý ảo

VŨ THỊ QUỲNH NHƯ
NHU.VTQ215110@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Đỗ Tuấn Anh _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Lời đầu tiên, xin được gửi lời cảm ơn chân thành và sâu sắc tới TS. Đỗ Tuấn Anh – giảng viên hướng dẫn, người đã tận tâm chỉ dẫn, gợi ý và định hướng trong suốt quá trình thực hiện đồ án. Nhờ có sự hỗ trợ quý báu ấy, quá trình phát triển và hoàn thiện đồ án đã diễn ra một cách thuận lợi.

Xin trân trọng cảm ơn quý thầy cô Trường Công Nghệ Thông Tin và Truyền Thông, những người đã tận tình giảng dạy và truyền đạt nền tảng kiến thức vững chắc, tạo điều kiện thuận lợi cho việc thực hiện đồ án.

Chân thành cảm ơn gia đình và những người thân yêu, những người luôn đồng hành, quan tâm, động viên và yêu thương trong suốt quá trình thực hiện. Chính sự động viên ấy đã trở thành nguồn động lực to lớn và điểm tựa tinh thần vững chắc để vượt qua mọi khó khăn.

Sau cùng, xin gửi lời cảm ơn tới những người bạn đồng hành, những người đã luôn giúp đỡ, chia sẻ và đóng góp ý kiến quý báu trong suốt quá trình thực hiện đồ án. Những khoảnh khắc học tập, thảo luận và làm việc nhóm đã tạo nên một môi trường tích cực, thúc đẩy sự nỗ lực và hoàn thiện bản thân mỗi ngày.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh hệ thống giáo dục và thư viện số phát triển mạnh, nhu cầu quản lý hiệu quả tài liệu học thuật, đồng thời nâng cao trải nghiệm tra cứu của độc giả ngày càng được quan tâm. Tuy nhiên, nhiều hệ thống quản lý thư viện hiện nay mới chỉ dừng lại ở việc lưu trữ thông tin và mượn trả đơn thuần, chưa khai thác được tiềm năng của dữ liệu và trí tuệ nhân tạo để hỗ trợ tương tác thông minh.

Để khắc phục vấn đề trên, hướng tiếp cận xây dựng một hệ thống quản lý thư viện thông minh tích hợp trợ lý ảo để tìm kiếm và trả lời chính xác theo ngữ cảnh đã được xây dựng. Hướng tiếp cận này không chỉ nâng cao hiệu quả quản lý, mà còn hỗ trợ độc giả chủ động, nhanh chóng trong việc tiếp cận nội dung sách truyện và các quy định vận hành. Hệ thống cho phép quản lý thông tin sách, tác giả, độc giả, danh mục, quy trình mượn trả năm bước trực tuyến và thiết lập thời gian linh hoạt. Ngoài ra, việc tương tác với thư viện cũng trở nên đơn giản và hiệu quả hơn khi độc giả có thể trò chuyện trực tiếp với trợ lý ảo, nhận câu trả lời trung thực tuyệt đối từ dữ liệu sách và sổ tay nội quy mà không bị bỏ sót chi tiết.

Đóng góp chính của đồ án là một hệ thống dễ triển khai thực tế, có khả năng mở rộng và đồng bộ dữ liệu linh hoạt. Kết quả đạt được là một ứng dụng hoạt động ổn định, có độ chính xác câu trả lời và hỗ trợ rõ ràng cho ban quản lý cũng như độc giả trong quá trình vận hành thư viện số.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.1.1 Khảo sát từ độc giả.....	5
2.1.2 Khảo sát từ quản lý/thư thư viện.....	5
2.2 Tổng quan chức năng	5
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã Tìm kiếm và xem chi tiết sách.....	7
2.2.3 Biểu đồ use case phân rã Quản lý tài khoản	8
2.2.4 Biểu đồ use case phân rã Mượn sách.....	9
2.2.5 Biểu đồ use case phân rã Xử lý các yêu cầu mượn trả	10
2.2.6 Biểu đồ use case phân rã Quản lý danh mục thư viện.....	11
2.3 Đặc tả chức năng	12
2.3.1 Đặc tả use case Tìm kiếm sách	12
2.3.2 Đặc tả use case Gửi yêu cầu mượn sách.....	13
2.3.3 Đặc tả use case Hỏi đáp với AI Chatbot.....	14

2.3.4 Đặc tả use case Phê duyệt yêu cầu mượn sách.....	15
2.3.5 Đặc tả use case Ghi nhận đã trả sách.....	16
2.3.6 Đặc tả use case Quản lý sách.....	17
2.3.7 Đặc tả use case Đăng ký độc giả.....	18
2.3.8 Đặc tả use case Xem báo cáo phần trăm mượn trả	19
2.4 Yêu cầu phi chức năng	19
2.5 Tổng kết.....	20
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	21
3.1 HTML, CSS, JavaScript và Bootstrap.....	21
3.2 Ruby on Rails.....	22
3.3 PostgreSQL.....	23
3.4 Python FastAPI	24
3.5 LangChain	25
3.6 ChromaDB.....	26
3.7 Gemini LLM.....	26
3.8 Tổng kết.....	27
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	28
4.1 Thiết kế kiến trúc.....	28
4.1.1 Lựa chọn kiến trúc phần mềm	28
4.1.2 Thiết kế tổng quan.....	31
4.1.3 Thiết kế chi tiết gói	35

4.2 Thiết kế giao diện	37
4.2.1 Thông số kỹ thuật	37
4.2.2 Quy chuẩn thiết kế giao diện nhất quán	38
4.2.3 Bố cục phác thảo một số giao diện quan trọng	39
4.2.4 Thiết kế chi tiết lớp	42
4.2.5 Biểu đồ trình tự một số use case quan trọng	47
4.3 Thiết kế cơ sở dữ liệu	51
4.3.1 Biểu đồ thực thể liên kết (E-R Diagram)	51
4.3.2 Thiết kế chi tiết các bảng cơ sở dữ liệu	52
4.4 Xây dựng ứng dụng.....	59
4.4.1 Thư viện và công cụ sử dụng.....	59
4.4.2 Kết quả đạt được	60
4.4.3 Minh họa các chức năng chính	62
4.5 Kiểm thử.....	66
4.5.1 Các kỹ thuật kiểm thử sử dụng	66
4.5.2 Thiết kế các trường hợp kiểm thử cho chức năng quan trọng	67
4.6 Triển khai	70
4.6.1 Mô hình và cách thức triển khai.....	70
4.6.2 Cấu hình hạ tầng triển khai thử nghiệm	70
4.6.3 Đánh giá kết quả triển khai	71
4.7 Tổng kết.....	72

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	73
5.1 Giải pháp tìm kiếm lai và tái xếp hạng tài liệu (Hybrid Search & Reranking)	73
5.2 Hệ thống định tuyến thông minh kết hợp truy vấn SQL trực tiếp	75
5.3 Quy trình phân mảnh phân cấp và nạp dữ liệu bất đồng bộ	77
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	80
6.1 Kết luận	80
6.1.1 Tự đánh giá kết quả thực hiện đồ án	80
6.1.2 Đóng góp nổi bật	81
6.1.3 Bài học kinh nghiệm rút ra	81
6.2 Hướng phát triển	82
6.2.1 Hoàn thiện các chức năng hiện tại	82
6.2.2 Nâng cấp và mở rộng chức năng	82
6.3 Tổng kết	83
TÀI LIỆU THAM KHẢO	84

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát	6
Hình 2.2	Biểu đồ use case phân rã Tìm kiếm và xem chi tiết sách	7
Hình 2.3	Biểu đồ use case phân rã Quản lý tài khoản	8
Hình 2.4	Biểu đồ use case phân rã Mượn sách	9
Hình 2.5	Biểu đồ use case phân rã Xử lý các yêu cầu mượn trả	10
Hình 2.6	Biểu đồ use case phân rã Quản lý danh mục thư viện	11
Hình 4.1	Mô hình kiến trúc MVC	29
Hình 4.2	Biểu đồ gói tổng quan	31
Hình 4.3	Biểu đồ gói chi tiết	35
Hình 4.4	Phác thảo bố cục trang chủ độc giả tích hợp Chatbot AI	39
Hình 4.5	Phác thảo bố cục mockup Chatbot AI	40
Hình 4.6	Phác thảo bố cục trang tìm kiếm sách của độc giả	40
Hình 4.7	Phác thảo bố cục trang xem chi tiết sách của độc giả	41
Hình 4.8	Phác thảo bố cục trang chủ của thủ thư/Admin	41
Hình 4.9	Phác thảo bố cục trang chủ quản lý user của thủ thư/Admin . .	42
Hình 4.10	Biểu đồ trình tự luồng Độc giả tạo yêu cầu mượn sách	47
Hình 4.11	Biểu đồ trình tự luồng hỏi đáp Chatbot RAG	48
Hình 4.12	Biểu đồ trình tự luồng Thủ thư duyệt yêu cầu mượn sách . . .	50
Hình 4.13	Biểu đồ thực thể liên kết (E-R Diagram)	51
Hình 4.14	Giao diện Giỏ sách mượn của độc giả	63
Hình 4.15	Giao diện Trang chủ của user tích hợp trợ lý ảo Thủ thư online	64
Hình 4.16	Bảng điều khiển hoạt động dành cho Quản trị viên	64
Hình 4.17	Trang quản lý các yêu cầu mượn sách của thủ thư	65
Hình 4.18	Giao diện quản lý danh sách độc giả	66
Hình 5.1	Quy trình tìm kiếm lai Hybrid Search và tái xếp hạng Cohere Rerank	74

Hình 5.2	Định nghĩa cấu trúc phân loại ý định đầu ra	76
Hình 5.3	Sơ đồ định tuyến ý định câu hỏi của Agent Router	76
Hình 5.4	Quy trình số hóa sách PDF và đồng bộ Vector DB bất đồng bộ	78
Hình 5.5	Cơ chế Caching khắc phục "bom nổ chậm" RAM	79

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả use case Tìm kiếm sách	12
Bảng 2.2	Đặc tả use case Gửi yêu cầu mượn sách	13
Bảng 2.3	Đặc tả use case Hỏi đáp với AI Chatbot	14
Bảng 2.4	Đặc tả use case Phê duyệt yêu cầu mượn sách	15
Bảng 2.5	Đặc tả use case Ghi nhận đã trả sách	16
Bảng 2.6	Đặc tả use case Quản lý sách	17
Bảng 2.7	Đặc tả use case Đăng ký độc giả	18
Bảng 2.8	Đặc tả use case Xem báo cáo phần trăm mượn trả	19
Bảng 4.1	Đặc tả phương thức lớp ChatbotController	42
Bảng 4.2	Đặc tả thuộc tính và hằng số lớp Admin::BorrowRequestsController	43
Bảng 4.3	Đặc tả phương thức lớp Admin::BorrowRequestsController . .	43
Bảng 4.4	Đặc tả thuộc tính và hằng số lớp BorrowRequestController . .	44
Bảng 4.5	Đặc tả phương thức lớp BorrowRequestController	44
Bảng 4.6	Đặc tả thuộc tính lớp RAGEngine	45
Bảng 4.7	Đặc tả phương thức chính lớp RAGEngine	46
Bảng 4.8	Cấu trúc chi tiết bảng users	53
Bảng 4.9	Cấu trúc chi tiết bảng books	54
Bảng 4.10	Cấu trúc chi tiết bảng authors	54
Bảng 4.11	Cấu trúc chi tiết bảng publishers	55
Bảng 4.12	Cấu trúc chi tiết bảng categories	55
Bảng 4.13	Cấu trúc chi tiết bảng book_categories	55
Bảng 4.14	Cấu trúc chi tiết bảng book_sections	56
Bảng 4.15	Cấu trúc bảng borrow_requests (Phần 1: Thông tin đặt mượn)	57
Bảng 4.16	Cấu trúc bảng borrow_requests (Phần 2: Thông tin kiểm duyet & Nhật ký)	57
Bảng 4.17	Cấu trúc chi tiết bảng borrow_request_items	58

Bảng 4.18	Cấu trúc chi tiết bảng reviews	58
Bảng 4.19	Cấu trúc chi tiết bảng favorites	59
Bảng 4.20	Danh sách thư viện và công cụ sử dụng	60
Bảng 4.21	Thông kê các số liệu kỹ thuật của hệ thống	62
Bảng 4.22	Các trường hợp kiểm thử chức năng Tạo phiếu mượn sách . .	67
Bảng 4.23	Các trường hợp kiểm thử chức năng Phê duyệt phiếu mượn . .	68
Bảng 4.24	Các trường hợp kiểm thử chức năng Hỏi đáp Chatbot RAG . .	69

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
CNTT		Công nghệ thông tin
ĐATN		Đồ án tốt nghiệp
ERD	Entity-Relationship Diagram	Sơ đồ mối quan hệ thực thể
EUD	End-User Development	Phát triển ứng dụng người dùng cuối
GWT	Google Web Toolkit	Công cụ lập trình Javascript bằng Java của Google
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MVC	Model-View-Controller	Mô hình Model-View-Controller
PDF	Portable Document Format	Định dạng tài liệu di động
RAG	Retrieval-Augmented Generation	Tạo văn bản tăng cường truy xuất
SV		Sinh viên

Thuật ngữ	Ý nghĩa / Giải thích
Backend	Phần xử lý logic nghiệp vụ và lưu trữ dữ liệu phía máy chủ
Chatbot	Robot đối thoại trực tuyến hỗ trợ tương tác tự động với độc giả
Database	Cơ sở dữ liệu lưu trữ thông tin sách, người dùng và yêu cầu mượn trả
Embedding	Kỹ thuật biểu diễn từ hoặc đoạn văn bản thành các vectơ số thực mang ngữ nghĩa
Framework	Khung phần mềm cung cấp các thư viện và cấu trúc để phát triển ứng dụng
Frontend	Phần giao diện hiển thị trực quan tương tác trực tiếp với người dùng
Prompt	Câu lệnh hoặc câu hỏi đầu vào cung cấp cho mô hình ngôn ngữ lớn
Semantic Search	Phương thức tìm kiếm dựa trên sự tương đồng ngữ nghĩa thay vì chỉ so khớp từ khóa
Vector Database	Cơ sở dữ liệu chuyên dụng để lưu trữ và truy vấn nhanh các vectơ ngữ nghĩa

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong những năm gần đây, xu hướng chuyển đổi số trong giáo dục và thư viện số tại Việt Nam không ngừng tăng trưởng, kéo theo nhu cầu quản lý tài liệu học thuật một cách hiệu quả và chuyên nghiệp. Sự gia tăng ngày càng cao của khối lượng tri thức và yêu cầu từ độc giả khiến các đơn vị phải liên tục nâng cao chất lượng dịch vụ, tối ưu quy trình vận hành và chủ động trong việc hỗ trợ người dùng tiếp cận thông tin. Tuy nhiên, nhiều thư viện hiện vẫn đang quản lý mượn trả một cách thủ công hoặc sử dụng các phần mềm rời rạc, thiếu tính liên kết và khả năng khai thác sâu kho dữ liệu sẵn có.

Bên cạnh đó, một trong những thách thức lớn mà các thư viện thường gặp phải là việc người dùng gặp khó khăn khi tra cứu nội dung sách, cũng như nắm bắt các điều khoản phức tạp trong sổ tay nội quy vận hành. Việc thiếu một công cụ tương tác thông minh, tự động trả lời chính xác dựa trên ngữ cảnh thực tế không chỉ làm tăng gánh nặng công việc cho đội ngũ thủ thư, mà còn gián tiếp làm giảm hiệu suất khai thác tài nguyên và vòng quay của tài liệu.

Ngoài ra, xu hướng của độc giả hiện nay là ưu tiên các dịch vụ nhanh chóng, tiện lợi, đặc biệt là khả năng đặt mượn trực tuyến một cách dễ dàng và tương tác không giới hạn thời gian qua các nền tảng số. Trong bối cảnh công nghệ phát triển mạnh mẽ, nếu các thư viện không bắt kịp xu hướng tích hợp trí tuệ nhân tạo để cá nhân hóa trải nghiệm tra cứu, họ sẽ khó thu hút được độc giả và có nguy cơ khiến không gian tri thức trở nên xa lạ với thế hệ người dùng hiện đại.

Nếu vấn đề này được giải quyết tốt, nó không chỉ giúp ban quản lý thư viện điều hành hệ thống một cách khoa học, tự động hóa quy trình nghiệp vụ, mà còn mang lại trải nghiệm tương tác thuận tiện, trung thực tuyệt đối cho độc giả trong việc tìm kiếm kiến thức – từ đó góp phần gia tăng hiệu quả vận hành và nâng cao văn hóa đọc trong môi trường số.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, trên thị trường đã xuất hiện nhiều hệ thống hỗ trợ quản lý thư viện và tra cứu tài liệu trực tuyến, từ các nền tảng lớn dành cho trường học, tổ chức cho đến các phần mềm quản lý nội bộ được phát triển riêng cho từng thư viện hoặc chuỗi thư viện số. Các hệ thống này thường cung cấp những chức năng cơ bản như quản lý thông tin đầu sách, theo dõi tình trạng mượn trả, xử lý thẻ độc giả và lưu trữ dữ liệu người dùng. Nhờ đó, các thư viện có thể giảm thiểu thao tác thủ công, nâng

cao hiệu quả vận hành và cải thiện chất lượng dịch vụ.

Tuy nhiên, phần lớn các nền tảng hiện có vẫn tập trung chủ yếu vào các chức năng truyền thống, chưa tận dụng tốt tiềm năng của cơ sở dữ liệu tri thức để mang lại giá trị tương tác cao hơn trong hoạt động tra cứu. Một số nền tảng cao cấp có tích hợp công cụ tìm kiếm nâng cao, song mức chi phí triển khai và duy trì khá cao, cùng với khả năng tùy biến thấp khiến các thư viện vừa và nhỏ gặp nhiều khó khăn trong việc tiếp cận và khai thác hiệu quả. Điều này tạo ra một khoảng trống rõ rệt giữa nhu cầu thực tế và các giải pháp hiện hành.

Về phía người dùng, xu hướng chung là ưu tiên sự nhanh chóng, tiện lợi và khả năng tiếp cận dịch vụ mọi lúc, mọi nơi. Do đó, một hệ thống thư viện hiện đại cần phải đáp ứng tốt các tiêu chí như giao diện trực quan, dễ sử dụng, tốc độ phản hồi nhanh và khả năng hoạt động ổn định trên nhiều nền tảng khác nhau như trình duyệt web, ứng dụng điện thoại hoặc máy tính bảng. Trải nghiệm người dùng (UX) ngày càng trở thành yếu tố cạnh tranh quan trọng bên cạnh số lượng đầu sách sẵn có.

Mặc dù công nghệ đã có nhiều bước tiến đáng kể, nhưng một số hạn chế vẫn còn tồn tại. Đặc biệt, khả năng tương tác thông minh và tra cứu thông tin theo ngữ cảnh tự nhiên – vốn là một trong những yếu tố then chốt giúp độc giả chủ động nắm bắt nội dung tài liệu và các quy định vận hành phức tạp – vẫn chưa được quan tâm đúng mức. Đồng thời, một số sản phẩm vẫn chưa đầu tư bài bản vào thiết kế giao diện và nâng cao trải nghiệm người dùng, khiến độc giả dễ bị mất thời gian hoặc gặp khó khăn trong quá trình thao tác mượn trả.

Từ những phân tích đã trình bày, có thể nhận thấy rằng việc xây dựng một hệ thống quản lý thư viện thông minh tích hợp trợ lý ảo là nhu cầu cấp thiết trong bối cảnh hiện nay. Hệ thống không chỉ đáp ứng các chức năng quản lý cơ bản mà còn có khả năng trả lời chính xác, trung thực các câu hỏi về sách và nội quy dựa trên ngữ cảnh được cung cấp. Hệ thống này sẽ giúp ban quản lý thư viện điều hành hiệu quả hơn, tối ưu hóa nguồn lực và tự động hóa quy trình nghiệp vụ trong từng thời điểm. Đồng thời, đối với độc giả, hệ thống cũng sẽ mang đến trải nghiệm tra cứu thuận tiện, nhanh chóng và đáng tin cậy. Các chức năng chính của hệ thống bao gồm: quản lý sách, mượn trả trực tuyến, quản lý độc giả và tích hợp trợ lý ảo để tìm kiếm ngữ cảnh.

1.3 Định hướng giải pháp

Để giải quyết bài toán đã đặt ra, hệ thống web ứng dụng theo kiến trúc Client-Server đã được định hướng phát triển. Công nghệ được sử dụng gồm: HTML, CSS, Javascript cho phần giao diện người dùng nhằm đảm bảo tính tương tác và phản

hồi nhanh; Ruby on Rails cho backend giúp xử lý nghiệp vụ mượn trả và kết nối cơ sở dữ liệu. Đặc biệt, hệ thống tích hợp một mô hình trợ lý ảo sử dụng kỹ thuật sinh tăng cường tìm kiếm (RAG) kết hợp Hybrid Search, triển khai độc lập bằng Python (FastAPI) với LangChain, ChromaDB và giao tiếp qua API, để giải đáp tự động và tìm kiếm thông tin theo ngữ cảnh dựa trên dữ liệu thư viện sẵn có.

Hệ thống giúp độc giả đặt mượn sách một cách thuận tiện và dễ dàng, có thể theo dõi lịch sử mượn trả và gia hạn thời gian lưu giữ tài liệu trực tuyến. Độc giả cũng có thể tương tác trực tiếp với trợ lý ảo để được giải đáp thắc mắc. Bên cạnh đây, hệ thống giúp nhân viên thư viện dễ dàng quản lý các danh mục đầu sách, xác nhận phiếu mượn, tiếp nhận trả sách. Ngoài ra, hệ thống còn cung cấp công cụ trợ lý ảo thông minh, giúp ban quản lý tối ưu hóa quy trình hỗ trợ người dùng và vận hành thư viện hiệu quả hơn.

Đóng góp chính của đề án là xây dựng được một hệ thống có khả năng vận hành thực tế, tích hợp trợ lý ảo thông minh một cách linh hoạt, dễ mở rộng. Kết quả đạt được là phần mềm hoàn chỉnh với giao diện thân thiện, hiệu suất ổn định và mô hình tìm kiếm ngữ cảnh có độ chính xác trong bối cảnh ứng dụng thực tiễn.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp được tổ chức như sau.

Chương 2 tập trung khảo sát thực tế nhu cầu tra cứu và quản lý thư viện, đồng thời xác định các yêu cầu cần thiết cho hệ thống cần xây dựng. Qua việc thu thập ý kiến từ người dùng và tham khảo các giải pháp công nghệ hiện đại, chương sẽ xác định rõ các chức năng cần thiết. Ngoài ra, chương cũng đưa ra đặc tả yêu cầu chức năng và phi chức năng chi tiết, làm cơ sở cho các bước thiết kế và triển khai hệ thống trong các chương tiếp theo.

Chương 3 trình bày tổng quan về các công nghệ, nền tảng và công cụ được lựa chọn để xây dựng hệ thống, bao gồm cả công nghệ phần mềm và mô hình trợ lý ảo RAG phục vụ cho việc tìm kiếm ngữ cảnh. Mỗi công nghệ đều được phân tích về ưu điểm, nhược điểm và khả năng ứng dụng trong bối cảnh bài toán. Việc lựa chọn được đưa ra dựa trên các yêu cầu đã xác định ở Chương 2, đảm bảo tính phù hợp, khả thi và hiệu quả khi triển khai vào hệ thống thực tế.

Chương 4 tập trung trình bày quá trình thiết kế và xây dựng hệ thống quản lý thư viện thông minh tích hợp trợ lý ảo. Nội dung bao gồm lựa chọn kiến trúc phần mềm phù hợp, thiết kế các biểu đồ UML để thể hiện cấu trúc hệ thống và mối quan hệ giữa các thành phần. Chương cũng mô tả chi tiết quá trình xây dựng giao diện người dùng, thiết kế cơ sở dữ liệu, phát triển các chức năng chính như mượn trả

trực tuyến, đồng bộ tài liệu hệ thống và thực hiện kiểm thử, triển khai hệ thống để đánh giá mức độ đáp ứng yêu cầu thực tế.

Chương 5 trình bày những đóng góp quan trọng trong quá trình thực hiện đồ án, bao gồm các giải pháp xử lý bài toán nghiệp vụ đặc thù trong quản lý thư viện, ứng dụng kỹ thuật sinh tăng cường tìm kiếm (RAG) kết hợp Hybrid Search (Vector Search + BM25) và Cohere Rerank để tối ưu hóa khả năng truy vấn nội dung sách, xây dựng Agent Router định tuyến ý định câu hỏi đầu vào, cũng như thiết kế phương thức nạp dữ liệu bất đồng bộ giúp số hóa tài liệu PDF dung lượng lớn. Mỗi nội dung đều được trình bày có hệ thống với phần nêu vấn đề, giải pháp kỹ thuật được đề xuất và kết quả đạt được sau triển khai.

Chương 6 tổng kết quá trình thực hiện đồ án tốt nghiệp, đánh giá những kết quả đã đạt được, các điểm còn hạn chế, cùng những bài học kinh nghiệm rút ra trong suốt quá trình triển khai. Bên cạnh đó, chương cũng đề xuất các hướng phát triển trong tương lai nhằm hoàn thiện hệ thống, bao gồm việc nâng cấp các tính năng hiện có, cải thiện độ chính xác và tốc độ phản hồi của trợ lý ảo RAG, tối ưu hiệu năng hệ thống và mở rộng quy mô phục vụ cho nhiều thư viện hơn.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này tập trung vào việc khảo sát hiện trạng thực tế liên quan đến bài toán quản lý thư viện và tra cứu tài liệu, bao gồm việc phân tích các hệ thống hiện có, thu thập nhu cầu từ người dùng và đánh giá các xu hướng công nghệ liên quan. Trên cơ sở đó, chương sẽ đưa ra tổng quan về các chức năng chính cần phát triển trong đồ án. Đồng thời, chương cũng tiến hành đặc tả chi tiết các yêu cầu chức năng và phi chức năng, làm cơ sở quan trọng cho việc thiết kế, xây dựng và đánh giá hệ thống phần mềm ở các chương tiếp theo.

2.1 Khảo sát hiện trạng

Để phục vụ cho quá trình xây dựng một hệ thống quản lý thư viện thông minh và tra cứu tài liệu hiệu quả, quá trình khảo sát đã được tiến hành dựa trên hai nguồn thông tin chính. Đầu tiên là phản hồi từ người dùng tiềm năng bao gồm ban quản lý thư viện (thủ thư) và độc giả (học sinh, sinh viên).

2.1.1 Khảo sát từ độc giả

Độc giả thường là những người có thói quen tìm kiếm sách và mượn trả trực tuyến thông qua các cổng thông tin thư viện điện tử hoặc ứng dụng số. Nhóm này yêu cầu hệ thống tiện lợi, sử dụng dễ dàng và cung cấp thông tin đầy đủ về các đầu sách, tác giả, thể loại và tình trạng khả dụng của tài liệu. Đồng thời họ mong muốn có thể dễ dàng tương tác, đặt mượn trực tuyến và đặc biệt là nhận được câu trả lời giải đáp nhanh chóng về nội dung sách hoặc quy định thư viện mà không cần mất thời gian tìm kiếm thủ công từng trang tài liệu.

2.1.2 Khảo sát từ quản lý/thủ thư thư viện

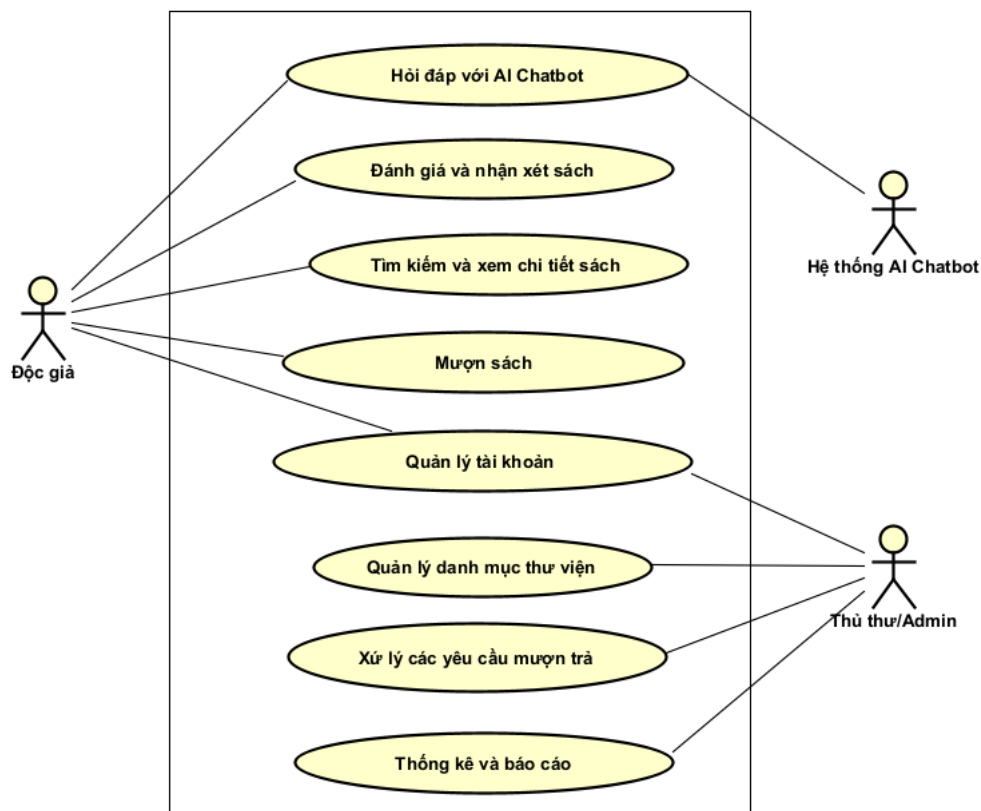
Nhóm này có các yêu cầu tập trung vào tính hiệu quả trong quản lý vận hành. Họ cần một hệ thống có khả năng quản lý đầy đủ các thông tin về sách, danh mục, tài khoản độc giả, và trạng thái mượn trả theo thời gian thực. Việc phân quyền người dùng theo vai trò cũng rất quan trọng để đảm bảo tính bảo mật và phù hợp với quy trình nghiệp vụ. Ngoài ra, hệ thống cần cung cấp các chức năng thống kê, kiểm soát. Đặc biệt, nhóm này thể hiện sự quan tâm đến việc tích hợp một trợ lý ảo thông minh dựa nhằm giảm tải việc giải đáp các câu hỏi lặp đi lặp lại về quy chế, nội quy và hỗ trợ độc giả định vị tài liệu nhanh chóng.

2.2 Tổng quan chức năng

Phần 2.2 này có nhiệm vụ tóm tắt các chức năng của phần mềm. Trong phần này, các chức năng được tiếp cận và mô tả ở mức cao (tổng quan) nhằm định hình phạm vi, hệ thống và mối quan hệ giữa các tác nhân. Toàn bộ các đặc tả chức năng

chi tiết sẽ được trình bày cụ thể trong phần 2.3.

2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát

Biểu đồ use case tổng quát gồm có 3 tác nhân chính là Độc giả, Thủ thư / Admin và Hệ thống AI Chatbot.

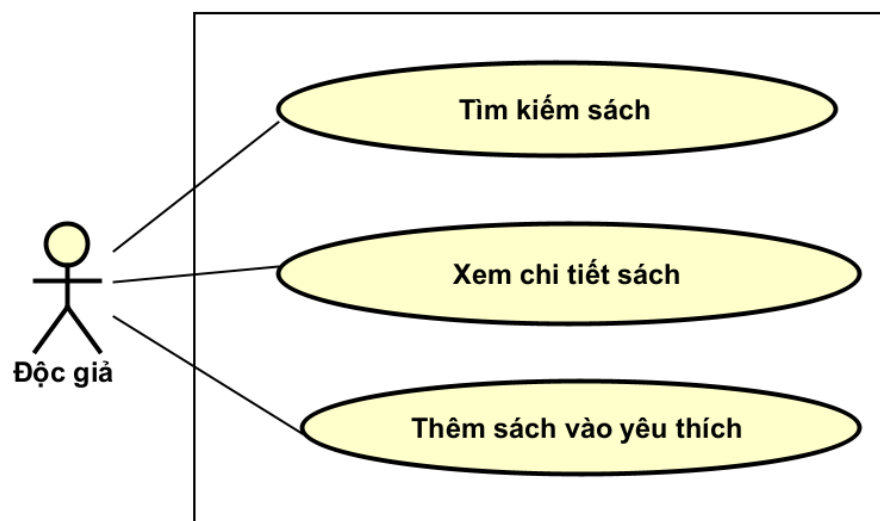
- **Độc giả (User):** Người dùng sử dụng các dịch vụ của thư viện, có nhu cầu tra cứu tài liệu, đăng ký mượn sách trực tuyến, gửi phản hồi và tương tác hỏi đáp thông minh.
- **Thủ thư / Admin:** Người quản trị hệ thống, chịu trách nhiệm quản lý danh mục, kiểm duyệt đơn mượn/trả, quản lý tài khoản người dùng và theo dõi báo cáo hoạt động vận hành.
- **Hệ thống AI Chatbot (System Actor):** Hệ thống phụ trợ API ngoài kết nối với mô hình ngôn ngữ lớn (LLM) để tiếp nhận, phân tích câu hỏi và sinh câu trả lời tự nhiên từ ngữ cảnh tài liệu.

Biểu đồ này gồm 8 use case chính như sau:

1. **Hỏi đáp với AI Chatbot:** Cho phép độc giả gửi thắc mắc và trò chuyện thông minh với Chatbot.

2. Đánh giá và nhận xét sách: Cho phép độc giả viết bình luận và chấm điểm số sao cho các đầu sách.
3. Tìm kiếm và xem chi tiết sách: Giúp độc giả tra cứu nhanh thông tin, kiểm tra tình trạng tồn kho và số lượng khả dụng của sách.
4. Mượn sách: Cho phép độc giả quản lý giỏ mượn, chọn thời gian lưu trú sách và gửi đơn yêu cầu mượn trực tuyến.
5. Quản lý tài khoản: Chức năng cơ bản cho phép người dùng đăng ký, đăng nhập và cập nhật hồ sơ cá nhân.
6. Quản lý danh mục thư viện: Dành cho Thủ thư/Admin cập nhật các thông tin bao gồm độc giả, sách, tác giả, thể loại và nhà xuất bản.
7. Xử lý các yêu cầu mượn trả: Giúp thủ thư duyệt/từ chối đơn mượn trực tuyến và ghi nhận trạng thái sách khi độc giả đến nhận hoặc trả sách tại quầy.
8. Thống kê và báo cáo: Cho phép thủ thư theo dõi các số liệu, tỷ lệ phần trăm mượn trả và danh sách các đầu sách được yêu chuộng nhất.

2.2.2 Biểu đồ use case phân rã Tìm kiếm và xem chi tiết sách



Hình 2.2: Biểu đồ use case phân rã Tìm kiếm và xem chi tiết sách

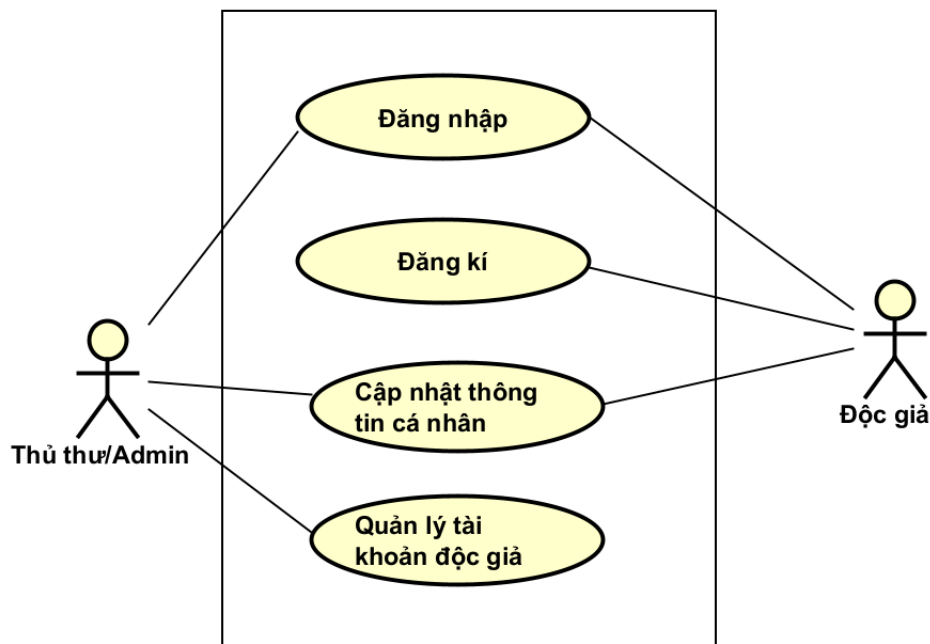
Biểu đồ use case phân rã Tìm kiếm và xem chi tiết sách được phân rã thành các use case với tác nhân Độc giả như sau:

- Tìm kiếm sách: Cho phép độc giả tra cứu danh mục sách qua các từ khóa liên quan đến tiêu đề, tác giả hoặc thể loại.
- Xem chi tiết sách: Cho phép độc giả xem thông tin sâu của sách bao gồm nội

dung tóm tắt, số trang và số lượng khả dụng trong kho.

- Thêm sách vào yêu thích: Giúp độc giả lưu trữ nhanh các đầu sách quan tâm vào danh mục cá nhân để tiện tra cứu lại.

2.2.3 Biểu đồ use case phân rã Quản lý tài khoản

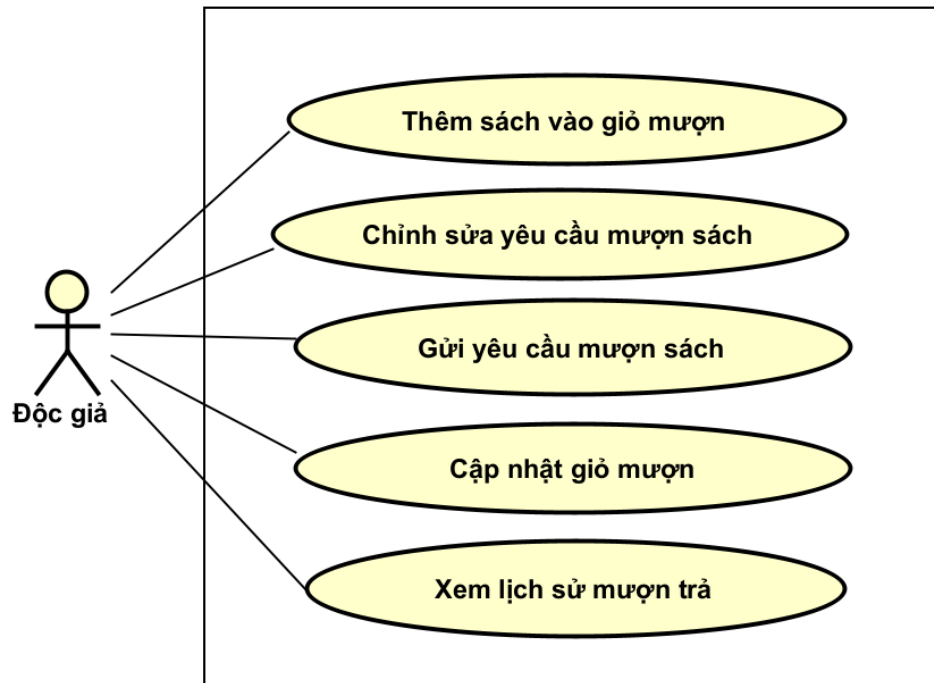


Hình 2.3: Biểu đồ use case phân rã Quản lý tài khoản

Biểu đồ use case phân rã Quản lý tài khoản được phân rã với hai tác nhân là Độc giả và Thủ thư / Admin như sau:

- Đăng nhập: Cho phép Độc giả và Thủ thư/Admin đăng nhập hệ thống để xác thực quyền hạn.
- Đăng ký: Cho phép độc giả mới khởi tạo tài khoản trên hệ thống.
- Cập nhật thông tin cá nhân: Cho phép người dùng chỉnh sửa các thông tin cá nhân và thay đổi mật khẩu bảo mật.
- Quản lý tài khoản độc giả: Dành cho Thủ thư/Admin, cho phép kiểm tra danh sách, kích hoạt hoặc vô hiệu hóa tài khoản của độc giả.

2.2.4 Biểu đồ use case phân rã Mượn sách

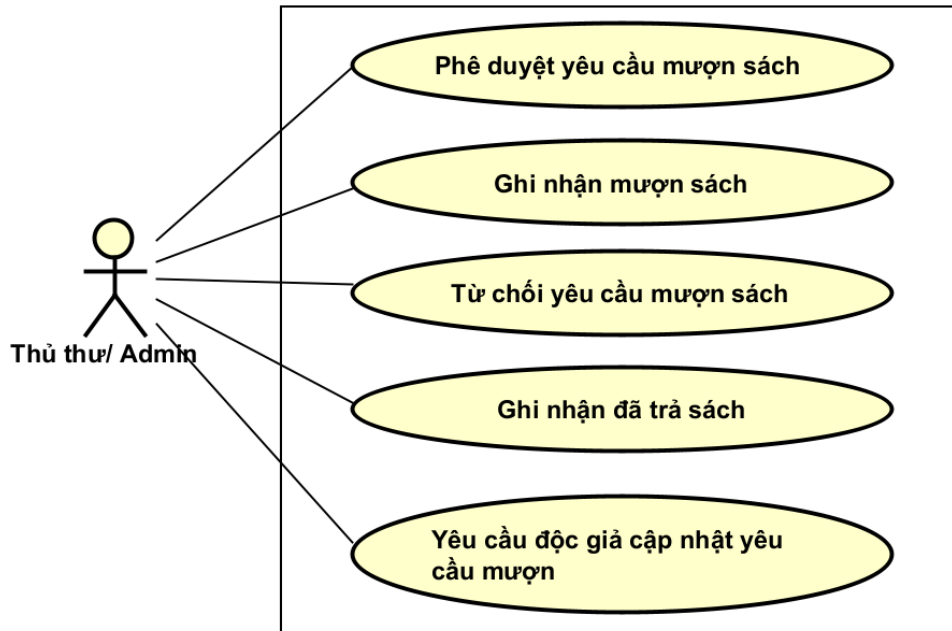


Hình 2.4: Biểu đồ use case phân rã Mượn sách

Biểu đồ use case phân rã Mượn sách được phân rã thành các usecase phục vụ tác nhân Độc giả như sau:

- Thêm sách vào giỏ mượn: Tích hợp đầu sách cần mượn vào danh sách hàng đợi.
- Chỉnh sửa yêu cầu mượn sách: Điều chỉnh thời gian bắt đầu mượn và thời hạn trả dự kiến, sách mượn.
- Cập nhật giỏ mượn: Tăng, giảm hoặc xóa bỏ số lượng các đầu sách đã chọn.
- Gửi yêu cầu mượn sách: Xác nhận đơn và đẩy yêu cầu lên hàng đợi chờ thủ thư phê duyệt.
- Xem lịch sử mượn trả: Theo dõi danh sách, trạng thái và mốc thời gian của các đơn mượn cũ và hiện tại.

2.2.5 Biểu đồ use case phân rã Xử lý các yêu cầu mượn trả

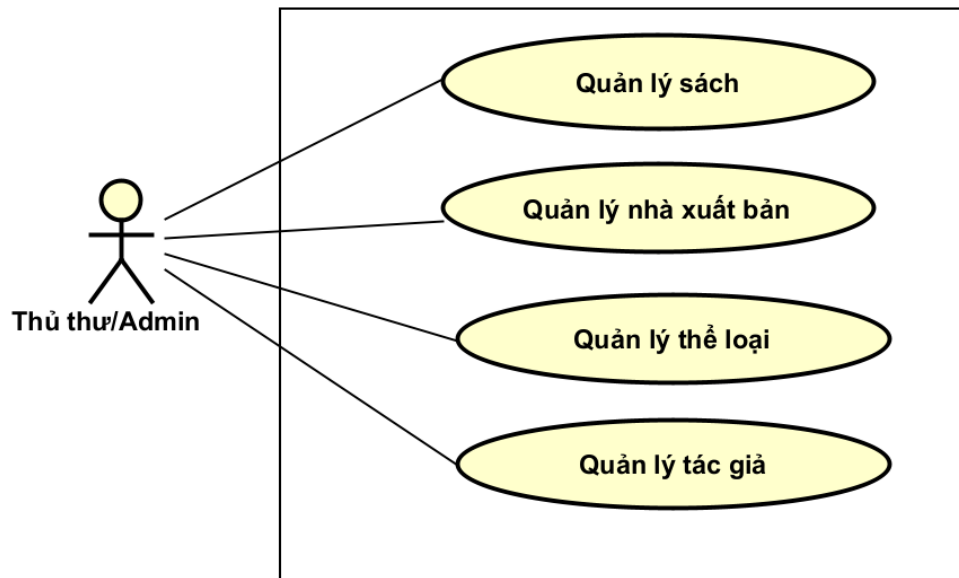


Hình 2.5: Biểu đồ use case phân rã Xử lý các yêu cầu mượn trả

Biểu đồ use case phân rã Xử lý các yêu cầu mượn trả được phân rã thành các usecase dành cho Thủ thư / Admin như sau:

- Phê duyệt yêu cầu mượn sách: Chấp thuận đơn mượn hợp lệ của độc giả trên hệ thống.
- Từ chối yêu cầu mượn sách: Hủy đơn mượn kèm lý do gửi tới độc giả nếu không đáp ứng điều kiện.
- Yêu cầu độc giả cập nhật yêu cầu mượn: Trả đơn yêu cầu về trạng thái sửa đổi nếu thông tin hoặc thời hạn mượn chưa đúng quy định.
- Ghi nhận mượn sách: Xác nhận trạng thái "Đang mượn" khi bàn giao sách vật lý cho độc giả tại quầy.
- Ghi nhận đã trả sách: Xác nhận độc giả hoàn trả sách, đưa đơn mượn về trạng thái hoàn tất và khôi phục số lượng kho.

2.2.6 Biểu đồ use case phân rã Quản lý danh mục thư viện



Hình 2.6: Biểu đồ use case phân rã Quản lý danh mục thư viện

Biểu đồ use case phân rã Quản lý danh mục thư viện bao gồm usecase cốt lõi cho tác nhân Thủ thư / Admin thao tác:

- Quản lý sách: Cho phép xem, thêm, sửa, xóa dữ liệu sách và tải lên tệp văn bản PDF gốc phục vụ cơ sở dữ liệu trí tuệ nhân tạo.
- Quản lý nhà xuất bản: Cập nhật thông tin các đối tác, đơn vị phát hành.
- Quản lý thể loại: Cho phép thêm, sửa, xóa, cập nhật danh mục thể loại.
- Quản lý tác giả: Thêm mới, chỉnh sửa thông tin tiểu sử, lý lịch của các tác giả sách.

2.3 Đặc tả chức năng

2.3.1 Đặc tả use case Tìm kiếm sách

Bảng 2.1: Đặc tả use case Tìm kiếm sách

Mã usecase	UC001	Tên usecase	Tìm kiếm sách
Tác nhân	Độc giả		
Tiền điều kiện	Độc giả đã truy cập vào trang chủ hoặc trang tìm kiếm của hệ thống.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Độc giả	Nhập từ khóa tìm kiếm (tiêu đề sách, tác giả, nhà xuất bản, thể loại) vào thanh tìm kiếm.
	2	Độc giả	(Tùy chọn) Chọn các bộ lọc nâng cao như thể loại cụ thể hoặc nhà xuất bản.
	3	Độc giả	Nhấn nút "Tìm kiếm" hoặc nhấn phím Enter.
	4	System	Thực hiện truy vấn tìm kiếm dữ liệu sách trong cơ sở dữ liệu.
	5	System	Hiển thị danh sách kết quả các cuốn sách phù hợp kèm theo hình ảnh, tên sách, tác giả và trạng thái khả dụng.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	5a	System	Nếu không tìm thấy cuốn sách nào phù hợp, hệ thống hiển thị thông báo "Không tìm thấy kết quả phù hợp. Vui lòng thử từ khóa khác!".
Hậu điều kiện	Danh sách kết quả tìm kiếm sách hiển thị trực quan cho độc giả.		

2.3.2 Đặc tả use case Gửi yêu cầu mượn sách

Bảng 2.2: Đặc tả use case Gửi yêu cầu mượn sách

Mã usecase	UC002	Tên usecase	Gửi yêu cầu mượn sách
Tác nhân	Độc giả		
Tiền điều kiện	Độc giả đã đăng nhập thành công và có ít nhất 1 cuốn sách trong giỏ mượn.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Độc giả	Truy cập vào trang "Giỏ mượn sách".
	2	Độc giả	Kiểm tra danh sách sách và điều chỉnh số lượng mượn mong muốn cho từng cuốn.
	3	Độc giả	Chọn Ngày bắt đầu mượn và Ngày trả dự kiến.
	4	Độc giả	Nhấn chọn nút "Gửi yêu cầu mượn sách".
	5	System	Kiểm tra số lượng sách khả dụng trong kho đối với các cuốn sách được yêu cầu.
	6	System	Tạo mới bản ghi đơn mượn sách với trạng thái ban đầu là "Chờ duyệt" (Pending).
	7	System	Làm sạch giỏ mượn sách hiện tại và thông báo gửi yêu cầu thành công.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	3a	System	Ngày mượn/trả không hợp lệ (ngày mượn trong quá khứ hoặc ngày trả trước ngày mượn). Hệ thống báo lỗi và yêu cầu chọn lại.
	5a	System	Nếu số lượng khả dụng của một hoặc nhiều cuốn trong kho không đủ, hệ thống báo lỗi cạn kho và dừng tạo đơn.
Hậu điều kiện	Đơn mượn sách mới được khởi tạo thành công ở trạng thái Chờ duyệt (Pending) trong cơ sở dữ liệu.		

2.3.3 Đặc tả use case Hỏi đáp với AI Chatbot

Bảng 2.3: Đặc tả use case Hỏi đáp với AI Chatbot

Mã usecase	UC003	Tên usecase	Hỏi đáp với AI Chatbot
Tác nhân	Độc giả, Hệ thống AI Chatbot (Tác nhân phụ trợ)		
Tiền điều kiện	Độc giả truy cập vào giao diện Cổng thông tin Thư viện có tích hợp Chatbot.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Độc giả	Nhập câu hỏi tự nhiên về nội dung tài liệu, danh mục sách hoặc nội quy thư viện và nhấn gửi.
	2	System (Client)	Tự động nạp lịch sử hội thoại từ <code>localStorage</code> và gửi yêu cầu dạng Streaming (SSE) trực tiếp đến RAG AI Server.
	3	AI Chatbot	Thực hiện phân loại ý định (<i>Intent Routing</i>) để định tuyến câu hỏi (Truy vấn SQL qua Rails API hoặc tìm kiếm vector trên ChromaDB).
	4	AI Chatbot	Thu thập các phân đoạn văn bản liên quan và tự động khôi phục ngữ cảnh lớn tương ứng (<i>Parent Context Expansion</i>).
	5	AI Chatbot	Tổng hợp ngữ cảnh vào prompt mẫu và gửi yêu cầu sinh câu trả lời dạng luồng (Streaming) sang LLM (Gemini).
	6	AI Chatbot	Stream liên tục phản hồi văn bản cùng danh mục tài liệu nguồn trích dẫn (<i>sources</i>) trực tiếp về trình duyệt.
	7	System (Client)	Hiển thị trực tiếp câu trả lời theo thời gian thực và đính kèm danh sách tài liệu tham chiếu rõ ràng.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	2a	System (Client)	Mất kết nối mạng hoặc AI Server ngoại tuyến. Hệ thống hiển thị lỗi: "Không thể kết nối với Thư viện AI".
	5a	System (Client)	Lỗi cạn quota hoặc API phản hồi chậm quá thời gian. Trả về thông báo: "AI đang bận xử lý, vui lòng chờ ít phút".
Hậu điều kiện	Cuộc hội thoại được ghi nhận và lưu trữ lâu dài vào bộ nhớ cục bộ (<code>localStorage</code>) của trình duyệt Độc giả.		

2.3.4 Đặc tả use case Phê duyệt yêu cầu mượn sách

Bảng 2.4: Đặc tả use case Phê duyệt yêu cầu mượn sách

Mã usecase	UC004	Tên usecase	Phê duyệt yêu cầu mượn sách
Tác nhân	Thủ thư / Admin		
Tiền điều kiện	Thủ thư đã đăng nhập thành công vào bảng quản lý Admin và có đơn mượn đang "Chờ duyệt" (Pending).		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Thủ thư	Truy cập vào trang quản lý đơn mượn chờ duyệt.
	2	Thủ thư	Nhấn xem chi tiết đơn mượn của độc giả cần kiểm tra.
	3	Thủ thư	Nhấp nút "Phê duyệt" để đồng ý cho mượn sách.
	4	System	Chuyển trạng thái đơn mượn thành "Đang mượn" (Borrowed) và giảm số lượng khả dụng của sách trong kho.
	5	System	Gửi email thông báo tự động cho độc giả về việc đơn mượn được duyệt thành công.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	3a	Thủ thư	Chọn "Từ chối" đơn do phát hiện sách bị hỏng, mất hoặc độc giả quá hạn cũ → Đơn mượn chuyển sang trạng thái "Từ chối" (Rejected).
Hậu điều kiện	Trạng thái đơn mượn được cập nhật thành công, số lượng sách khả dụng trong DB giảm xuống.		

2.3.5 Đặc tả use case Ghi nhận đã trả sách

Bảng 2.5: Đặc tả use case Ghi nhận đã trả sách

Mã usecase	UC005	Tên usecase	Ghi nhận đã trả sách
Tác nhân	Thủ thư / Admin		
Tiền điều kiện	Thủ thư đã đăng nhập vào hệ thống Admin và đơn mượn liên quan đang ở trạng thái "Đang mượn" (Borrowed).		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Thủ thư	Tiếp nhận sách vật lý độc giả đem hoàn trả tại quầy.
	2	Thủ thư	Tìm đơn mượn đang hoạt động của độc giả theo email hoặc mã đơn.
	3	Thủ thư	Kiểm tra số lượng sách, tình trạng vật lý (mất trang, hư hại bìa).
	4	Thủ thư	Nhấn nút "Ghi nhận trả sách" trên hệ thống.
	5	System	Đổi trạng thái đơn mượn từ "Đang mượn" sang "Đã trả" (Returned).
	6	System	Tự động cộng lại số lượng khả dụng của sách vào kho và đưa ra thông báo thành công.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	3a	Thủ thư	Sách bị mất hoặc hư hỏng nghiêm trọng → Thủ thư áp dụng phạt đền theo quy chế trước khi bấm nút xác nhận ghi nhận trả sách trên hệ thống.
Hậu điều kiện	Trạng thái đơn mượn được cập nhật thành Đã trả, số lượng khả dụng của sách được hoàn trả về kho dữ liệu gốc.		

2.3.6 Đặc tả use case Quản lý sách

Bảng 2.6: Đặc tả use case Quản lý sách

Mã usecase	UC006	Tên usecase	Quản lý sách
Tác nhân	Thủ thư / Admin		
Tiền điều kiện	Thủ thư đã đăng nhập thành công vào trang quản lý Admin và mở trang quản trị sách.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Thủ thư	Chọn một hành động muốn thực hiện: "Thêm đầu sách mới", "Cập nhật thông tin sách" hoặc "Xóa đầu sách".
	2	Thủ thư	Điền các thông tin sách (tiêu đề, tác giả, thể loại, NXB, số lượng kho) và tải tệp tin đính kèm (ảnh bìa, tệp PDF gốc).
	3	Thủ thư	Click chọn nút "Lưu sách" hoặc "Xác nhận".
	4	System	Kiểm tra tính hợp lệ của dữ liệu biểu mẫu nhập vào.
	5	System	Cập nhật cơ sở dữ liệu sách, lưu trữ tệp tin tải lên vào storage hệ thống (ActiveStorage).
	6	System	Hiển thị thông báo tác vụ hoàn tất thành công và cập nhật lại danh sách hiển thị sách.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	4a	System	Tệp PDF gốc tải lên vượt quá dung lượng cho phép (>50MB). Hệ thống báo lỗi và giữ nguyên trang soạn thảo.
	5a	System	Thủ thư chọn hành động "Xóa sách" nhưng cuốn sách này đang thuộc về một đơn mượn chưa được trả. Hệ thống chặn tác vụ và báo lỗi ràng buộc.
Hậu điều kiện	Thông tin kho sách thư viện thay đổi và cập nhật chính xác trong cơ sở dữ liệu.		

2.3.7 Đặc tả use case Đăng ký độc giả

Bảng 2.7: Đặc tả use case Đăng ký độc giả

Mã usecase	UC007	Tên usecase	Đăng ký độc giả
Tác nhân	Độc giả		
Tiền điều kiện	Độc giả chưa có tài khoản và đã mở giao diện đăng ký tài khoản.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Độc giả	Nhập thông tin cá nhân: Họ tên, Email, Mật khẩu, Xác thực mật khẩu, Ngày sinh, Giới tính.
	2	Độc giả	Click chọn nút "Đăng ký".
	3	System	Kiểm tra định dạng email hợp lệ, độ dài mật khẩu và tính duy nhất của email trên DB.
	4	System	Tạo bản ghi thông tin người dùng mới với trạng thái "Chưa kích hoạt" (Inactive).
	5	System	Tự động gửi email đính kèm đường link xác nhận tài khoản cho độc giả và hiển thị thông báo.
	6	Độc giả	Truy cập hộp thư email cá nhân và nhấp vào đường link kích hoạt tài khoản.
	7	System	Chuyển trạng thái người dùng thành "Đang hoạt động" (Active) và thông báo thành công.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	3a	System	Email đã được sử dụng hoặc xác nhận mật khẩu không trùng khớp. Hệ thống báo lỗi và yêu cầu chỉnh sửa thông tin.
	6a	System	Liên kết trong email đã hết hạn. Hệ thống yêu cầu độc giả click gửi lại mail kích hoạt mới.
Hậu điều kiện	Tài khoản độc giả chuyển sang hoạt động và có thể đăng nhập vào hệ thống thư viện.		

2.3.8 Đặc tả use case Xem báo cáo phần trăm mượn trả

Bảng 2.8: Đặc tả use case Xem báo cáo phần trăm mượn trả

Mã usecase	UC008	Tên usecase	Xem báo cáo phần trăm mượn trả
Tác nhân	Thủ thư / Admin		
Tiền điều kiện	Thủ thư đã đăng nhập thành công vào trang quản trị Admin.		
Luồng sự kiện chính	STT	Thực hiện bởi	Hành động
	1	Thủ thư	Nhấp chọn trang "Báo cáo & Thống kê" trên bảng điều khiển quản trị.
	2	System	Thực hiện đếm tổng số lượng bản ghi đơn mượn đang có trạng thái "Đang mượn" (Borrowed) và "Đã trả" (Returned).
	3	System	Tính toán tỷ lệ phần trăm tương đối của từng trạng thái đơn mượn.
	4	System	Hiển thị trực quan dữ liệu mượn trả dưới dạng biểu đồ tỉ lệ để thủ thư kiểm duyệt.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	2a	System	Cơ sở dữ liệu chưa có đơn mượn/trả nào được thực hiện. Hệ thống hiển thị biểu đồ trống và hiển thị thông báo "Chưa có dữ liệu giao dịch mượn trả".
Hậu điều kiện	Báo cáo thống kê được hiển thị chính xác dựa trên dữ liệu giao dịch thực tế của thư viện.		

2.4 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng cốt lõi, để đảm bảo hệ thống vận hành hiệu quả và mang lại trải nghiệm tối ưu cho người dùng, hệ thống cần đáp ứng thêm các yêu cầu phi chức năng quan trọng như hiệu năng, tính tin cậy, tính dễ sử dụng, tính dễ bảo trì và tính bảo mật.

Về hiệu năng và tính khả dụng, hệ thống được tối ưu hóa tốc độ phản hồi thông qua việc ứng dụng công nghệ Turbo Frames và Turbo Streams, giúp cập nhật dữ liệu cục bộ mà không cần tải lại toàn bộ trang web. Điều này đặc biệt quan trọng trong các thao tác tương tác liên tục như đánh dấu yêu thích hay viết đánh giá sách.

Về tính bảo mật và phân quyền, hệ thống thiết lập cơ chế kiểm soát truy cập dựa trên vai trò. Thông qua các thư viện tiêu chuẩn như Devise và CanCanCan, mọi dữ liệu cá nhân và quyền hạn thực thi chức năng được bảo vệ, ngăn chặn các truy cập trái phép từ người dùng vào trang quản trị.

Về độ tin cậy và khả năng bảo trì, tính toàn vẹn dữ liệu được đặt lên hàng đầu thông qua việc sử dụng các giao dịch (Database Transactions) cho các quy trình phức tạp như mượn trả và cập nhật kho hàng. Hệ thống có khả năng tự động xử lý

và ghi log các tác vụ chạy ngầm như cập nhật yêu cầu quá hạn, giúp quản trị viên dễ dàng truy vết và khắc phục sự cố. Mã nguồn được xây dựng trên kiến trúc hướng đối tượng linh hoạt, sử dụng quan hệ đa hình, tạo điều kiện thuận lợi cho việc nâng cấp và mở rộng các tính năng mới trong tương lai.

2.5 Tổng kết

Tổng kết lại, chương này đã cung cấp cái nhìn toàn diện về bối cảnh thực tế và nhu cầu người dùng đối với hệ thống quản lý thư viện. Thông qua việc phân tích yêu cầu từ hai nhóm đối tượng chính và đánh giá xu hướng công nghệ, chương đã xác định rõ các chức năng trọng tâm cần triển khai. Bên cạnh đó, các yêu cầu chức năng và phi chức năng cũng đã được đặc tả cụ thể, làm nền tảng định hướng cho quá trình thiết kế kiến trúc hệ thống, triển khai giải pháp kỹ thuật và kiểm thử phần mềm trong các chương tiếp theo.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Trong chương này, đề án trình bày tổng quan, ưu điểm, nhược điểm và ứng dụng của các công nghệ, nền tảng và công cụ được sử dụng để xây dựng hệ thống quản lý thư viện thông minh tích hợp trợ lý ảo tìm kiếm ngữ cảnh. Các công nghệ này được lựa chọn dựa trên yêu cầu thực tế đã phân tích ở Chương 2, bao gồm cả yêu cầu chức năng lẫn phi chức năng, đồng thời cân nhắc đến tính khả thi, hiệu quả và mức độ phù hợp với quy mô của hệ thống.

3.1 HTML, CSS, JavaScript và Bootstrap

Đây là giải pháp phát triển giao diện truyền thống nhưng mang lại sự tối ưu tuyệt đối về tốc độ tải trang, tính tương thích cao và không đòi hỏi các bước biên dịch phức tạp.

a, Ưu điểm

HTML, CSS: Cung cấp cấu trúc trang web chuẩn hóa, hiệu năng render giao diện nhanh trực tiếp trên trình duyệt mà không cần qua Virtual DOM.

JavaScript thuần (Vanilla JS): Xử lý các sự kiện (Event handling), gọi API bất đồng bộ (Fetch API/Axios) và cập nhật DOM một cách trực tiếp, nhẹ nhàng, không tốn tài nguyên bộ nhớ.

Bootstrap: Thư viện giao diện (UI Framework) giúp xây dựng giao diện phản hồi (Responsive Design) nhanh đảm bảo trang web hiển thị đẹp mắt.

b, Nhược điểm

Khi giao diện phình to với nhiều logic phức tạp (như hộp thoại chat thời gian thực), việc quản lý mã nguồn bằng JavaScript thuần đòi hỏi phải tổ chức tường minh để tránh rối mã.

Ngoài ra, phải tự thao tác thủ công với DOM (Document Object Model) khi có dữ liệu mới từ API trả về, thay vì cơ chế tự động cập nhật như các framework hiện đại.

c, Ứng dụng trong đề án

Bộ công nghệ này được sử dụng để xây dựng toàn bộ giao diện người dùng (Frontend) của hệ thống. Bootstrap giúp thiết kế nhanh các màn hình quản lý sách, phiếu mượn của thủ thư. Trong khi đó, JavaScript thuần chịu trách nhiệm bắt sự kiện cuộn trang, gửi tin nhắn của độc giả và đổ dữ liệu phản hồi từ Trợ lý ảo RAG lên khung chat theo thời gian thực.

d, Lý do lựa chọn

Trong quá trình đánh giá phương án xây dựng giao diện, một số giải pháp thể hệ mới đã được cân nhắc:

- React.js / Vue.js: Cung cấp cơ chế quản lý giao diện theo component rất mạnh mẽ. Tuy nhiên, các framework này yêu cầu thời gian thiết lập môi trường phức tạp (Node v.v.), dung lượng file build nặng và tạo ra một lớp trừu tượng không cần thiết cho một hệ thống web tích hợp API độc lập.

Từ đó, việc kết hợp HTML, CSS, JavaScript thuần và Bootstrap được lựa chọn nhờ tính gọn nhẹ, dễ tùy biến giao diện trực tiếp, tốc độ phản hồi nhanh và phù hợp để tích hợp mượt mà với các file template view của hệ thống backend chính.

3.2 Ruby on Rails

Ruby on Rails (RoR) là một framework phát triển ứng dụng web nguồn mở viết bằng ngôn ngữ Ruby, tuân theo kiến trúc MVC (Model-View-Controller) và triết lý "Convention over Configuration" (Quy ước trên cấu hình).

a, Ưu điểm

- Tốc độ phát triển cực nhanh nhờ các công cụ sinh mã tự động (Scaffolding) và thư viện phong phú (Gems).
- Quản lý dữ liệu vật lý và đa phương tiện xuất sắc thông qua ActiveStorage tích hợp sẵn.
- Tính bảo mật mặc định cao, tự động chống các lỗi như SQL Injection, XSS và CSRF.
- Hỗ trợ Rake Task mạnh mẽ để xử lý các tác vụ nền và đồng bộ dữ liệu tự động.

b, Nhược điểm

- Tốc độ xử lý thô và hiệu năng tính toán đa luồng thấp hơn so với Node.js hoặc Go.
- Tiêu tốn nhiều tài nguyên bộ nhớ (RAM) hơn trong quá trình vận hành hệ thống.

c, Ứng dụng trong đồ án

Ruby on Rails đóng vai trò là nền tảng backend chính của hệ thống. Framework này chịu trách nhiệm xử lý toàn bộ logic nghiệp vụ cốt lõi bao gồm: quản lý sách, điều phối quy trình mượn trả trực tuyến 5 bước, kiểm soát vi phạm và cung cấp hệ thống API endpoint chuẩn hóa dữ liệu đầu vào cho trợ lý ảo.

d, Lý do lựa chọn

Trong quá trình khảo sát công nghệ backend, một số công nghệ khác đã được xem xét:

- Node.js và Express.js: Xử lý bất đồng bộ tốt, tốc độ nhanh. Tuy nhiên, thiếu các cấu trúc quy ước chặt chẽ, phải tự cấu hình thủ công nhiều thành phần như ORM, phân quyền, quản lý tệp tin khiến thời gian triển khai nghiệp vụ phức tạp bị kéo dài.
- Spring Boot (Java): Hiệu suất cao, mạnh mẽ. Tuy nhiên, cấu hình ban đầu quá phức tạp và rườm rà, thời gian viết code dài, không tối ưu cho tốc độ hoàn thiện đồ án trong thời gian hạn chế.

Từ đó, Ruby on Rails được lựa chọn vì sự toàn diện, cung cấp sẵn các giải pháp quản lý tệp tin (ActiveStorage) và cơ chế di chuyển dữ liệu (Migration) rất mạnh mẽ, giúp đẩy nhanh tiến độ xây dựng hệ thống quản lý nội bộ.

3.3 PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở mạnh mẽ và tiên tiến, nổi tiếng với độ tin cậy cao và khả năng xử lý dữ liệu phức tạp. Nó sử dụng ngôn ngữ truy vấn SQL tiêu chuẩn kết hợp với các tính năng mở rộng hiện đại để quản lý dữ liệu một cách tối ưu.

a, Ưu điểm

- Hiệu suất tối ưu với các truy vấn phức tạp và các phép liên kết bảng (Join) quy mô lớn.
- Hỗ trợ tuyệt vời cho kiểu dữ liệu JSON/JSONB phi cấu trúc, cho phép lưu trữ Metadata linh hoạt.
- Khả năng mở rộng mạnh mẽ, hỗ trợ tốt các tính năng tìm kiếm văn bản nâng cao (Full-text search).
- Tuân thủ nghiêm ngặt các tính chất ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo an toàn giao dịch tài chính hoặc mượn trả sách.

b, Nhược điểm

- Cấu hình tối ưu hiệu năng ban đầu phức tạp hơn so với MySQL.
- Tốc độ thực hiện các câu lệnh đọc/ghi đơn giản (CRUD) có thể chậm hơn một chút so với các hệ cơ sở dữ liệu dung lượng nhẹ

c, Ứng dụng trong đồ án

PostgreSQL được sử dụng để lưu trữ toàn bộ dữ liệu có cấu trúc của hệ thống thư viện bao gồm thông tin đầu sách, thông tin độc giả, các bản ghi mượn trả, lịch sử xử phạt vi phạm và các thông tin cấu hình hệ thống.

d, Lý do lựa chọn

Một số lựa chọn phổ biến đã từng được xem xét và cân nhắc khác như:

- MySQL: Hoạt động tốt với các thao tác CRUD cơ bản, nhưng khả năng hỗ trợ kiểu dữ liệu JSON và tìm kiếm toàn văn kém linh hoạt hơn PostgreSQL.
- MongoDB: Phù hợp với dữ liệu phi cấu trúc, nhưng không đảm bảo tính toàn vẹn dữ liệu khi xử lý các mối quan hệ chéo phức tạp như quy trình mượn trả nhiều bước và xử lý chế tài phạt của thư viện.

Vì dữ liệu hệ thống đòi hỏi tính nhất quán tuyệt đối, đồng thời cần lưu trữ dữ liệu Metadata linh hoạt dưới dạng JSON để đồng bộ sang công cụ AI, PostgreSQL là giải pháp tối ưu và an toàn nhất.

3.4 Python FastAPI

FastAPI là một web framework hiện đại, có hiệu suất cao dùng để xây dựng các API bằng Python, dựa trên các tiêu chuẩn ASGI và hỗ trợ khai báo kiểu dữ liệu (Type hints) tường minh.

a, Ưu điểm

- Tốc độ xử lý vượt trội, ngang ngửa với NodeJS và Go nhờ kiến trúc bất đồng bộ (async/await).
- Tự động kiểm tra tính hợp lệ của dữ liệu đầu vào (Data validation) qua Pydantic và tự động sinh tài liệu API (Swagger UI).
- Hệ sinh thái Python phong phú, tích hợp hoàn hảo với các thư viện Trí tuệ nhân tạo và Xử lý ngôn ngữ tự nhiên.

b, Nhược điểm

- Là một micro-framework nên cần phải tự tổ chức cấu trúc mã nguồn dự án thủ công từ đầu.
- Đòi hỏi lập trình viên phải nắm vững kiến trúc lập trình bất đồng bộ để tránh xung đột tài nguyên.

c, Ứng dụng trong đồ án

FastAPI được triển khai như một dịch vụ AI độc lập (AI Microservice). Dịch vụ này tiếp nhận các yêu cầu truy vấn ngôn ngữ tự nhiên từ backend chính, vận hành

chuỗi xử lý RAG và giao tiếp với cơ sở dữ liệu vector.

d, Lý do lựa chọn

Trong quá trình khảo sát, một số công nghệ khác đã được xem xét:

- Flask (Python): Gọn nhẹ nhưng là đồng bộ truyền thống, hiệu suất xử lý luồng yêu cầu đồng thời (Concurrency) kém hơn nhiều và không tự động sinh tài liệu Swagger như FastAPI.
- Django REST Framework: Toàn diện nhưng quá nặng nề và thừa thãi cho một dịch vụ chỉ chuyên trách xử lý các thuật toán và pipeline về AI.

FastAPI mang lại sự cân bằng hoàn hảo giữa hiệu suất thực thi bất đồng bộ cao và khả năng tích hợp mượt mà với các công cụ AI trong hệ sinh thái Python.

3.5 LangChain

LangChain là một framework mã nguồn mở mạnh mẽ được thiết kế để đơn giản hóa việc xây dựng các ứng dụng tích hợp Mô hình ngôn ngữ lớn (LLM).

a, Ưu điểm

- Cung cấp các module chuẩn hóa để kết nối dữ liệu, quản lý Prompt Template và tạo chuỗi liên kết (Chains).
- Hỗ trợ sẵn các công cụ trích xuất, chia nhỏ văn bản (Text Splitters) và quản lý bộ nhớ hội thoại (Memory).
- Dễ dàng chuyển đổi linh hoạt giữa các mô hình LLM hoặc Vector Database khác nhau mà không cần sửa đổi cấu trúc lõi.

b, Nhược điểm

- Tài liệu cập nhật liên tục do framework thay đổi phiên bản khá nhanh, dễ gây lỗi bất tương thích cú pháp cũ.
- Tạo ra một lớp trừu tượng hóa sâu, đôi khi gây khó khăn cho việc tinh chỉnh sâu các tham số thuật toán cơ bản.

c, Ứng dụng trong đồ án

LangChain được sử dụng để thiết lập Pipeline cho kiến trúc RAG bao gồm: Đọc tài liệu PDF sách/nội quy, thực hiện chia nhỏ văn bản (Chunking), chuyển đổi sang vector định dạng (Embeddings), và thiết lập chuỗi RetrievalQA để kết hợp ngữ cảnh với câu hỏi của người dùng.

d, Lý do lựa chọn

Nếu không sử dụng LangChain, việc tự lập trình thủ công các kết nối API, tự viết thuật toán chia nhỏ tài liệu và quản lý lịch sử trò chuyện sẽ tốn rất nhiều thời

gian và dễ phát sinh lỗi hệ thống. LangChain được chọn vì là tiêu chuẩn công nghệ hàng đầu hiện nay, giúp chuẩn hóa toàn bộ quy trình xử lý dữ liệu của trợ lý ảo.

3.6 ChromaDB

ChromaDB là một cơ sở dữ liệu vector (Vector Database) mã nguồn mở chuyên dụng, được thiết kế để lưu trữ và truy vấn các vector nhúng (Embeddings) một cách nhanh chóng và hiệu quả.

a, Ưu điểm

- Nhẹ, dễ cài đặt và có thể chạy trực tiếp trên bộ nhớ (In-memory) hoặc lưu trữ tập tin cục bộ, rất phù hợp cho môi trường thử nghiệm đồ án.
- Hỗ trợ tìm kiếm tương đồng vector (Similarity Search) với tốc độ cao dựa trên các thuật toán khoảng cách như Cosine hay L2.
- Tích hợp bộ lọc Metadata mạnh mẽ, cho phép lọc dữ liệu theo thẻ thông tin trước hoặc trong khi tìm kiếm.

b, Nhược điểm

- Khả năng quản lý cụm và phân tán (Clustering) trên môi trường production quy mô lớn chưa mạnh mẽ bằng các giải pháp như Pinecone hay Milvus.
- Giao diện quản lý trực quan đi kèm còn hạn chế.

c, Ứng dụng trong đồ án

ChromaDB đóng vai trò là kho lưu trữ tri thức cho trợ lý ảo. Nó lưu trữ toàn bộ các đoạn văn bản đã được vector hóa từ sách truyện và sổ tay nội quy thư viện, sẵn sàng cung cấp các đoạn ngữ cảnh có độ tương đồng cao nhất khi nhận được câu hỏi từ độc giả.

d, Lý do lựa chọn

So với Pinecone (đòi hỏi môi trường đám mây và kết nối Internet ổn định) hay Milvus (quá nặng nề, phức tạp trong việc cài đặt qua Docker), ChromaDB là sự lựa chọn tối ưu nhất cho đồ án nhờ tính gọn nhẹ, khả năng vận hành độc lập trên local và tốc độ truy vấn vượt trội với tập dữ liệu thư viện.

3.7 Gemini LLM

Gemini là thể hệ mô hình ngôn ngữ lớn (LLM) đa phương thức tiên tiến do Google phát triển, có khả năng hiểu, xử lý và tạo lập văn bản với độ chính xác và tư duy ngữ cảnh vượt trội.

a, Ưu điểm

- Khả năng hiểu tiếng Việt tự nhiên, lưu loát và chuẩn xác về mặt ngữ nghĩa.

- Hỗ trợ cửa sổ ngữ cảnh (Context Window) rất lớn, cho phép nạp nhiều đoạn trích tài liệu cùng lúc.
- Chi phí sử dụng qua API tối ưu, tốc độ phản hồi (Latency) nhanh và hoạt động ổn định.

b, Nhược điểm

- Hoàn toàn phụ thuộc vào kết nối API bên thứ ba, không thể chạy offline cục bộ trên máy trạm.
- Vẫn có tỉ lệ nhỏ xảy ra hiện tượng "ảo giác" (Hallucination) nếu prompt không được kiểm soát chặt chẽ.

c, Ứng dụng trong đồ án

Gemini LLM đóng vai trò là nơi tổng hợp thông tin của trợ lý ảo. Sau khi nhận được các đoạn văn bản ngữ cảnh chuẩn xác từ ChromaDB, Gemini sẽ xử lý, phân tích và biên soạn câu trả lời cuối cùng sao cho tự nhiên, đúng trọng tâm và gửi lại cho người dùng.

d, Lý do lựa chọn

Qua thử nghiệm so sánh với GPT-3.5/GPT-4 (chi phí cao, thủ tục đăng ký API phức tạp) hay các mô hình mã nguồn mở như Llama 3 (đòi hỏi phần cứng GPU máy trạm cực kỳ mạnh mẽ để deploy), Gemini LLM là sự lựa chọn hoàn hảo nhất nhờ khả năng xử lý tiếng Việt, miễn phí hạn mức thử nghiệm phát triển và tích hợp dễ dàng qua thư viện Google GenAI.

3.8 Tổng kết

Trong chương này, các công nghệ và mô hình chính được lựa chọn cho hệ thống đã được trình bày và phân tích kỹ lưỡng. Ở phía giao diện (Frontend), HTML, CSS, JavaScript và Bootstrap, được chọn nhờ khả năng xây dựng màn hình tương tác, tạo hộp thoại chat mượt mà. Hệ thống quản lý Backend sử dụng framework Ruby on Rails kết hợp cơ sở dữ liệu PostgreSQL nhằm bảo mật và xử lý chuẩn hóa nghiệp vụ mượn trả, đồng bộ tài liệu qua ActiveStorage.

Đối với phần Trợ lý ảo, một kiến trúc RAG hoàn chỉnh được xây dựng biệt lập bằng Python FastAPI kết hợp framework điều phối LangChain, cơ sở dữ liệu vector ChromaDB và mô hình ngôn ngữ lớn Gemini LLM. Đặc biệt, hệ thống tối ưu hóa bằng cơ chế Hybrid Search (Vector Search kết hợp từ khóa BM25) và kiến trúc tách biệt Ingest module cho dữ liệu động (Sách truyện) và dữ liệu tĩnh (Sổ tay hướng dẫn và Nội quy thư viện). Sự kết hợp đồng bộ này đảm bảo cho hệ thống đạt hiệu năng vận hành ổn định, giao diện thân thiện và trợ lý ảo đạt độ chính xác tối đa trong bối cảnh ứng dụng thực tiễn.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Khép lại Chương 3 với phần giới thiệu về các công nghệ nền tảng và các công cụ mô hình hóa trí tuệ nhân tạo được sử dụng, Chương 4 sẽ trình bày toàn bộ quá trình thiết kế và hiện thực hóa hệ thống quản lý thư viện thông minh tích hợp trợ lý ảo tìm kiếm ngữ cảnh, từ kiến trúc tổng thể đến triển khai chi tiết. Mở đầu là việc lựa chọn và áp dụng kiến trúc phần mềm phù hợp nhằm đảm bảo tính phân tầng, khả năng xử lý bất đồng bộ, mở rộng và bảo trì. Tiếp theo, chương trình bày các mô hình thiết kế UML như biểu đồ gói, biểu đồ lớp và biểu đồ trình tự, làm rõ cấu trúc hệ thống và mối quan hệ giữa các thành phần nghiệp vụ và dịch vụ AI. Phần tiếp theo mô tả quá trình xây dựng giao diện người dùng trực quan với bộ ba HTML, CSS, JavaScript kết hợp Bootstrap, thiết kế cơ sở dữ liệu quan hệ PostgreSQL, xây dựng các module xử lý RAG độc lập bằng FastAPI và phát triển chức năng thông qua các công cụ và công nghệ đã lựa chọn. Cuối cùng, là kiểm thử nhằm đánh giá hiệu năng, độ ổn định và mức độ đáp ứng yêu cầu thực tế của cả độc giả lẫn ban quản lý thư viện.

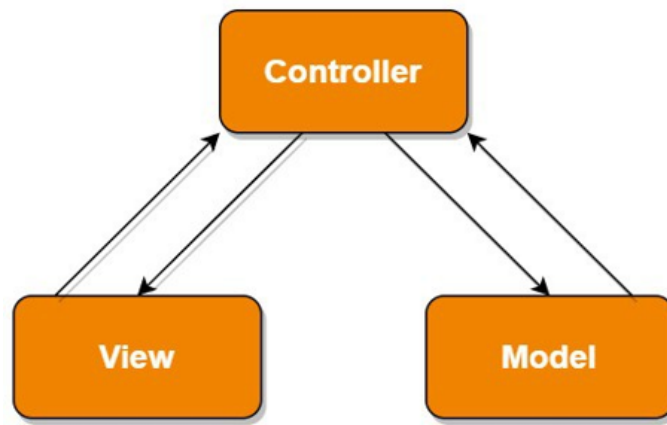
4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được thiết kế dựa trên sự kết hợp giữa Kiến trúc MVC (Model-View-Controller) truyền thống của framework Ruby on Rails và Kiến trúc hướng dịch vụ (SOA/Microservices) tích hợp thông qua HTTP API với dịch vụ AI Chatbot (FastAPI - Python). Sự phối hợp này nhằm giải phóng Web Server chính khỏi các tác vụ tính toán nặng nề, đồng thời đảm bảo hệ thống web được tổ chức chặt chẽ, dễ mở rộng và bảo trì.

a, Kiến trúc MVC (Model-View-Controller) tại Web App chính

Mô hình MVC phân rã ứng dụng Web Rails thành ba thành phần cốt lõi có sự độc lập cao về mặt nhiệm vụ:



Hình 4.1: Mô hình kiến trúc MVC

- **Model (Thành phần dữ liệu và logic nghiệp vụ):** Đảm nhận vai trò quản lý toàn bộ dữ liệu hệ thống, thực hiện kiểm tra tính hợp lệ (validations), thiết lập quan hệ đối tượng và tương tác trực tiếp với cơ sở dữ liệu quan hệ (PostgreSQL) thông qua cơ chế ActiveRecord ORM. Trong mã nguồn cài đặt thực tế của hệ thống, thành phần Model (M) bao gồm các lớp lớp cụ thể sau:
 - Lớp cơ sở trừu tượng: `ApplicationRecord` kết nối cấu trúc hạ tầng cơ sở dữ liệu.
 - Các thực thể thông tin cốt lõi: Lớp `User` (Độc giả/Thủ thư), lớp `Book` (Thông tin sách), lớp `Author` (Tác giả), lớp `Publisher` (Nhà xuất bản), và lớp `Category` (Thể loại).
 - Các thực thể giao dịch: Lớp `BorrowRequest` (Đơn mượn sách) và lớp `BorrowRequestItem` (Chi tiết từng cuốn sách trong đơn mượn).
 - Các thực thể tương tác mở rộng: Lớp `Review` (Đánh giá sách), lớp `Favorite` (Sách/Tác giả yêu thích).
 - Thực thể phục vụ dịch vụ AI: Lớp `BookSection` lưu trữ các phân đoạn nội dung sách đã được băm nhỏ phục vụ thuật toán vector hóa của RAG.
- **View (Thành phần giao diện):** Chịu trách nhiệm kết xuất giao diện trực quan và thu nhận phản hồi, thao tác từ người dùng. View trong hệ thống được cấu thành bởi các tệp tin giao diện động `ActionView (.html.erb)` chạy trên máy chủ Web và các thành phần mã nguồn giao diện ở phía Client.
- **Controller (Thành phần điều khiển):** Đóng vai trò là bộ phận điều phối trung gian, tiếp nhận các truy vấn HTTP gửi tới từ trình duyệt, kiểm tra xác thực quyền hạn thông qua lớp phân quyền `Ability (CanCanCan)`, điều phối các Model liên quan thực hiện đọc/ghi dữ liệu, và chỉ định giao diện View thích hợp để hiển thị kết quả cho người dùng. Các lớp Controller chính được

triển khai gồm:

- Nhóm nghiệp vụ độc giả: `BooksController`, `BorrowRequestController`, `ChatbotController`, `UsersController`, `OmniauthCallbacksController` (Xử lý xác thực Google).
- Nhóm nghiệp vụ thủ thư: Bộ điều khiển cơ sở
`Admin::ApplicationController` và các bộ điều khiển kế thừa
gồm `Admin::BooksController`,
`Admin::BorrowRequestsController`,
`Admin::UsersController`,
`Admin::ReportsController` (Xử lý thống kê biểu đồ mượn trả).

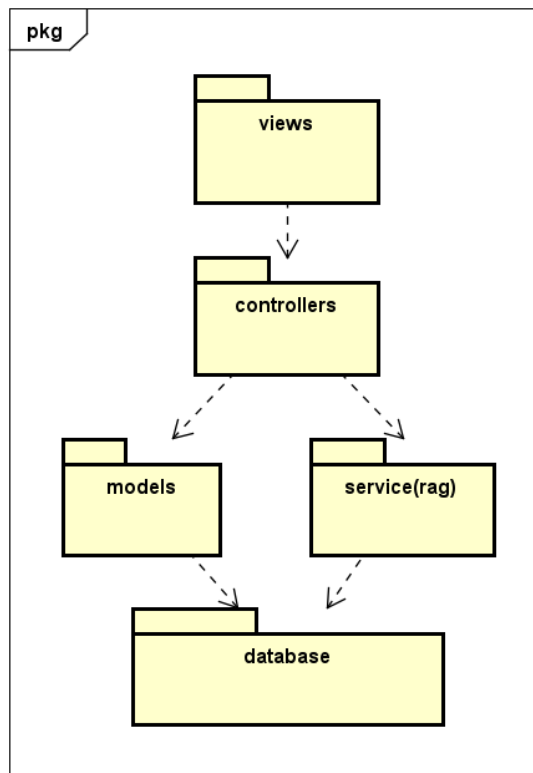
b, Kiến trúc dịch vụ AI Chatbot (FastAPI RAG Service)

Để đáp ứng nhu cầu hỏi đáp thông minh dựa trên nội dung tài liệu lưu giữ trong thư viện, hệ thống được mở rộng bằng một dịch vụ AI chạy hoàn toàn độc lập bằng ngôn ngữ Python sử dụng thư viện FastAPI. Logic xử lý được phân rã như sau:

- Lớp kết nối tích hợp trong ứng dụng Rails Khi nhận được câu hỏi từ độc giả, `ChatbotController` chuyển tiếp dữ liệu để thực hiện yêu cầu HTTP API sang máy chủ AI.
- **FastAPI** chịu trách nhiệm cung cấp các endpoints nhận thông điệp, kiểm tra tính hợp lệ của tham số đầu vào bằng lớp kiểm duyệt `IntentSchema`.
- **RAG Engine**: Lớp xử lý nghiệp vụ AI `RAGEngine` thực hiện quy trình định tuyến ý định câu hỏi bằng các cấu trúc chỉ thị

(`RouterPrompts`, `ExecutionPrompts`), truy vấn tìm kiếm ngữ cảnh liên quan nhất từ cơ sở dữ liệu vector `ChromaDB` và gọi API của LLM Gemini để tổng hợp câu trả lời gửi ngược lại cho Rails Web App.

4.1.2 Thiết kế tổng quan



Hình 4.2: Biểu đồ gói tổng quan

a, Gói views

Mục đích: Đây là gói chịu trách nhiệm chính cho việc hiển thị giao diện người dùng và tiếp nhận các tương tác từ người dùng như độc giả tìm kiếm sách, gửi yêu cầu mượn sách, xem lịch sử mượn, hay quản trị viên cập nhật danh mục, quản lý người dùng,...

Nhiệm vụ:

- Hiển thị dữ liệu được lấy từ controllers một cách trực quan, dễ hiểu.
- Đảm bảo trải nghiệm người dùng nhất quán, thân thiện và dễ sử dụng trên các màn hình công khai cũng như trang quản trị.
- Gửi các yêu cầu (HTTP request) về controllers để xử lý: như đăng nhập, tạo phiếu mượn, đánh giá sách,...
- Giao tiếp một chiều với controllers, không phụ thuộc trực tiếp vào models, database hay dịch vụ AI RAG.

b, Gói controllers

Mục đích: Đây là gói trung tâm xử lý logic điều hướng, đóng vai trò là cầu

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

nối giữa giao diện người dùng (`views`) và các tài nguyên nghiệp vụ của hệ thống (`models` hoặc dịch vụ `service(rag)`). Toàn bộ các thao tác người dùng gửi từ `views` đều được tiếp nhận và xử lý tại đây.

Nhiệm vụ:

- Tiếp nhận các yêu cầu từ `views`, thực hiện phân tích tham số (`params`) và điều hướng luồng xử lý.
- Gọi các logic nghiệp vụ tương ứng từ `models` (cho các chức năng quản lý thông thường) hoặc dịch vụ `service(rag)` (cho các câu hỏi chatbot).
- Thực hiện bảo mật, kiểm tra xác thực và phân quyền truy cập hệ thống (như phân hệ Admin và phân hệ người dùng công khai).
- Trả kết quả xử lý (`render views` hoặc `redirect`) về cho gói `views` hiển thị.
- Giữ cho gói `views` không cần biết cách xử lý nghiệp vụ nội bộ hay cấu trúc lưu trữ cơ sở dữ liệu.

c, Gói `models`

Mục đích: Đây là gói quản lý các thực thể (Entities) và quy tắc nghiệp vụ cốt lõi (Business Logic) của hệ thống quản lý thư viện, đồng thời chịu trách nhiệm giao tiếp với cơ sở dữ liệu thông qua cơ chế ánh xạ đối tượng (Object-Relational Mapping - ORM).

Nhiệm vụ:

- Định nghĩa cấu trúc dữ liệu và các mối quan hệ thực thể trong thư viện (Sách, Tác giả, Nhà xuất bản, Người dùng, Phiếu mượn,...).
- Thực hiện các ràng buộc kiểm tra tính hợp lệ của dữ liệu (`validations`) và các hành động tự động (`callbacks`) trước khi ghi dữ liệu.
- Cung cấp các API truy vấn, đọc/ghi dữ liệu bền vững để controllers sử dụng thông qua ORM Active Record.
- Tách biệt với `views`, hoạt động độc lập và chỉ giao tiếp gián tiếp thông qua controllers.

d, Gói `service(rag)`

Mục đích: Đây là gói dịch vụ AI nâng cao chịu trách nhiệm xử lý các tác vụ hỏi đáp thông minh dựa trên tri thức thư viện bằng phương pháp RAG (Retrieval-Augmented Generation), hỗ trợ chatbot trả lời độc giả một cách chính xác.

Nhiệm vụ:

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

- Định cấu hình, đọc/ghi các tham số liên quan đến Vector Database và mô hình ngôn ngữ lớn (Gemini LLM).
- Tiếp nhận câu hỏi từ controller, thực hiện phân loại ý định câu hỏi (Intent Routing) của độc giả.
- Truy xuất thông tin/ngữ cảnh liên quan từ cơ sở dữ liệu vector dựa trên độ tương đồng ngữ nghĩa.
- Xây dựng prompt tối ưu và kết hợp với LLM để sinh câu trả lời tự nhiên, chính xác dựa trên tài liệu thư viện.
- Cung cấp dịch vụ nạp và đồng bộ dữ liệu sách sang không gian vector.

e, Gói database

Mục đích: Đây là gói quản lý việc lưu trữ dữ liệu vật lý cho toàn hệ thống, đảm bảo lưu trữ an toàn và truy xuất hiệu quả cho các thành phần khác.

Nhiệm vụ:

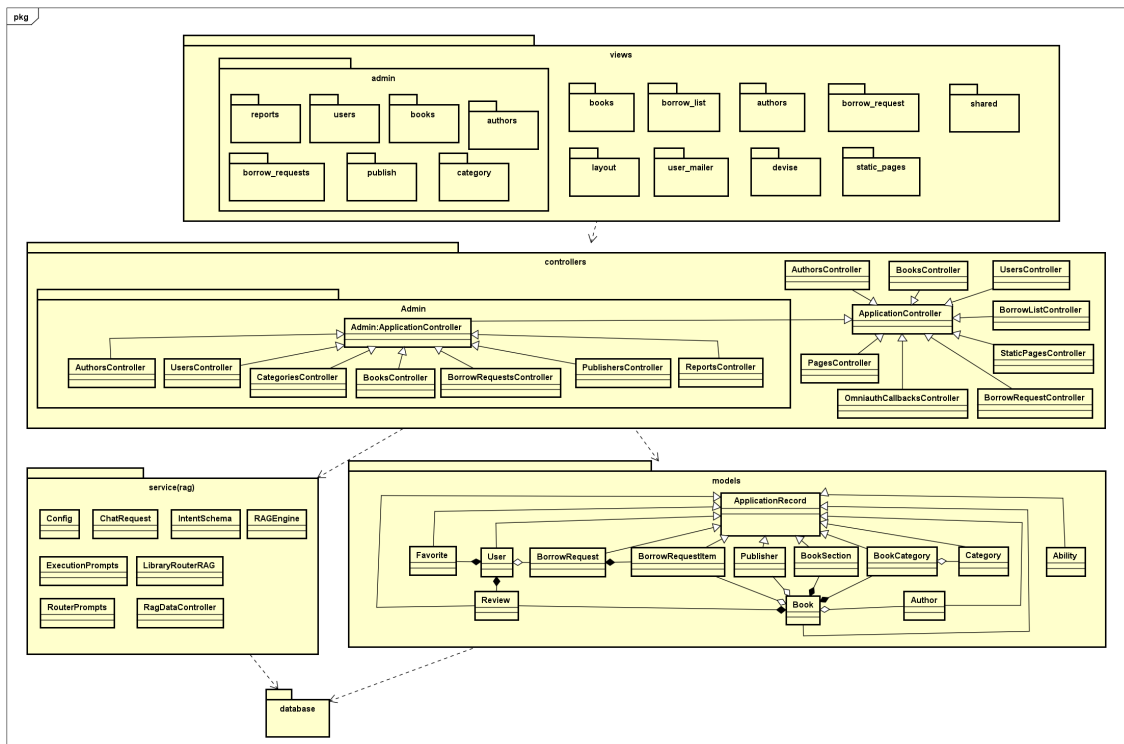
- Lưu trữ dữ liệu có cấu trúc (thông tin người dùng, sách, tác giả, nhà xuất bản, phiếu mượn, đánh giá...) trong cơ sở dữ liệu quan hệ (MySQL).
- Lưu trữ dữ liệu phi cấu trúc dạng vector nhúng (các phân đoạn sách) trong Vector Database (ChromaDB) để phục vụ tìm kiếm ngữ nghĩa.
- Đảm bảo tính nhất quán, toàn vẹn và an toàn dữ liệu.
- Tách biệt với views và controllers, chỉ tương tác trực tiếp với các gói ở tầng nghiệp vụ (models và service(rag)).

f, Mối quan hệ giữa các gói

- `views` → `controllers`: Gói `views` phụ thuộc vào `controllers` để gửi các yêu cầu xử lý từ giao diện và nhận lại kết quả hiển thị.
- `controllers` → `models`: Gói `controllers` phụ thuộc vào `models` để thực hiện các thao tác quản lý dữ liệu thư viện như thêm sách, cập nhật thông tin tác giả, phê duyệt phiếu mượn,...
- `controllers` → `service(rag)`: Gói `controllers` phụ thuộc vào `service(rag)` khi độc giả gửi câu hỏi trò chuyện, nhằm định tuyến câu hỏi và gọi dịch vụ RAG để sinh câu trả lời của AI.
- `models` → `database`: Gói `models` phụ thuộc vào `database` để lưu trữ dữ liệu thực thể xuống cơ sở dữ liệu quan hệ MySQL thông qua ORM Active Record.

- `service(rag) → database`: Gói `service(rag)` phụ thuộc vào `database` để truy xuất các đoạn văn bản tương đồng lưu trong Vector Database (ChromaDB) phục vụ cho quá trình sinh câu trả lời.

4.1.3 Thiết kế chi tiết gói



Hình 4.3: Biểu đồ gói chi tiết

Hệ thống được chia thành các gói chính: views, controllers, models, service(rag) và database, tuân thủ kiến trúc phân tầng rõ ràng theo mô hình MVC kết hợp với dịch vụ AI độc lập:

a, Views

- **admin:** Chứa nhóm giao diện phục vụ riêng cho công tác quản lý của thủ thư/admin, bao gồm quản trị sách (books), người dùng (users), tác giả (authors), yêu cầu mượn (borrow_requests), nhà xuất bản (publish), chuyên mục (category) và báo cáo thống kê (reports).
- **books, borrow_list, authors, borrow_request:** Chứa các giao diện công khai cho độc giả thực hiện tra cứu sách, xem thông tin tác giả, gửi yêu cầu mượn sách và theo dõi lịch sử mượn.
- **shared:** Chứa các thành phần giao diện dùng chung như thanh thông báo, thông điệp lỗi, hoặc thanh điều hướng phụ.
- **layout:** Định nghĩa khung bố cục trang web chung (header, sidebar) giúp giao diện nhất quán.
- **user_mailer:** Chứa các mẫu giao diện email tự động gửi cho người dùng (email thông báo duyệt mượn sách, email nhắc trả sách).

- **devise:** Chứa hệ thống giao diện phục vụ việc đăng ký, đăng nhập và khôi phục mật khẩu.

b, Controllers

- **ApplicationController:** Lớp điều khiển cơ sở xử lý bảo mật, các ngoại lệ và cấu hình dùng chung của toàn hệ thống.
- **Admin (Admin::ApplicationController):** chứa bộ điều khiển dùng riêng cho quản trị viên, kế thừa từ ApplicationController chung và tích hợp bộ lọc kiểm tra quyền admin.

- **Các controller nghiệp vụ admin:** Kế thừa từ

`Admin::ApplicationController`, thực hiện các thao tác quản lý dữ liệu bao gồm: `AuthorsController`, `UsersController`, `CategoriesController`, `BooksController`, `BorrowRequestsController`, `PublishersController`, và `ReportsController`.

- **Các controller nghiệp vụ công khai:** Kế thừa trực tiếp từ

`ApplicationController` phục vụ người dùng thông thường, bao gồm: `AuthorsController`, `BooksController`, `UsersController`, `BorrowListController`, `StaticPagesController`, `PagesController`, `OmniauthCallbacksController`, và `BorrowRequestController`.

c, Models

- **ApplicationRecord:** Lớp cơ sở kết nối trực tiếp với ORM Active Record của Rails, cung cấp các phương thức CRUD và truy vấn CSDL quan hệ.
- **User:** Lưu trữ thông tin người dùng, có quan hệ sở hữu các phiếu mượn (`BorrowRequest`), danh sách yêu thích (`Favorite`) và các lượt đánh giá sách (`Review`).
- **BorrowRequest & BorrowRequestItem:** Lưu trữ phiếu mượn sách, trong đó `BorrowRequest` có quan hệ hợp thành (`Composition`) với các dòng chi tiết sách được mượn `BorrowRequestItem`.
- **Book:** Thực thể trung tâm lưu thông tin sách, liên kết với tác giả (`Author`), nhà xuất bản (`Publisher`), có quan hệ hợp thành với các chương sách (`BookSection`) và danh mục thể loại qua bảng trung gian `BookCategory`.
- **BookSection:** Lưu trữ phân đoạn nội dung sách nhằm phục vụ cho tác vụ cắt nhỏ văn bản.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

- **Category & BookCategory:** Thể hiện phân loại sách và mối quan hệ nhiều-nhiều giữa Sách và Thể loại.
- **Ability:** Định nghĩa các quyền hạn cụ thể cho từng vai trò người dùng (User Role).

d, Service_RAG

- **Config:** Quản lý tập trung cấu hình hệ thống bao gồm: các biến môi trường, khóa API (Gemini), tham số tìm kiếm lai (Hybrid Search) và đường dẫn thư mục Vector DB cục bộ.
- **ChatRequest:** Lớp Pydantic đóng gói dữ liệu đầu vào của yêu cầu trò chuyện gửi từ Client (gồm câu hỏi của người dùng và mảng lịch sử hội thoại gần nhất).
- **IntentSchema:** Định nghĩa lược đồ dữ liệu có cấu trúc (*Structured Output Schema*) giúp mô hình Gemini phân loại chính xác câu hỏi vào các nhãn ý định phù hợp.
- **RAGEngine:** Trung tâm xử lý chính của phân hệ RAG, tích hợp mô hình phân loại ý định (*Intent Routing*), cơ chế tìm kiếm lai (Vector Search + BM25), cơ chế tự động khôi phục ngữ cảnh cha (*Parent Context Expansion*), cơ chế lọc mảnh tóm tắt (*Summary Retrieval*), và kết nối với Rails Search API khi định tuyến danh mục.
- **ExecutionPrompts & RouterPrompts:** Định nghĩa các mẫu Prompt chỉ dẫn hệ thống phân loại ý định và sinh câu trả lời tự nhiên bám sát theo ngữ cảnh.
- **ingestor_books.py & ingestor_rules.py:** Các kịch bản nạp (*Ingestion*) độc lập chịu trách nhiệm phân mảnh phân cấp (Parent-Child), sinh mảnh tóm tắt (*Summary Chunk*), tính toán vector hóa (*Embedding*) sử dụng mô hình nội bộ BAAI/bge-m3 và đẩy dữ liệu vào ChromaDB.
- **RagDataController:** Controller phía Ruby on Rails chịu trách nhiệm đồng bộ siêu dữ liệu sách phục vụ nạp liệu và cung cấp API tìm kiếm SQL chính xác khi RAG định tuyến đến nhãn `FIND_BOOK`.

4.2 Thiết kế giao diện

Phần này trình bày tài liệu thiết kế giao diện người dùng (User Interface - UI), đặc tả các thông số kỹ thuật màn hình mục tiêu, các tiêu chuẩn nhất quán khi thiết kế các điều khiển, và bố cục phác thảo (wireframe) của các màn hình chức năng quan trọng.

4.2.1 Thông số kỹ thuật

- **Màn hình máy tính cá nhân (Desktop/Laptop):**

- *Độ phân giải tối ưu*: 1920x1080 (Full HD) và tối thiểu là 1366x768.
- *Kích thước màn hình*: Từ 13 inch đến 27 inch trở lên.
- *Bố cục thiết kế*: Thiết kế theo chiều ngang (Grid System 12 cột) tận dụng tối đa chiều rộng màn hình để hiển thị các bảng dữ liệu lớn của trang quản trị.
- **Số lượng màu sắc hỗ trợ**: Hệ thống hỗ trợ dải màu chuẩn 24-bit True Color (khoảng 16.7 triệu màu), tương thích với hầu hết các trình duyệt Web hiện đại.

4.2.2 Quy chuẩn thiết kế giao diện nhất quán

Để đảm bảo trải nghiệm người dùng nhất quán (Consistency) và tăng tính thẩm mỹ chuyên nghiệp, hệ thống áp dụng các tiêu chuẩn thiết kế sau:

a, Phối màu

Hệ thống sử dụng bảng màu hiện đại, tối giản tạo cảm giác thư viện tri thức chuyên nghiệp kết hợp công nghệ thông minh:

- **Màu sắc chủ đạo (Primary Color)**: Màu xanh thẫm (#1E3A8A - Deep Blue) đại diện cho sự tin cậy, tri thức thư viện và tính bền vững. Được áp dụng cho Header, các nút hành động chính (Primary Buttons).
- **Màu sắc bổ trợ (Accent Color)**: Màu xanh ngọc (#0D9488 - Teal) đại diện cho công nghệ, trí tuệ nhân tạo (AI Chatbot) và chuyển đổi số. Sử dụng cho các chỉ số hoạt động, biểu tượng AI và trạng thái trực tuyến.
- **Màu nền (Background Color)**: Màu trắng tinh khiết (#FFFFFF) kết hợp màu xám nhạt (#F9FAFB) cho các vùng đệm để làm giảm mỏi mắt cho người dùng.
- **Màu sắc cảnh báo và trạng thái (Feedback Colors)**:
 - Màu đỏ (#DC2626) thông báo lỗi hoặc từ chối hành động.
 - Màu xanh lá (#16A34A) thông báo thành công hoặc đã duyệt.
 - Màu vàng (#D97706) thông báo trạng thái đang chờ xử lý.

b, Kiểu chữ

Sử dụng bộ font hiện đại là **Inter** hoặc **Roboto** để tối ưu hóa khả năng đọc trên màn hình điện tử:

- **Tiêu đề lớn (H1, H2)**: Cỡ chữ từ 20px đến 28px, định dạng đậm (Bold).
- **Văn bản nội dung (Body text)**: Cỡ chữ tiêu chuẩn 14px hoặc 16px, khoảng cách dòng (line-height) đặt ở mức 1.5 để người dùng không bị rối mắt

khi đọc văn bản dài của phân đoạn sách.

c, Nút bấm và các điều khiển nhập liệu

- **Nút bấm hành động (Buttons):** Được thiết kế dạng phẳng hoặc bo góc nhẹ (`border-radius: 6px`), chiều cao tối thiểu 40px để dễ bấm trên điện thoại. Khi rê chuột qua (hover), nút bấm tự động đổi tông màu (giảm độ sáng 10%) và chuyển con trỏ thành dạng bàn tay.
- **Trường nhập liệu (Input Fields):** Có đường viền xám mảnh, khi người dùng nhấp chọn (focus), đường viền sẽ đổi sang màu chủ đạo (`#1E3A8A`) và có viền sáng nhẹ xung quanh để báo hiệu trạng thái sẵn sàng nhập.

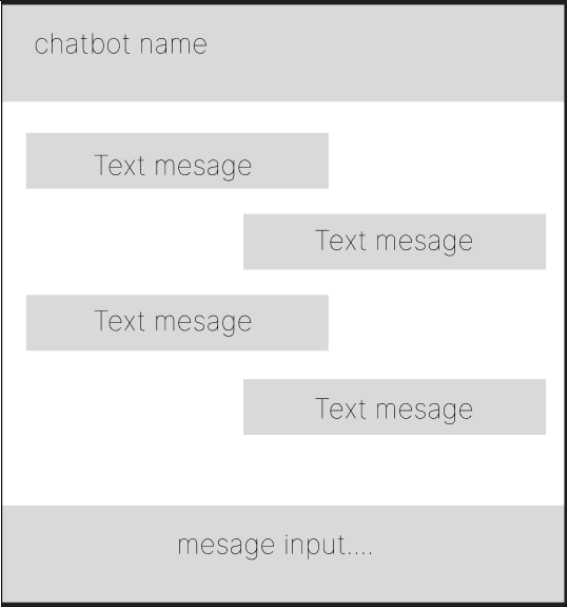
d, Vị trí hiển thị thông điệp phản hồi

- **Thông báo trạng thái nhanh (Toasts/Flash):** Hiển thị ở góc trên bên phải màn hình dưới dạng hộp thông tin tự động biến mất sau 3 giây (được dùng khi thêm sách vào giỏ, đăng nhập thành công).
- **Hộp thoại cảnh báo (Modal Popups):** Hiển thị ở chính giữa màn hình kèm một lớp nền xám mờ bao phủ bên dưới để yêu cầu người dùng xác nhận các thao tác quan trọng (như xác nhận duyệt mượn sách).
- **Lỗi nhập liệu (Validation Errors):** Hiển thị bằng chữ màu đỏ ngay phía dưới chân của hộp nhập liệu bị lỗi để người dùng dễ nhận biết và sửa lại.

4.2.3 Bố cục phác thảo một số giao diện quan trọng



Hình 4.4: Phác thảo bố cục trang chủ độc giả tích hợp Chatbot AI

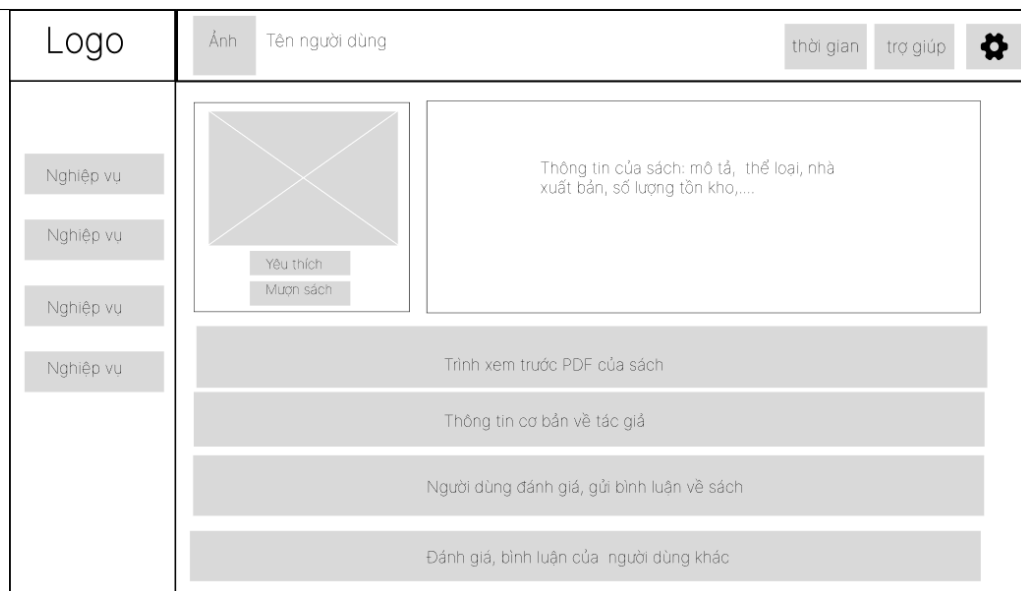


Hình 4.5: Phác thảo bố cục mockup Chatbot AI



Hình 4.6: Phác thảo bố cục trang tìm kiếm sách của độc giả

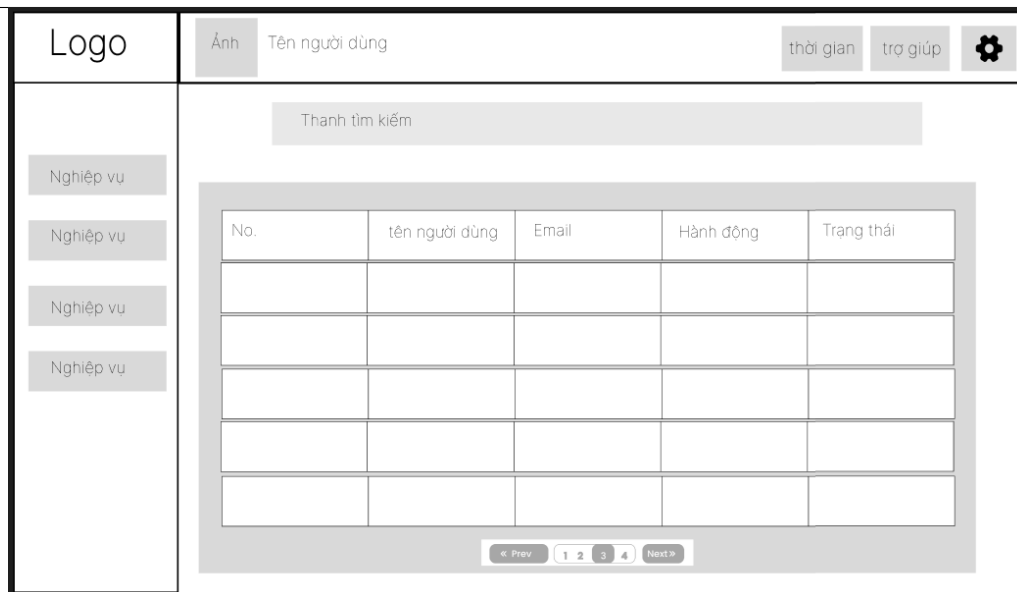
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.7: Phác thảo bố cục trang xem chi tiết sách của độc giả



Hình 4.8: Phác thảo bố cục trang chủ của thủ thư/Admin



Hình 4.9: Phác thảo bố cục trang chủ quản lý user của thủ thư/Admin

4.2.4 Thiết kế chi tiết lớp

Dưới đây là đặc tả chi tiết của các lớp quan trọng trong hệ thống: ChatbotController, Admin::BorrowRequestsController, BorrowRequestController (thuộc tầng Controller MVC) và lớp dịch vụ lõi RAGEngine.

a, Lớp ChatbotController

Lớp ChatbotController phía Rails hoạt động như một máy chủ ủy nhiệm và điều phối dự phòng (*Sync Fallback Proxy Controller*) kết nối giữa giao diện người dùng và máy chủ Python FastAPI. Lớp này cung cấp kênh xử lý đồng bộ tiêu chuẩn để bắt các ngoại lệ lỗi kết nối, quá thời gian (*Timeout*) và bảo vệ bảo mật luồng dữ liệu trước khi chuyển tiếp sang RAG Server.

Bảng 4.1: Đặc tả phương thức lớp ChatbotController

Tên phương thức	Kiểu trả về	Mô tả
query()	JSON	Tiếp nhận câu hỏi và lịch sử hội thoại dạng JSON từ phía trình duyệt qua phương thức HTTP POST. Thực hiện chuyển tiếp đồng bộ tới cổng 8000 (/ask), bắt và xử lý các ngoại lệ <code>Net::ReadTimeout</code> (sau 30 giây) hoặc lỗi dịch vụ AI ngoại tuyến để trả về thông báo lỗi thân thiện cho độc giả.

b, Lớp Admin::BorrowRequestsController

Lớp Admin::BorrowRequestsController quản lý toàn bộ các tác vụ xử lý phiếu yêu cầu mượn/trả sách dành cho quản trị viên và thủ thư, bao gồm duyệt phiếu, từ chối kèm lý do, cập nhật ngày nhận/trả thực tế và phân trang.

Bảng 4.2: Đặc tả thuộc tính và hằng số lớp Admin::BorrowRequestsController

Tên thuộc tính / Hằng số	Kiểu dữ liệu	Mô tả
PERMIT	Array	Hằng số danh sách các tham số an toàn được phép truyền vào (Strong Parameters) như: status, admin_note, actual_borrow_date, actual_return_date, v.v.
@borrow_request	BorrowRequest	Thực thể phiếu mượn đang được tác động (được gán qua bộ lọc set_borrow_request trước khi chạy các hành động chỉnh sửa).

Bảng 4.3: Đặc tả phương thức lớp Admin::BorrowRequestsController

Tên phương thức	Kiểu trả về	Mô tả
index()	HTML	Hiển thị danh sách toàn bộ phiếu mượn trong hệ thống, hỗ trợ tìm kiếm qua Ransack và phân trang qua thư viện Pagy.
show()	HTML	Xem thông tin chi tiết một phiếu mượn cụ thể.
edit_status()	Turbo Stream	Kết xuất biểu mẫu chỉnh sửa nhanh trạng thái phiếu mượn trực tiếp bằng cơ chế AJAX không tải lại trang.
change_status()	Redirect / JSON	Tiếp nhận yêu cầu đổi trạng thái từ Admin. Xử lý chuyển đổi trạng thái bằng phương thức nội bộ update_borrow_request_status và bắt lỗi ngoại lệ dữ liệu không hợp lệ.
status_extra_attributes (prev, new)	Hash	Xác định các thuộc tính bổ sung cần cập nhật ứng với trạng thái mới (ví dụ: gán người duyệt approved_by_admin_id khi chuyển sang trạng thái duyệt).

c, Lớp BorrowRequestController

Lớp BorrowRequestController phục vụ độc giả thực hiện quản lý giỏ sách mượn tạm thời lưu trong session và tiến hành checkout tạo yêu cầu mượn sách chính thức.

Bảng 4.4: Đặc tả thuộc tính và hằng số lớp BorrowRequestController

Tên thuộc tính / Hằng số	Kiểu dữ liệu	Mô tả
SELECTED	String	Đánh dấu mặt hàng sách được độc giả tích chọn để tạo phiếu mượn (giá trị “1”).
NOT_SELECTED	String	Đánh dấu mặt hàng trong giỏ sách nhưng không được tích chọn (giá trị “0”).

Bảng 4.5: Đặc tả phương thức lớp BorrowRequestController

Tên phương thức	Kiểu trả về	Mô tả
index()	HTML	Đọc danh sách sách độc giả đã lưu tạm trong session[:borrow_cart], nạp thông tin chi tiết sách từ CSDL và phân trang hiển thị lên giỏ sách mượn.
update_borrow_cart()	Redirect / JSON	Cập nhật số lượng sách, ngày bắt đầu hên mượn và ngày trả của giỏ sách trong session.
remove_from_borrow_cart()	Redirect / JSON	Loại bỏ một cuốn sách ra khỏi giỏ mượn trong session.
checkout()	Redirect	Kiểm tra điều kiện mượn sách, khởi tạo thực thể BorrowRequest cùng các BorrowRequestItem tương ứng với các đầu sách được chọn, và dọn dẹp các sách đã checkout ra khỏi giỏ.

d, Lớp RAGEngine

Lớp RAGEngine được xây dựng bằng Python, chịu trách nhiệm tiếp nhận câu hỏi của người dùng, phân tích ngữ nghĩa, truy xuất kiến thức tài liệu và kết hợp LLM sinh phản hồi.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.6: Đặc tả thuộc tính lớp RAGEngine

Tên thuộc tính	Kiểu dữ liệu	Mô tả
embeddings	HuggingFaceEmbeddings	Mô hình chuyển văn bản thành vector (sử dụng mô hình cục bộ BAAI/bge-m3).
vector_db	Chroma	Kết nối với Cơ sở dữ liệu Vector lưu trữ dữ liệu sách đã phân mảnh (lưu trên phân vùng Ext4).
llm	ChatGoogleGenerativeAI	Mô hình ngôn ngữ lớn chính (Gemini) để sinh phản hồi tự nhiên (cấu hình tối đa 5 lần thử lại tự động).
router_llm	ChatGoogleGenerativeAI	Mô hình sinh ngôn ngữ được cấu hình tối ưu để phân loại ý định của câu hỏi.
hybrid_retriever	ContextualCompressionRetriever	Bộ truy xuất lai tích hợp từ tìm kiếm vector và BM25, kết hợp Cohere Rerank để xếp hạng lại tài liệu.

Bảng 4.7: Đặc tả phương thức chính lớp RAGEngine

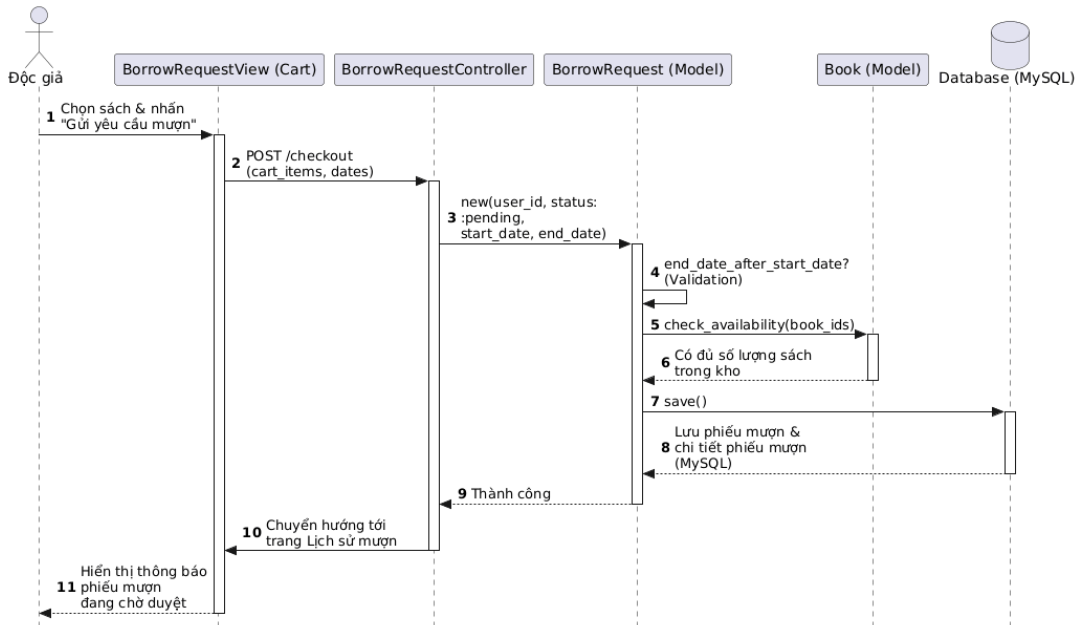
Tên phương thức	Kiểu trả về	Mô tả
_setup_hybrid_retriever()	Retriever	Phương thức nội bộ cấu hình bộ truy xuất lai tích hợp cơ chế Caching BM25 chống tràn RAM.
route_intent(query)	String	Phân loại câu hỏi của người dùng thành các nhãn cụ thể: FIND_BOOK, BOOK_INFO, POLICY, CHAT.
_expand_parents(docs)	List[Document]	Phương thức nội bộ thực hiện khôi phục ngữ cảnh cha (<i>Parent Context Expansion</i>) từ danh sách các mảnh con tìm thấy.
_retrieve_context(query)	List[Document]	Phương thức nội bộ truy xuất thông tin (lọc riêng mảnh tóm tắt nếu là câu hỏi tóm tắt hoặc truy xuất lai kết hợp khôi phục ngữ cảnh cha).
ask(query)	Dict	Thực thi luồng xử lý RAG đồng bộ: định tuyến → truy xuất dữ liệu (hoặc gọi Rails SQL API) → sinh câu trả lời và trả về JSON kèm danh sách nguồn trích dẫn.
ask_stream(query)	Iterator[String]	Thực thi luồng xử lý RAG bất đồng bộ: trả kết quả dạng luồng (<i>Server-Sent Events</i>) gồm các mảnh từ và nguồn trích dẫn theo thời gian thực về Client.

4.2.5 Biểu đồ trình tự một số use case quan trọng

Dưới đây là biểu đồ trình tự và phân tích luồng thông điệp giữa các đối tượng trong 3 use case cốt lõi của hệ thống.

a, Use Case 2: Độc giả tạo yêu cầu mượn sách

Use case mô tả luồng nghiệp vụ khi độc giả chọn các cuốn sách và thực hiện gửi phiếu yêu cầu mượn sách lên hệ thống.



Hình 4.10: Biểu đồ trình tự luồng Độc giả tạo yêu cầu mượn sách

Mô tả các bước thực hiện trong luồng:

1. Độc giả chọn sách vào danh sách mượn trên giao diện và nhấn “Gửi yêu cầu mượn”.
2. Giao diện gửi tham số phiếu mượn (ngày mượn, ngày trả hẹn, danh sách ID sách) về BorrowRequestController.
3. BorrowRequestController khởi tạo một đối tượng mới BorrowRequest với trạng thái mặc định là pending.
4. Đối tượng BorrowRequest tự động thực hiện các validations nghiệp vụ:
 - Kiểm tra ngày kết thúc mượn phải sau ngày bắt đầu.
 - Gọi lớp Book để kiểm tra xem số lượng sách hiện tại trong kho còn khả dụng ($available_quantity > 0$) hay không.
5. Sau khi kiểm tra thành công, đối tượng BorrowRequest gọi lệnh lưu dữ liệu vật lý xuống cơ sở dữ liệu quan hệ Database (MySQL).

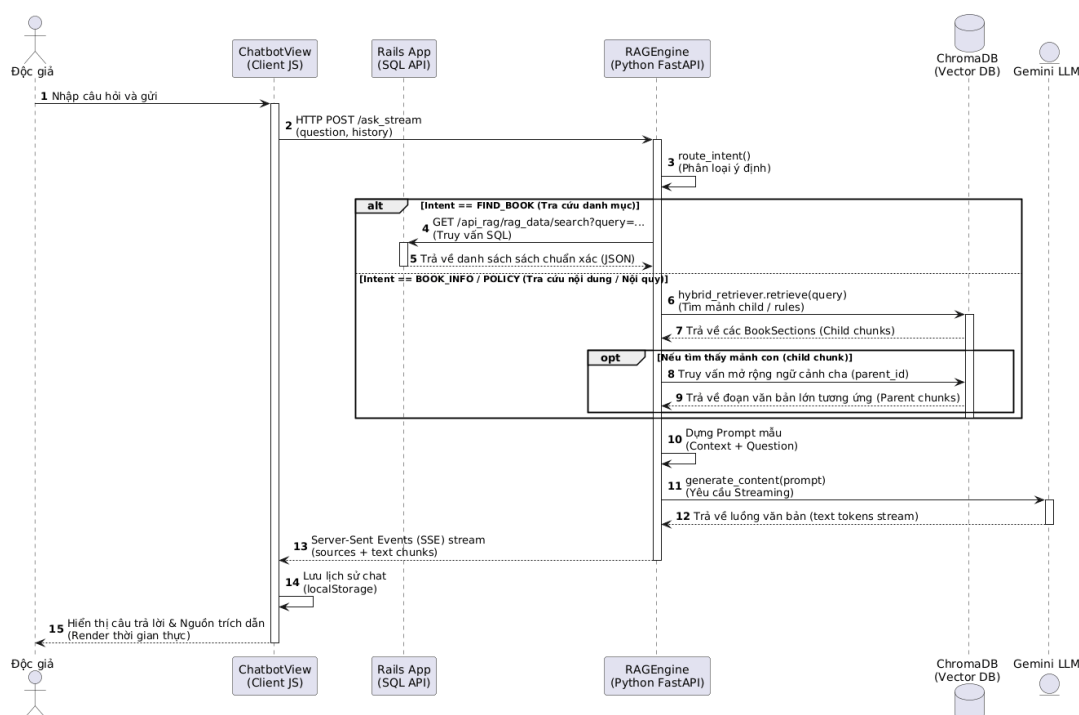
6. Database phản hồi lưu trữ thành công.

7. BorrowRequestController nhận kết quả lưu thành công và điều phối chuyển hướng giao diện của độc giả tới trang chi tiết lịch sử mượn.

8. Độc giả nhận được thông điệp thông báo yêu cầu mượn sách đang được chờ duyệt.

b, Use Case 3: Độc giả gửi câu hỏi cho Chatbot AI (Luồng RAG)

Use case này mô tả luồng khi độc giả gửi một câu hỏi tự nhiên đến chatbot để tra cứu quy định thư viện hoặc thông tin tài liệu. Phân hệ RAG sẽ can thiệp để tìm tài liệu liên quan trước khi trả lời.



Hình 4.11: Biểu đồ trình tự luồng hỏi đáp Chatbot RAG

Mô tả các bước thực hiện trong luồng:

1. Độc giả nhập câu hỏi tự nhiên lên giao diện ChatbotView và nhấn gửi. Lịch sử trò chuyện và nội dung chat được đồng bộ vào localStorage của trình duyệt.
2. ChatbotView (Javascript Client) gửi trực tiếp yêu cầu HTTP POST kết nối luồng (Streaming) tới API Python /ask_stream của RAGEngine, truyền kèm câu hỏi và lịch sử 6 lượt hội thoại gần nhất.
3. RAGEngine gọi phương thức route_intent() để phân loại ý định (Intent) của câu hỏi.
4. Xử lý định tuyến thông minh:

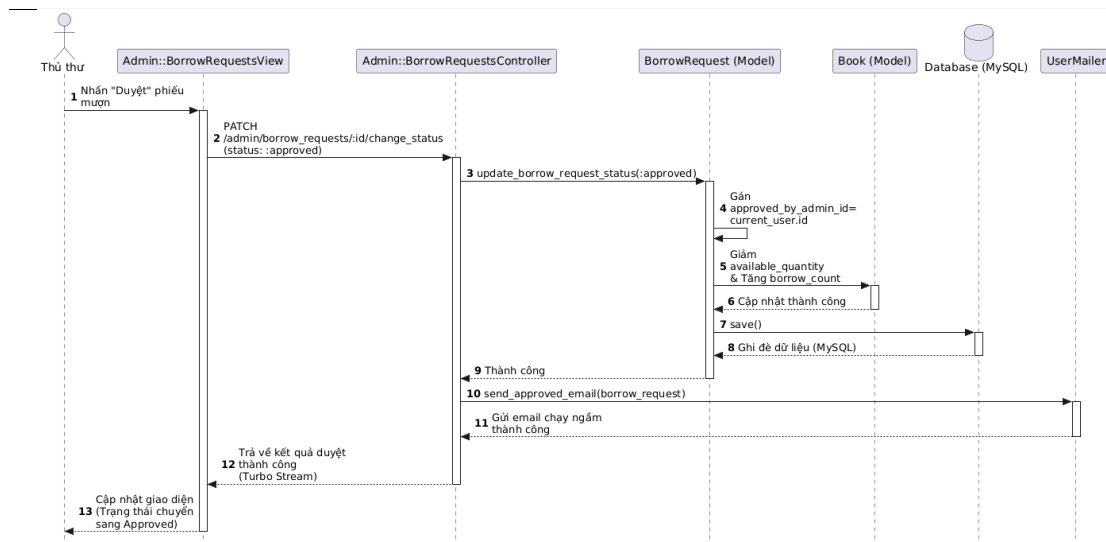
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

- Nếu Intent là tra cứu danh mục (FIND_BOOK): RAGEngine trích xuất từ khóa tìm kiếm và gửi yêu cầu SQL API tới Rails Backend để lấy danh sách sách chính xác 100
 - Nếu Intent là tra cứu nội dung (BOOK_INFO): RAGEngine thực hiện tìm kiếm trên các mảnh con (child chunks) trong ChromaDB.
 - Nếu Intent là tóm tắt sách: RAGEngine chỉ tìm kiếm trên các mảnh tóm tắt sách (book_summary).
5. ChromaDB trả về các phân đoạn văn bản phù hợp. Với các mảnh con, RAGEngine tự động truy vấn tìm kiếm mảnh cha lớn tương ứng (Parent Context Expansion) để lấy đầy đủ ngữ cảnh.
 6. RAGEngine tổng hợp dữ liệu ngữ cảnh phong phú này vào mẫu prompt (trong ExecutionPrompts) cùng câu hỏi của Độc giả.
 7. RAGEngine gửi yêu cầu sinh văn bản dạng luồng (Streaming) tới dịch vụ Gemini LLM.
 8. Dịch vụ Gemini LLM sinh câu trả lời tự nhiên dựa trên ngữ cảnh và trả về các mảnh từ (token text) theo thời gian thực cho RAGEngine.
 9. RAGEngine chuyển tiếp luồng dữ liệu (bao gồm thông tin nguồn trích dẫn sources và các mảnh từ text) về trực tiếp cho ChatbotView dưới dạng Server-Sent Events (SSE).
 10. ChatbotView nhận các mảnh dữ liệu, giải mã và hiển thị dần câu trả lời lên giao diện cho Độc giả ngay lập tức, đồng thời cập nhật trạng thái lưu trữ cục bộ.

c, Use Case 3: Thủ thư phê duyệt yêu cầu mượn sách

Use case mô tả luồng khi thủ thư kiểm tra danh sách chờ và duyệt cấp sách cho độc giả đến nhận.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.12: Biểu đồ trình tự luồng Thủ thư duyệt yêu cầu mượn sách

Mô tả các bước thực hiện trong luồng:

1. Thủ thư truy cập trang danh sách yêu cầu chờ duyệt trên giao diện
Admin::BorrowRequestsView và nhấn nút “Duyệt” trên một phiếu mượn cụ thể.
2. Giao diện gửi yêu cầu cập nhật trạng thái phiếu mượn lên Admin::BorrowRequestsController.
3. Admin::BorrowRequestsController tìm nạp thực thể BorrowRequest tương ứng và gọi phương thức cập nhật trạng thái từ pending sang approved.
4. Thực thể BorrowRequest thực thi nghiệp vụ:
 - Cập nhật thời điểm phê duyệt và định danh thủ thư duyệt (approved_by_admin_id).
 - Đối với mỗi cuốn sách nằm trong chi tiết phiếu mượn (BorrowRequestItem), BorrowRequest gửi thông điệp yêu cầu thực thể Book giảm số lượng sách khả dụng trong kho (available_quantity) đi 1 đơn vị, đồng thời tăng borrow_count lên 1.
5. Thực thể BorrowRequest thực hiện ghi đè dữ liệu cập nhật xuống cơ sở dữ liệu Database (MySQL).
6. Sau khi CSDL lưu thành công, Admin::BorrowRequestsController kích hoạt dịch vụ UserMailer chạy ngầm để gửi email thông báo cho độc giả.
7. UserMailer gửi email thông báo thời gian hạn đến nhận sách vật lý tới tài khoản email của Độc giả.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

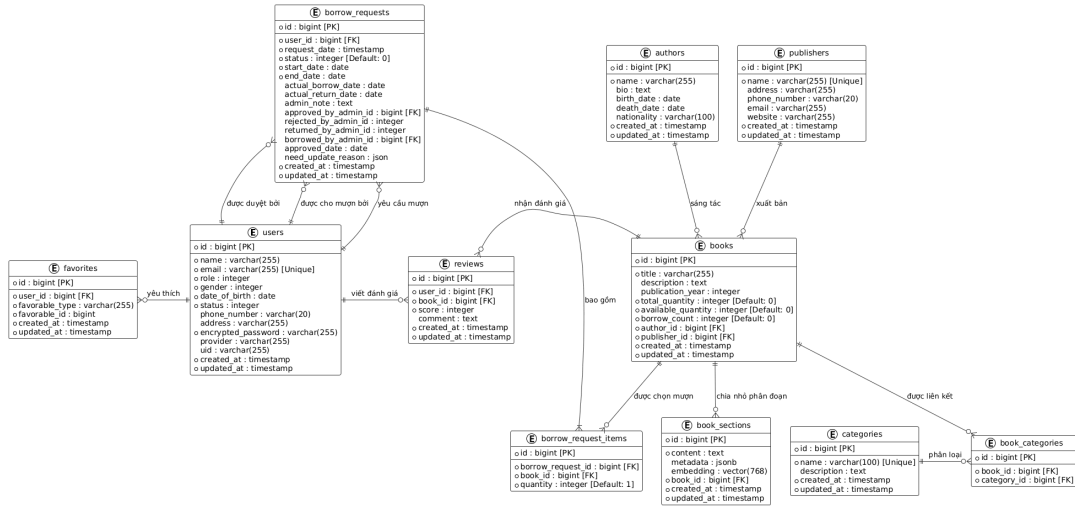
8. Admin::BorrowRequestsController trả kết quả duyệt thành công về cho giao diện quản trị của Thủ thư.

4.3 Thiết kế cơ sở dữ liệu

Phần này trình bày thiết kế cơ sở dữ liệu của hệ thống Quản lý thư viện tích hợp RAG. Hệ thống sử dụng kiến trúc cơ sở dữ liệu lai (Hybrid Database) bao gồm: hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL đóng vai trò lưu trữ các thông tin giao dịch có cấu trúc và tiện ích mở rộng pgvector hỗ trợ lưu trữ dữ liệu vector nhưng phục vụ chatbot AI tìm kiếm ngữ nghĩa.

4.3.1 Biểu đồ thực thể liên kết (E-R Diagram)

Dưới đây là biểu đồ thực thể liên kết mô tả cấu trúc dữ liệu tổng thể và các mối quan hệ giữa các bảng trong hệ thống.



Hình 4.13: Biểu đồ thực thể liên kết (E-R Diagram)

Dưới đây là giải thích các mối quan hệ trong biểu đồ E-R trong hình 4.13:

Mối quan hệ giữa User và BorrowRequest (1 - N): Một độc giả (User) có thể gửi nhiều yêu cầu mượn sách (BorrowRequest) qua các thời điểm khác nhau. Ngược lại, mỗi yêu cầu mượn sách chỉ thuộc quyền sở hữu của duy nhất một người dùng.

Mối quan hệ duyệt phiếu mượn (User - BorrowRequest): Các liên kết `approved_by_admin`, `rejected_by_admin`, `returned_by_admin` thể hiện mối quan hệ giữa các tài khoản thủ thư/admin (User) chịu trách nhiệm cập nhật trạng thái phiếu mượn. Đây là quan hệ liên kết khóa ngoại tự tham chiếu tới bảng User.

Mối quan hệ hợp thành BorrowRequest và BorrowRequestItem (1 - N): Mỗi phiếu mượn sách (BorrowRequest) có cấu trúc bao gồm một hoặc nhiều mặt hàng chi tiết (BorrowRequestItem) ghi nhận từng cuốn sách mượn. Mối quan hệ này là

quan hệ hợp thành (Composition): nếu phiếu mượn bị hủy hoặc xóa bỏ, các chi tiết phiếu liên quan cũng tự động bị hủy theo.

Mối quan hệ liên kết giữa Book và BorrowRequestItem (1 - N): Mỗi chi tiết phiếu mượn (BorrowRequestItem) liên kết đến một cuốn sách cụ thể (Book) được mượn. Một cuốn sách có thể xuất hiện trên nhiều phiếu mượn của các độc giả khác nhau.

Mối quan hệ giữa Book, Author và Publisher (N - 1): Một tác giả (Author) có thể viết nhiều cuốn sách khác nhau, và một nhà xuất bản (Publisher) có thể ấn hành nhiều đầu sách. Ngược lại, mỗi cuốn sách chỉ thuộc về một tác giả và một nhà xuất bản quản lý trong hệ thống.

Mối quan hệ nhiều - nhiều giữa Book và Category (N - M): Một cuốn sách có thể thuộc nhiều thể loại danh mục khác nhau (Category), và một thể loại danh mục có thể chứa nhiều cuốn sách. Mối quan hệ này được chuẩn hóa thông qua bảng liên kết trung gian BookCategory.

Mối quan hệ giữa Book và BookSection (1 - N): Để phục vụ cho mô hình hỏi đáp AI RAG, mỗi cuốn sách (Book) được phân chia và lưu trữ thành nhiều phân đoạn hoặc chương nội dung nhỏ (BookSection). Mỗi phân đoạn lưu trữ một đoạn văn bản thô cùng với một vector nhúng đa chiều.

Mối quan hệ đánh giá và yêu thích (User - Book - Review/Favorite): Độc giả có thể viết đánh giá (Review) cho các cuốn sách đã đọc và đánh dấu các cuốn sách/tác giả yêu thích (Favorite).

4.3.2 Thiết kế chi tiết các bảng cơ sở dữ liệu

Dưới đây là cấu trúc chi tiết của toàn bộ 11 bảng trong cơ sở dữ liệu PostgreSQL của hệ thống.

a, Bảng users (Quản lý người dùng)

Lưu trữ thông tin cá nhân và tài khoản xác thực của độc giả, thủ thư và quản trị viên.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.8: Cấu trúc chi tiết bảng users

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính định danh tài khoản.
name	varchar(255)	Not Null	Họ và tên đầy đủ người dùng.
email	varchar(255)	Not Null, Unique	Email đăng nhập hệ thống.
role	integer	Not Null, Default 0	Vai trò: 0 (Độc giả), 1 (Admin).
gender	integer	Not Null	Giới tính: 0 (Nam), 1 (Nữ), 2 (Khác).
date_of_birth	date	Not Null	Ngày sinh của độc giả.
status	integer	Not Null, Default 0	Trạng thái: 0 (Chưa kích hoạt), 1 (Hoạt động).
phone_number	varchar(20)	Allow Null	Số điện thoại.
address	varchar(255)	Allow Null	Địa chỉ cư trú.
encrypted_password	varchar(255)	Not Null	Mật khẩu đã được băm.
provider	varchar(255)	Allow Null	Nhà cung cấp dịch vụ OAuth (như google_oauth2).
uid	varchar(255)	Allow Null	Mã định danh của người dùng trên hệ thống OAuth.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

b, Bảng books (Quản lý đầu sách)

Lưu trữ thông tin chi tiết của các đầu sách và trạng thái số lượng bản sách vật lý trong kho thư viện.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.9: Cấu trúc chi tiết bảng books

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính định danh sách.
title	varchar(255)	Not Null	Tiêu đề sách (có index tìm kiếm nhanh).
description	text	Allow Null	Tóm tắt nội dung sách.
publication_year	integer	Allow Null	Năm xuất bản sách.
total_quantity	integer	Not Null, Default 0	Tổng số lượng sách nhập về kho.
available_quantity	integer	Not Null, Default 0	Số lượng sách còn lại trong kho có thể cho mượn.
borrow_count	integer	Not Null, Default 0	Số lượt mượn tích lũy của cuốn sách.
author_id	bigint	Foreign Key, Not Null	Khóa ngoại trỏ đến bảng authors.
publisher_id	bigint	Foreign Key, Not Null	Khóa ngoại trỏ đến bảng publishers.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

c, Bảng authors (Quản lý tác giả)

Lưu trữ thông tin tiểu sử và quốc tịch của tác giả viết sách.

Bảng 4.10: Cấu trúc chi tiết bảng authors

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính định danh tác giả.
name	varchar(255)	Not Null	Họ và tên tác giả.
bio	text	Allow Null	Tiểu sử tóm tắt.
birth_date	date	Allow Null	Ngày sinh của tác giả.
death_date	date	Allow Null	Ngày mất của tác giả (nếu có).
nationality	varchar(100)	Allow Null	Quốc tịch tác giả.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

d, Bảng publishers (Quản lý nhà xuất bản)

Lưu trữ thông tin liên lạc của các nhà xuất bản sách phục vụ việc nhập kho.

Bảng 4.11: Cấu trúc chi tiết bảng publishers

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính định danh nhà xuất bản.
name	varchar(255)	Not Null, Unique	Tên nhà xuất bản.
address	varchar(255)	Allow Null	Địa chỉ trụ sở.
phone_number	varchar(20)	Allow Null	Số điện thoại liên hệ.
email	varchar(255)	Allow Null	Email liên hệ.
website	varchar(255)	Allow Null	Địa chỉ website nhà xuất bản.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

e, Bảng categories (Quản lý các thể loại sách)

Lưu trữ danh sách thể loại phân loại kho sách.

Bảng 4.12: Cấu trúc chi tiết bảng categories

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính định danh thể loại.
name	varchar(100)	Not Null, Unique	Tên thể loại (Ví dụ: Tin học, Ngoại ngữ).
description	text	Allow Null	Mô tả thể loại sách.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

f, Bảng book_categories (Liên kết sách - thể loại)

Bảng trung gian thể hiện mối quan hệ nhiều - nhiều giữa Sách và Thể loại.

Bảng 4.13: Cấu trúc chi tiết bảng book_categories

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính tự sinh.
book_id	bigint	Foreign Key, Not Null	Liên kết với bảng books (có index).
category_id	bigint	Foreign Key, Not Null	Liên kết với bảng categories (có index).

g, Bảng book_sections (Phân đoạn sách phục vụ RAG)

Đây là bảng cốt lõi phục vụ hệ thống AI RAG, sử dụng tính năng mở rộng pgvector của PostgreSQL để hỗ trợ truy vấn tìm kiếm vector ngữ nghĩa.

Bảng 4.14: Cấu trúc chi tiết bảng book_sections

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính tự sinh.
content	text	Not Null	Nội dung văn bản thô của phân đoạn sách.
metadata	jsonb	Allow Null	Các dữ liệu bổ sung kèm theo (thông tin chương, trang, tiêu đề sách).
embedding	vector(768)	Allow Null	Vector nhúng biểu diễn ngữ nghĩa (mô hình Embedding có số chiều là 768).
book_id	bigint	Foreign Key, Not Null	Khóa ngoại liên kết tới sách gốc.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

h, Bảng borrow_requests (Phiếu mượn sách)

Lưu trữ thông tin tổng quát của các yêu cầu mượn sách từ độc giả và thông tin kiểm duyệt của thủ thư.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.15: Cấu trúc bảng borrow_requests (Phần 1: Thông tin đặt mượn)

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key, Auto-increment	Khóa chính của phiếu mượn.
user_id	bigint	Foreign Key, Not Null	Độc giả thực hiện mượn sách.
request_date	timestamp	Not Null	Ngày độc giả gửi yêu cầu mượn.
status	integer	Not Null, Default 0	Trạng thái phiếu: 0 (Đang chờ), 1 (Đã duyệt), 2 (Từ chối), 3 (Đã trả), 4 (Quá hạn), v.v.
start_date	date	Not Null	Ngày hẹn đến thư viện nhận sách.
end_date	date	Not Null	Hạn chót độc giả phải mang trả sách.
actual_borrow_date	date	Allow Null	Ngày thực tế nhận sách.
actual_return_date	date	Allow Null	Ngày thực tế mang trả sách.
admin_note	text	Allow Null	Ghi chú phản hồi của thủ thư.

Bảng 4.16: Cấu trúc bảng borrow_requests (Phần 2: Thông tin kiểm duyệt & Nhật ký)

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
approved_by_admin_id	bigint	Foreign Key, Allow Null	Thủ thư duyệt yêu cầu.
rejected_by_admin_id	integer	Allow Null	Thủ thư từ chối yêu cầu.
returned_by_admin_id	integer	Allow Null	Thủ thư nhận trả sách.
borrowed_by_admin_id	bigint	Foreign Key, Allow Null	Thủ thư cấp sách thực tế.
approved_date	date	Allow Null	Ngày phê duyệt yêu cầu.
need_update_reason	json	Allow Null	Lý do thủ thư yêu cầu độc giả cập nhật lại phiếu.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

i, Bảng borrow_request_items (Chi tiết phiếu mượn)

Bảng trung gian thể hiện mối quan hệ chứa đựng giữa phiếu mượn và danh sách sách mượn cụ thể.

Bảng 4.17: Cấu trúc chi tiết bảng borrow_request_items

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính tự sinh.
borrow_request_id	bigint	Foreign Key, Not Null	Liên kết với bảng borrow_requests.
book_id	bigint	Foreign Key, Not Null	Đầu sách được chọn mượn.
quantity	integer	Not Null, Default 1	Số lượng bản sách mượn.

j, Bảng reviews (Đánh giá sách)

Lưu trữ thông tin đánh giá, nhận xét điểm số (Rating) của độc giả đối với các đầu sách.

Bảng 4.18: Cấu trúc chi tiết bảng reviews

Tên cột	Kiểu dữ liệu	Mô tả	Chi tiết ràng buộc
id	bigint	Primary Key	Khóa chính định danh đánh giá.
user_id	bigint	Foreign Key, Not Null	Độc giả viết đánh giá.
book_id	bigint	Foreign Key, Not Null	Đầu sách được đánh giá.
score	integer	Not Null	Điểm số đánh giá (thang điểm 1-5).
comment	text	Allow Null	Nhận xét chi tiết của người dùng.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

k, Bảng favorites (Danh sách yêu thích)

Lưu trữ danh sách các tài nguyên (sách, tác giả) được độc giả bấm thích/theo dõi.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.19: Cấu trúc chi tiết bảng favorites

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigint	Primary Key	Khóa chính định danh lượt thích.
user_id	bigint	Foreign Key, Not Null	Độc giả thực hiện bấm thích.
favorable_type	varchar(255)	Not Null	Lớp thực thể được thích (Ví dụ: “Book”, “Author”).
favorable_id	bigint	Not Null	Khóa chính định danh của thực thể tương ứng.
created_at	timestamp	Not Null	Thời gian bản ghi khởi tạo.
updated_at	timestamp	Not Null	Thời gian bản ghi cập nhật mới nhất.

4.4 Xây dựng ứng dụng

4.4.1 Thư viện và công cụ sử dụng

Hệ thống được phát triển dựa trên sự kết hợp giữa khung ứng dụng web Ruby on Rails (cho phần quản trị và giao dịch) và dịch vụ FastAPI Python (cho phần động cơ RAG). Bảng dưới đây liệt kê chi tiết các công cụ và thư viện được áp dụng cùng với phiên bản cụ thể và địa chỉ URL tham khảo.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.20: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ	Địa chỉ URL
Ngôn ngữ lập trình chính phía Backend	Ruby v3.2.2	https://www.ruby-lang.org/
Khung phát triển ứng dụng Web (MVC)	Ruby on Rails v7.0.7	https://rubyonrails.org/
Ngôn ngữ lập trình dịch vụ AI RAG	Python v3.10	https://www.python.org/
Hệ quản trị cơ sở dữ liệu quan hệ	PostgreSQL v15	https://www.postgresql.org/
Tiện ích mở rộng lưu trữ vector	pgvector v0.5	https://github.com/pgvector/pgvector
Khung phát triển API hiệu năng cao	FastAPI v0.100	https://fastapi.tiangolo.com/
Máy chủ chạy ứng dụng ASGI cho FastAPI	Uvicorn v0.22	https://www.uvicorn.org/
Thư viện điều phối dịch vụ LLM & RAG	LangChain v0.1.0	https://www.langchain.com/
Bộ tích hợp mô hình ngôn ngữ Gemini	langchain-google-genai	https://github.com/langchain-ai/langchain-google
Cơ sở dữ liệu Vector lưu trữ dữ liệu	langchain-chroma	https://github.com/langchain-ai/langchain-chroma
Dịch vụ chấm điểm, tái xếp hạng tài liệu	Cohere Rerank API	https://cohere.com/
Mô hình ngôn ngữ lớn xử lý hội thoại	Gemini 2.5 Flash API	https://ai.google.dev/
Thư viện kết nối vector DB trong Rails	Neighbor v0.2.3	https://github.com/ankane/neighbor
Thư viện vẽ biểu đồ thống kê trực quan	Chartkick v5.0	https://chartkick.com/
Môi trường phát triển tích hợp (IDE)	VS Code / Ruby LSP	https://code.visualstudio.com/

4.4.2 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và triển khai, dự án đã xây dựng hoàn chỉnh hệ thống Quản lý thư viện tích hợp chatbot hỏi đáp thông minh RAG.

a, Các sản phẩm đóng gói và vai trò

Hệ thống được đóng gói thành ba sản phẩm/phân hệ chính:

- **Web Quản lý Thư viện (Ruby on Rails):**

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

- *Thành phần:* Mã nguồn MVC (Models, Controllers, Views), các Job chạy nền tự động (Cronjobs qua Whenever), và hệ thống gửi thư điện tử tự động (UserMailer).
- *Vai trò:* Là cổng giao tiếp chính của hệ thống, chịu trách nhiệm quản lý cơ sở dữ liệu sách, quản lý người dùng, xử lý các nghiệp vụ mượn/trả sách vật lý, xác thực đăng nhập (Devise) và phân quyền (CanCanCan).
- **Dịch vụ Trợ lý Hỏi đáp Thông minh (FastAPI & Python RAG):**
 - *Thành phần:* API FastAPI, Động cơ RAG lõi (RAGEngine), Bộ truy xuất lại (BM25 + Vector Search), Bộ xếp hạng lại Cohere Rerank, và Vector Database (ChromaDB).
 - *Vai trò:* Đóng vai trò là bộ não AI của hệ thống. Phân hệ này tiếp nhận các câu hỏi tự nhiên từ độc giả, thực hiện tìm kiếm ngữ nghĩa trên Vector DB, tích hợp ngữ cảnh vào prompt và gọi mô hình ngôn ngữ lớn (Gemini LLM) để sinh câu trả lời tự nhiên, chính xác.
- **Cơ sở dữ liệu lai (PostgreSQL & ChromaDB):**
 - *Thành phần:* Cơ sở dữ liệu quan hệ PostgreSQL (lưu trữ giao dịch có cấu trúc) và Vector Store ChromaDB (lưu trữ vector embeddings của tài liệu).
 - *Vai trò:* Lưu trữ bền vững toàn bộ dữ liệu nghiệp vụ của thư viện và đảm bảo khả năng truy xuất vector tốc độ cao phục vụ tìm kiếm thông minh.

b, Thống kê số liệu kỹ thuật của ứng dụng

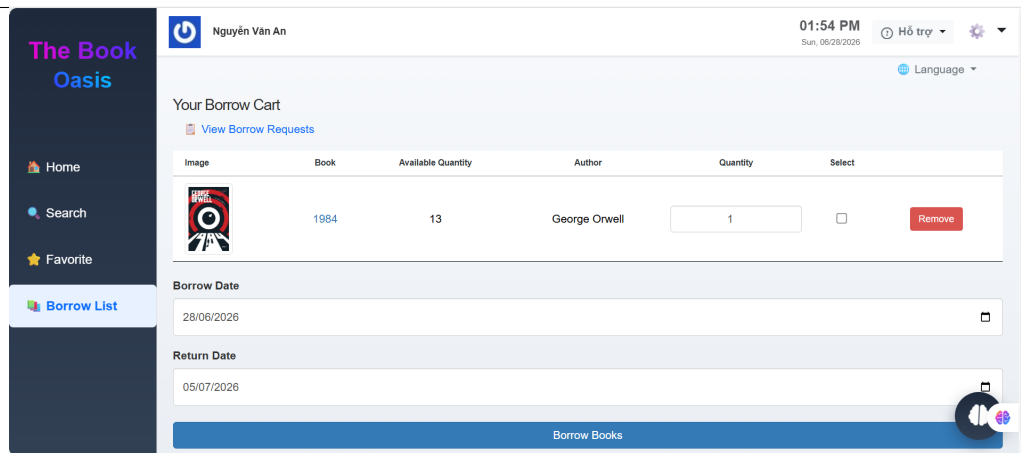
Bảng 4.21: Thống kê các số liệu kỹ thuật của hệ thống

Thông số thống kê	Giá trị	Mô tả / Ý nghĩa
Tổng số dòng code Rails (Ruby, ERB)	10.759 dòng	Bao gồm các tệp cấu hình, controllers, models, views và các kịch bản kiểm thử rspec.
Tổng số dòng code RAG (Python)	788 dòng	Chứa mã nguồn động cơ RAG, cấu hình prompts, API FastAPI và mã nguồn nạp dữ liệu (ingestor).
Tổng số dòng code hệ thống	11.547 dòng	Quy mô mã nguồn tự phát triển của dự án (loại trừ các thư viện bên thứ ba).
Số lượng lớp nghiệp vụ	36 lớp	Các lớp thực thể (Models) và lớp điều hướng (Controllers) trong hệ thống Rails và Python.
Số lượng gói cấu trúc (Packages)	5 gói	Bao gồm: views, controllers, models, service(rag) và database.
Tổng dung lượng thư mục dự án	558 MB	Tổng dung lượng toàn bộ thư mục (bao gồm mã nguồn, các tệp sách PDF mẫu và dữ liệu nạp ban đầu).
Dung lượng phân hệ Rails (Backend)	≈ 12 MB	Chỉ tính riêng các thư mục mã nguồn sạch (app, config, db, lib).
Dung lượng phân hệ RAG (Python API)	≈ 2 MB	Chỉ tính mã nguồn dịch vụ Python (loại trừ thư mục ảo venv và dữ liệu ChromaDB vật lý).
Dung lượng cơ sở dữ liệu & tệp PDF sách	≈ 544 MB	Dung lượng của các tệp PDF tài liệu sách mẫu cùng với cơ sở dữ liệu vector ChromaDB và dữ liệu MySQL/PostgreSQL.

4.4.3 Minh họa các chức năng chính

a, Giao diện Giỏ sách mượn (Your Borrow Cart) - Dành cho Độc giả

Giao diện này giúp độc giả quản lý danh sách sách muốn mượn và chọn ngày mượn/trả dự kiến trước khi gửi yêu cầu chính thức đến thủ thư.



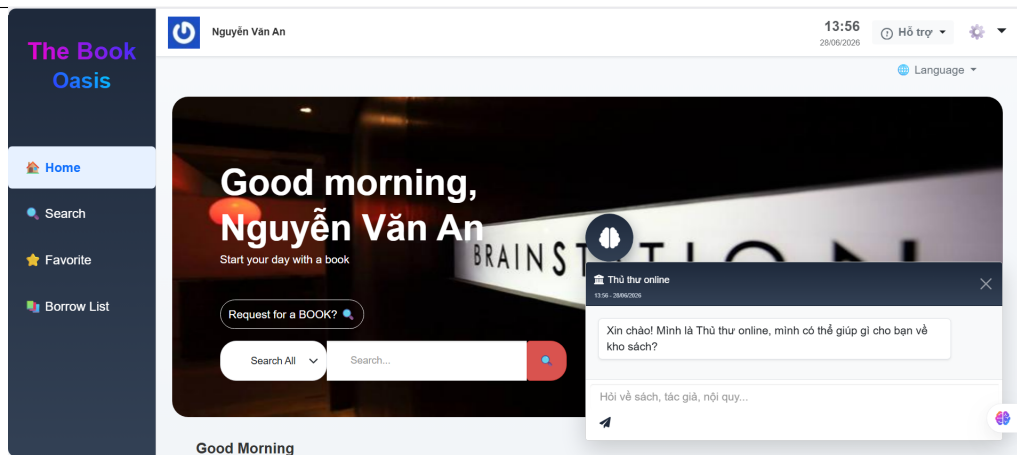
Hình 4.14: Giao diện Giỏ sách mượn của độc giả

Các thành phần chính trên giao diện bao gồm:

- **Danh sách sách chọn mượn:** Hiển thị chi tiết bìa sách (Image), tên sách kèm liên kết (Book), số lượng sẵn có trong kho (Available Quantity - ví dụ: 13 cuốn sách 1984), tên tác giả (Author), số lượng đăng ký mượn (Quantity - mặc định là 1) và nút **Remove** màu đỏ để xóa sách khỏi giỏ.
- **Thời gian mượn/trả sách (Borrow & Return Date):** Độc giả sử dụng bộ chọn ngày (Date Picker) để xác định ngày bắt đầu mượn (Borrow Date) và ngày dự kiến trả sách (Return Date).
- **Nút xác nhận "Borrow Books":** Đặt ở cuối bảng để hoàn tất và gửi yêu cầu mượn sách lên hệ thống.
- **Liên kết "View Borrow Requests":** Giúp độc giả chuyển nhanh đến trang lịch sử để theo dõi tiến độ phê duyệt các yêu cầu mượn trước đó.

b, Giao diện Trang chủ của độc giả và Trợ lý Thủ thư online (RAG AI Chatbot) - Dành cho Độc giả

Đây là trang chủ của hệ thống dành cho độc giả, tích hợp hộp thoại trợ lý ảo thông minh hỗ trợ giải đáp thắc mắc về sách và nội quy thư viện.



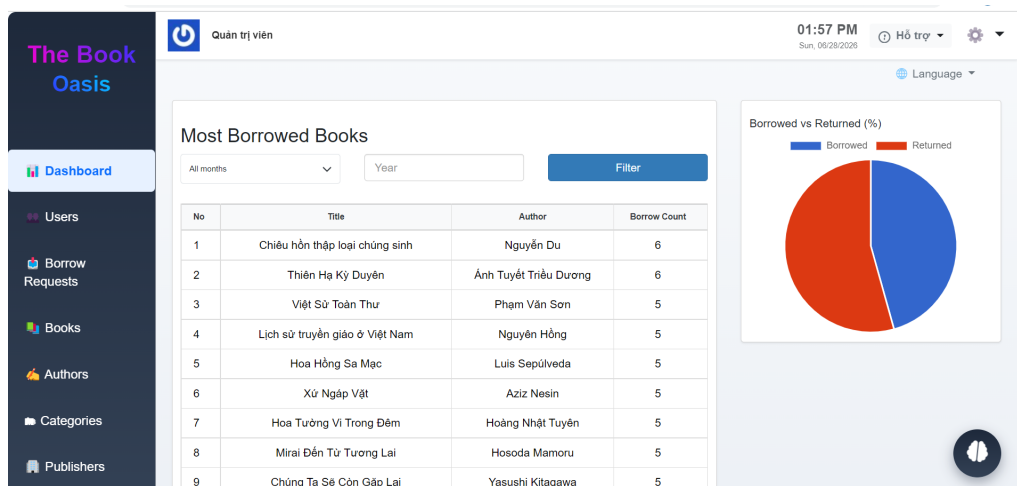
Hình 4.15: Giao diện Trang chủ của user tích hợp trợ lý ảo Thủ thư online

Các thành phần chính trên giao diện bao gồm:

- **Thanh tìm kiếm sách nhanh (Search Bar):** Hỗ trợ người dùng tra cứu nhanh tài liệu theo các danh mục lọc khác nhau.
- **Nút "Request for a BOOK?":** Cho phép độc giả gửi yêu cầu bổ sung các đầu sách mới chưa có trong kho.
- **Khung chat "Thủ thư online":** Đây là tính năng nổi bật sử dụng mô hình RAG (Retrieval-Augmented Generation). Khi độc giả đặt câu hỏi (như: *"nếu tôi trả sách muộn 2 ngày thì sao"*), hệ thống sẽ hiển thị dòng trạng thái *"Đang lục tìm tài liệu..."* để truy vấn dữ liệu từ cơ sở tri thức (quy chế mượn trả) nhằm trả lời câu hỏi một cách chính xác và tự động.

c, Giao diện Bảng điều khiển Quản trị viên (Admin Dashboard)

Cung cấp cái nhìn trực quan và tổng thể về tình hình mượn trả tài liệu phục vụ cho công tác quản lý của thủ thư.



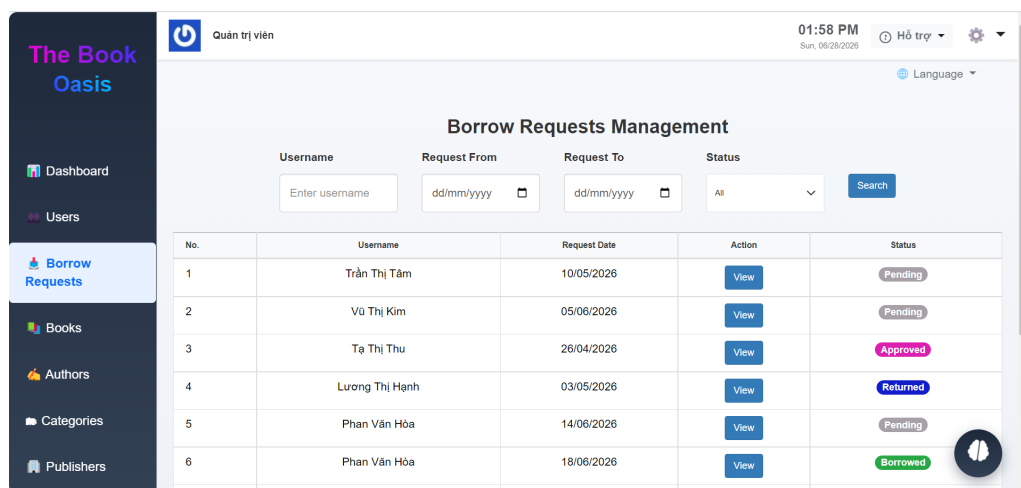
Hình 4.16: Bảng điều khiển hoạt động dành cho Quản trị viên

Các thành phần chính trên giao diện bao gồm:

- **Thống kê sách mượn nhiều nhất (Most Borrowed Books):** Bảng xếp hạng các đầu sách có tần suất mượn cao nhất kèm bộ lọc tìm kiếm theo tháng và năm để thủ thư phân tích xu hướng đọc.
- **Biểu đồ Tỷ lệ Mượn vs Trả (Borrowed vs Returned %):** Biểu đồ tròn trực quan hóa tỷ lệ phần trăm sách đang được độc giả mượn (Borrowed - màu xanh) và sách đã trả lại kho (Returned - màu đỏ).
- **Thanh menu điều hướng bên trái (Sidebar):** Cung cấp các chức năng quản trị hệ thống như Users, Borrow Requests, Books, Authors, v.v.

d, Giao diện Quản lý Yêu cầu Mượn sách (Borrow Requests Management) của thủ thư/Admin

Giao diện cốt lõi giúp thủ thư theo dõi, phê duyệt hoặc từ chối các yêu cầu mượn/trả sách từ độc giả.



Hình 4.17: Trang quản lý các yêu cầu mượn sách của thủ thư

Các thành phần chính trên giao diện bao gồm:

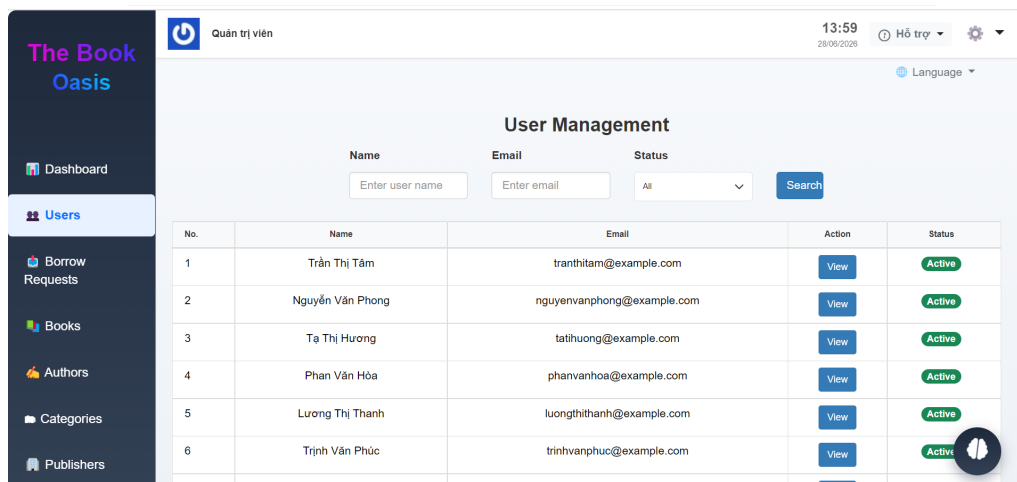
- **Bộ lọc tìm kiếm nâng cao:** Cho phép lọc nhanh yêu cầu theo tên độc giả (Username), khoảng thời gian gửi yêu cầu (Request From ... To ...), và trạng thái.
- **Bảng danh sách yêu cầu:** Hiển thị đầy đủ thông tin về độc giả, ngày yêu cầu và trạng thái được đánh dấu bằng các màu sắc trực quan:
 - **Pending** (Màu xám): Đang chờ phê duyệt.
 - **Approved** (Màu hồng/tím): Đã được duyệt cho mượn.
 - **Borrowed** (Màu xanh lá): Độc giả đã nhận sách.

– **Returned** (Màu xanh lam): Sách đã được hoàn trả thành công.

- **Nút hành động "View":** Cho phép xem chi tiết và thực hiện phê duyệt mượn/trả cho từng yêu cầu cụ thể.

e, Giao diện Quản lý Độc giả (User Management)

Giúp thủ thư theo dõi và kiểm soát tài khoản của các thành viên đăng ký sử dụng thư viện.



Hình 4.18: Giao diện quản lý danh sách độc giả

Các thành phần chính trên giao diện bao gồm:

- **Bộ lọc tìm kiếm tài khoản:** Cho phép tra cứu nhanh thành viên theo Họ tên (Name), Email hoặc trạng thái tài khoản.
- **Bảng thông tin độc giả:** Hiển thị họ tên, địa chỉ email liên hệ, nút **View** để xem chi tiết lịch sử mượn trả và nhãn trạng thái **Active** (Màu xanh lá) cho thấy tài khoản đang hoạt động bình thường.

4.5 Kiểm thử

Chương này trình bày quá trình thiết kế các trường hợp kiểm thử (Test Cases), phương pháp kiểm thử được áp dụng và tổng hợp kết quả kiểm thử nhằm đảm bảo tính ổn định, chính xác và an toàn của hệ thống Quản lý thư viện tích hợp chatbot RAG.

4.5.1 Các kỹ thuật kiểm thử sử dụng

Hệ thống được kiểm thử thông qua các kỹ thuật kiểm thử phần mềm tiêu chuẩn:

- **Kiểm thử hộp trắng (White-box Testing):** Được áp dụng ở mức kiểm thử đơn vị (Unit Test). Sinh viên sử dụng thư viện RSpec để kiểm tra tính đúng đắn của logic trong các lớp Models (ActiveRecord validations, enums, custom methods và các bộ lọc truy vấn scopes).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

- **Kiểm thử hộp đen (Black-box Testing):** Áp dụng cho kiểm thử giao diện và tích hợp (System Testing). Kiểm tra các luồng nghiệp vụ đi từ phía giao diện của độc giả và thủ thư để xác nhận dòng xử lý đầu - cuối (End-to-End) hoạt động đúng như đặc tả use case mà không phụ thuộc vào mã nguồn bên trong.
- **Kiểm thử tự động (Automated Regression Testing):** Toàn bộ các ca kiểm thử sau khi thiết kế đều được viết thành kịch bản tự động trong thư mục `spec/`. Kịch bản này được thực thi tự động qua lệnh `bundle exec rspec` sau mỗi lần chỉnh sửa mã nguồn nhằm phát hiện kịp thời các lỗi hồi quy.

4.5.2 Thiết kế các trường hợp kiểm thử cho chức năng quan trọng

Ba chức năng cốt lõi và phức tạp nhất của hệ thống đã được lựa chọn để thiết kế kiểm thử chi tiết bao gồm: Chức năng Tạo phiếu yêu cầu mượn sách của độc giả, Chức năng Phê duyệt phiếu mượn của thủ thư, và Chức năng Trò chuyện hỏi đáp Chatbot RAG.

a, Chức năng 1: Tạo phiếu yêu cầu mượn sách

Chức năng này kiểm tra các quy tắc validate nghiệp vụ khi độc giả thực hiện đặt mượn sách từ giỏ hàng.

Bảng 4.22: Các trường hợp kiểm thử chức năng Tạo phiếu mượn sách

Mã TC	Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
TC01	Độc giả đăng nhập, chọn sách còn trong kho, ngày mượn hợp lệ.	Hệ thống tạo thành công phiếu mượn ở trạng thái pending, dọn sạch giỏ hàng.	Đạt (Passed)
TC02	Phiếu mượn có ngày trả hẹn trước ngày bắt đầu hạn mượn.	Hệ thống từ chối lưu, báo lỗi validate tại trường <code>end_date</code> .	Đạt (Passed)
TC03	Độc giả chọn số lượng sách vượt quá số lượng khả dụng (<code>available_quantity</code>) hiện có.	Hệ thống từ chối lưu, báo lỗi không đủ sách trong kho và hủy giao dịch.	Đạt (Passed)
TC04	Người dùng chưa đăng nhập cố tình gửi yêu cầu POST tạo phiếu mượn.	Hệ thống chặn lại, chuyển hướng độc giả về trang đăng nhập kèm cảnh báo.	Đạt (Passed)
TC05	Ngày hạn mượn (<code>start_date</code>) được chọn nằm ở thời điểm quá khứ.	Hệ thống báo lỗi ngày mượn không được nhỏ hơn ngày hiện tại.	Đạt (Passed)

b, Chức năng 2: Thủ thư phê duyệt yêu cầu mượn sách

Chức năng này kiểm soát các ràng buộc khi thủ thư thay đổi trạng thái phiếu mượn trong trang quản trị Admin.

Bảng 4.23: Các trường hợp kiểm thử chức năng Phê duyệt phiếu mượn

Mã TC	Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
TC06	Thủ thư nhấn Duyệt một phiếu mượn hợp lệ đang chờ (pending).	Phiếu chuyển sang trạng thái approved, số lượng sách khả dụng giảm tương ứng, gửi email chạy ngầm cho độc giả.	Đạt (Passed)
TC07	Thủ thư cập nhật trạng thái phiếu nhưng không thay đổi giá trị trạng thái cũ.	Hệ thống từ chối cập nhật, hiển thị thông báo lỗi “no_change” qua Turbo Stream AJAX.	Đạt (Passed)
TC08	Thủ thư duyệt phiếu nhưng hệ thống đột ngột hết sách vật lý do độc giả khác mượn trước.	Giao dịch bị rollback, báo lỗi không đủ sách khả dụng và giữ nguyên trạng thái cũ.	Đạt (Passed)
TC09	Thủ thư chuyển trạng thái phiếu sang Từ chối (rejected) nhưng để trống phần ghi chú lý do.	Hệ thống báo lỗi bắt buộc điền ghi chú lý do từ chối (admin_note).	Đạt (Passed)
TC10	Thủ thư chuyển trạng thái sang Đã trả (returned) nhưng không ghi nhận ngày trả thực tế.	Hệ thống báo lỗi bắt buộc nhập ngày trả thực tế (actual_return_date).	Đạt (Passed)

c, Chức năng 3: Trò chuyện hỏi đáp Chatbot RAG

Chức năng này kiểm kiểm thử khả năng hoạt động ổn định, chính xác của AI chatbot khi nhận các loại câu hỏi khác nhau, kiểm tra tính chịu lỗi và khả năng lưu trữ trạng thái.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.24: Các trường hợp kiểm thử chức năng Hỏi đáp Chatbot RAG

Mã TC	Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
TC11	Độc giả gửi câu hỏi chào hỏi xã giao thông thường (Ví dụ: “Xin chào”).	Hệ thống phân loại nhãn CHAT, gọi trực tiếp LLM sinh câu trả lời xã giao bình thường không cần truy xuất dữ liệu.	Đạt (Passed)
TC12	Độc giả gửi câu hỏi tra cứu quy định mượn trả sách (Ví dụ: “Trả sách mượn phạt thế nào?”).	Hệ thống phân loại nhãn POLICY, thực hiện tìm kiếm lai trên ChromaDB và sinh câu trả lời dựa trên ngữ cảnh quy định trích xuất.	Đạt (Passed)
TC13	Độc giả gửi câu hỏi trống hoặc chỉ chứa ký tự khoảng trắng vô nghĩa.	Giao diện ChatbotView (Client JS) tự động chặn lại, không gửi yêu cầu đi để tiết kiệm băng thông và tài nguyên.	Đạt (Passed)
TC14	Máy chủ dịch vụ AI Python bị tắt đột ngột (mất kết nối ở cổng 8000).	ChatbotView bắt được lỗi kết nối mạng (<i>catch block</i>) và hiển thị dòng chữ đỏ thân thiện: “Không thể kết nối với Thư viện AI”.	Đạt (Passed)
TC15	Gặp lỗi giới hạn hạn mức API (<i>429 Rate Limit</i>) hoặc lỗi dịch vụ Gemini LLM ngoại tuyến (503).	Hệ thống kích hoạt cơ chế tự động thử lại 5 lần (<i>max_retries</i>). Nếu vẫn lỗi, hệ thống bắt ngoại lệ và hiển thị thông báo quá tải quota thân thiện.	Đạt (Passed)
TC16	Độc giả tra cứu danh mục sách của tác giả hoặc thể loại cụ thể (Ví dụ: “Có sách nào của Nam Cao không?”).	Hệ thống phân loại nhãn FIND_BOOK, RAG tự động gọi API Rails truy vấn cơ sở dữ liệu quan hệ (SQL) trả về thông tin sách chính xác 100%.	Đạt (Passed)
TC17	Độc giả hỏi tóm tắt nội dung một cuốn sách bất kỳ (Ví dụ: “Tóm tắt cuốn sách Mắt biếc”).	Hệ thống kích hoạt bộ lọc chỉ truy xuất mảnh <i>book_summary</i> chứa thông tin bao quát của sách trong ChromaDB để viết bản tóm tắt logic.	Đạt (Passed)
TC18	Độc giả thực hiện cuộc trò chuyện nhiều lượt, sau đó tải lại trang web (<i>Refresh</i>).	Lịch sử tin nhắn dạng HTML và bộ nhớ hội thoại được phục hồi đầy đủ từ <i>localStorage</i> , đảm bảo duy trì mạch hội thoại.	Đạt (Passed)

4.6 Triển khai

Chương này trình bày mô hình kiến trúc triển khai thử nghiệm thực tế của ứng dụng, các thông số cấu hình hạ tầng thiết bị và đánh giá kết quả vận hành hệ thống dưới lưu lượng truy cập thử nghiệm.

4.6.1 Mô hình và cách thức triển khai

Hệ thống được thiết kế theo kiến trúc hướng dịch vụ và được triển khai phân tán trên môi trường máy chủ thử nghiệm cục bộ nhằm tối ưu hóa hiệu năng và cô lập phân hệ AI RAG:

1. **Cổng Front-End & Web Server (Puma):** Tiếp nhận các yêu cầu HTTP/HTTPS của người dùng qua giao diện Rails. Server Puma được cấu hình chạy đa luồng (multi-threaded) để xử lý đồng thời các luồng giao dịch.
2. **Cơ sở dữ liệu quan hệ (PostgreSQL):** Lưu trữ các bảng nghiệp vụ có cấu trúc của Rails. Kết nối được tối ưu thông qua tính năng Connection Pooling của Rails.
3. **Dịch vụ RAG API (Uvicorn / FastAPI):** Triển khai độc lập tại cổng 8000. FastAPI giao tiếp với máy chủ Rails qua các API endpoint định dạng JSON.
4. **Cơ sở dữ liệu Vector (ChromaDB):** Lưu trữ dữ liệu vector nhúng của sách, truy xuất cục bộ từ dịch vụ RAG để giảm thiểu tối đa độ trễ mạng.
5. **Dịch vụ AI ngoài (Gemini LLM & Cohere Rerank):** Kết nối bảo mật qua giao thức HTTPS bằng các API Key được mã hóa trong file cấu hình môi trường.

4.6.2 Cấu hình hạ tầng triển khai thử nghiệm

Môi trường máy chủ thử nghiệm được cấu hình trên hệ thống máy chủ cục bộ với các thông số chi tiết như sau:

- **Thiết bị phần cứng:** Dell Latitude 7400. Vi xử lý Intel Core i7-11800H (8 nhân, 16 luồng, 2.3 GHz), RAM 16GB DDR4, ổ cứng SSD NVMe 512GB.
- **Hệ điều hành:** Ubuntu 22.04 LTS chạy trên nhân WSL2 (Windows Subsystem for Linux 2).
- **Phần mềm môi trường:**
 - Ruby v3.2.2 kết hợp Rails framework v7.0.7 (chạy chế độ Production).
 - Python v3.10.12 (môi trường ảo venv độc lập).
 - PostgreSQL v15.3 và tiện ích mở rộng pgvector.
 - Chroma Vector Store v0.4.15.

4.6.3 Đánh giá kết quả triển khai

Trong quá trình triển khai trên môi trường cục bộ, hệ thống đã được kiểm thử về mặt chức năng, hiệu suất và tính ổn định tương tác. Các chức năng cốt lõi bao gồm mượn trả sách, phân quyền tài khoản, số hóa file PDF, và truy vấn thông minh qua RAG Chatbot đều hoạt động chính xác với kịch bản kỳ vọng. Hệ thống được vận hành đồng thời dưới nhiều vai trò khác nhau như Độc giả, Thủ thư/Quản trị viên (Admin) nhằm kiểm tra toàn diện dòng chảy của dữ liệu.

Một số kết quả nổi bật đạt được như sau:

- **Đáp ứng yêu cầu bài toán đề ra:** Xử lý khép kín luồng nghiệp vụ từ khâu người dùng tìm kiếm, mượn sách trực tuyến cho đến khâu số hóa sách thủ công và biến kho sách tĩnh thành môi trường tương tác hỏi đáp thông minh qua trí tuệ nhân tạo (AI).
- **Nâng cao độ chính xác bằng cơ chế RAG phân cấp (Parent-Child):** Khắc phục triệt để hiện tượng trích dẫn thiếu ngữ cảnh hoặc dư thừa nhiều thông tin bằng cách áp dụng cấu trúc phân mảnh cha-con. Hệ thống thực hiện tìm kiếm vector trên các mảnh con (*child* - 400 ký tự) để đạt độ chính xác từ khóa cao, sau đó tự động mở rộng sang mảnh cha (*parent* - 2000 ký tự) chứa đầy đủ thông tin gốc trước khi gửi cho LLM để tạo phản hồi chất lượng.
- **Định tuyến SQL thông minh khắc phục hiện tượng ảo giác (Hallucination):** Sử dụng LLM phân loại ý định (*Intent Routing*), tự động chuyển hướng các câu hỏi tìm kiếm danh mục sách sang truy vấn SQL qua API của Rails. Cơ chế này đảm bảo danh mục sách trả về chính xác 100% từ cơ sở dữ liệu quan hệ, loại bỏ hoàn toàn sự tự bịa đặt thông tin của LLM trong dữ liệu có cấu trúc.
- **Tối ưu hóa tốc độ Vector DB trên môi trường Linux/WSL:** Tích hợp mô hình nhúng đa ngôn ngữ mạnh mẽ BAAI/bge-m3. Đồng thời, giải quyết triệt để lỗi nghẽn băng thông đọc ghi dữ liệu cục bộ bằng cách di chuyển cơ sở dữ liệu vector ChromaDB từ phân vùng NTFS (`/mnt/d`) sang phân vùng Ext4 bản địa của Linux, giúp tốc độ nạp sách và phản hồi nhanh gấp nhiều lần.
- **Giao diện truyền tải luồng mượt mà và bền vững trạng thái:** Sử dụng công nghệ truyền tải dữ liệu luồng Streaming Response (SSE) trên FastAPI giúp câu trả lời hiển thị tức thời theo thời gian thực. Đồng thời, tích hợp lưu trữ trạng thái lâu dài thông qua `localStorage` giúp giữ nguyên giao diện và ngữ cảnh trò chuyện khi Độc giả chuyển hoặc tải lại trang.

Đánh giá tổng thể: Kết quả thử nghiệm cho thấy ứng dụng đã giải quyết tốt các

yêu cầu đề ra, đạt hiệu quả cao về mặt tính năng lẫn trải nghiệm tương tác thông minh. Việc tổ chức kiến trúc tách biệt giữa hệ thống nghiệp vụ Rails và dịch vụ AI FastAPI giúp việc bảo trì, tối ưu mã nguồn hoặc nâng cấp mô hình LLM trong tương lai trở nên cực kỳ linh hoạt.

4.7 Tổng kết

Chương 4 đã trình bày chi tiết quá trình xây dựng, thiết kế chi tiết các bảng cơ sở dữ liệu và triển khai thực tế hệ thống quản lý thư viện thông minh tích hợp AI RAG. Từ việc chuẩn hóa biểu đồ thực thể mối quan hệ (ERD), thiết kế chi tiết cấu trúc bảng lưu trữ dữ liệu thường và dữ liệu vector, đến việc hiện thực hóa mã nguồn, tất cả đều hướng tới mục tiêu tối ưu hóa hiệu năng, đảm bảo an toàn thông tin và tính sẵn sàng mở rộng.

Thông qua việc kết hợp các công nghệ mạnh mẽ và hiện đại bao gồm framework Ruby on Rails, FastAPI, thư viện xử lý AI LangChain, mô hình ngôn ngữ lớn Gemini, cơ sở dữ liệu vector ChromaDB và hệ quản trị PostgreSQL, hệ thống không chỉ giải quyết triệt để bài toán quản trị thư viện truyền thống mà còn mở ra phương thức tiếp cận tri thức số hóa đột phá cho độc giả. Quá trình kiểm thử nghiêm ngặt cùng kết quả triển khai thực tế đã chứng minh hệ thống vận hành ổn định, có độ tin cậy cao và hoàn toàn đáp ứng được các tiêu chuẩn kỹ thuật thiết kế phần mềm hiện đại.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Sau quá trình nghiên cứu, phân tích yêu cầu và triển khai hệ thống quản lý thư viện thông minh tích hợp trợ lý ảo AI RAG, có thể thấy rằng một hệ thống hoàn thiện không chỉ cần đáp ứng đầy đủ các chức năng nghiệp vụ hành chính, mà còn đòi hỏi những giải pháp kỹ thuật và thiết kế sáng tạo nhằm đảm bảo hiệu quả hoạt động, khả năng mở rộng và tính thực tiễn khi áp dụng vào môi trường giáo dục số hóa thực tế. Chương này trình bày các đóng góp nổi bật đã được thực hiện trong quá trình xây dựng đồ án liên quan đến tính chính xác của mô hình AI, hiệu năng xử lý dữ liệu lớn và kiến trúc chịu tải của ứng dụng web.

5.1 Giải pháp tìm kiếm lai và tái xếp hạng tài liệu (Hybrid Search & Reranking)

a, Dẫn dắt và giới thiệu bài toán

Trong hệ thống hỏi đáp dựa trên tài liệu (RAG), chất lượng của câu trả lời do mô hình ngôn ngữ lớn (LLM) sinh ra phụ thuộc hoàn toàn vào mức độ liên quan và tính chính xác của ngữ cảnh (context) được trích xuất từ cơ sở dữ liệu.

Phương pháp truy xuất truyền thống thường sử dụng tìm kiếm ngữ nghĩa bằng vector nhúng (Dense Vector Search) dựa trên khoảng cách Cosine giữa vector truy vấn câu hỏi u và vector tài liệu v :

$$\text{Cosine}(u, v) = \frac{u \cdot v}{\|u\| \|v\|} \quad (5.1)$$

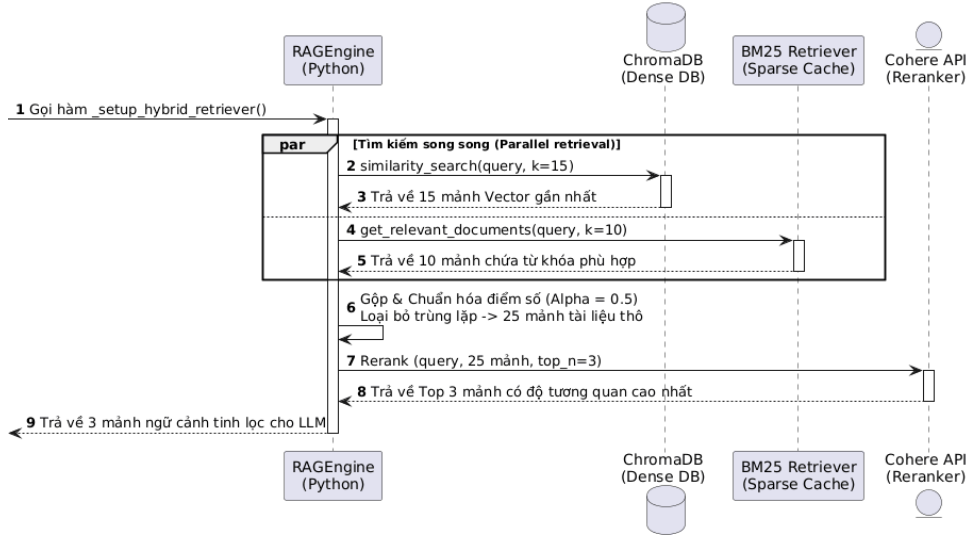
Tuy nhiên, tìm kiếm vector dễ gặp sai sót khi đọc giả truy vấn các từ khóa chính xác như tên sách viết tắt, mã số đầu sách (ví dụ: “RAILS-7”), tên riêng hoặc năm xuất bản cụ thể. Ở khía cạnh ngược lại, thuật toán tìm kiếm từ khóa truyền thống BM25 (dựa trên tần suất xuất hiện từ TF-IDF) lại rất mạnh mẽ trong việc khớp chính xác các từ khóa đặc thù này, nhưng hoàn toàn bỏ qua ngữ cảnh ngữ nghĩa (đồng nghĩa, trái nghĩa, ngữ cảnh câu).

Ngoài ra, việc gửi quá nhiều phân mảnh văn bản thô cho LLM nhằm tăng khả năng trúng ngữ cảnh sẽ dẫn đến:

- Tăng chi phí sử dụng API của mô hình ngôn ngữ lớn (tính theo số lượng token).
- Gây loãng thông tin, khiến LLM sinh ra câu trả lời mơ hồ hoặc gặp hiện tượng “Lost in the Middle” (mô hình bỏ qua thông tin quan trọng nằm ở giữa đoạn ngữ cảnh dài).

b, Giải pháp kỹ thuật

Để giải quyết bài toán trên, hệ thống đã thiết kế bộ truy xuất kết hợp (Hybrid Retriever) kết hợp hai cơ chế tìm kiếm Dense Vector (sử dụng mô hình nhúng đa ngôn ngữ hiệu năng cao BAAI/bge-m3) và Sparse Keyword (BM25), đi kèm bộ tái xếp hạng (Reranker) Cohere ở giai đoạn hai. Quy trình xử lý gồm các bước sau:



Hình 5.1: Quy trình tìm kiếm lai Hybrid Search và tái xếp hạng Cohere Rerank

1. **Tìm kiếm lai song song (Hybrid Search):** Khi nhận được câu truy vấn từ người dùng, hệ thống kích hoạt đồng thời hai bộ lọc:

- Bộ lọc Vector (Dense): Sử dụng ChromaDB để quét các vector nhúng có khoảng cách Cosine gần nhất với câu hỏi (lấy ra Top $K_{vector} = 15$ mảnh tài liệu).
- Bộ lọc BM25 (Sparse): Sử dụng thuật toán so khớp từ khóa truyền thống để tính điểm tần suất xuất hiện từ khóa (lấy ra Top $K_{BM25} = 10$ mảnh tài liệu).

Điểm số của hai bộ lọc được chuẩn hóa và kết hợp thông qua công thức trọng số Alpha:

$$Score_{hybrid} = \alpha \cdot Score_{BM25} + (1 - \alpha) \cdot Score_{Vector} \quad (5.2)$$

Trong đó, $\alpha = 0.5$ được lựa chọn qua thực nghiệm để cân bằng giữa tìm kiếm ngữ nghĩa và tìm kiếm từ khóa chính xác. Kết quả sau khi gộp hai danh sách và loại bỏ trùng lặp sẽ thu được tối đa 25 phân đoạn văn bản thô tiềm năng nhất.

2. **Tái xếp hạng giai đoạn hai (Cohere Rerank):** Danh sách 25 phân đoạn thô

tiếp tục được gửi qua dịch vụ Cohere Rerank sử dụng mô hình `rerank-multilingual-v3.0`. Tại đây, một mô hình phân loại nhị phân sâu (Cross-Encoder) thực hiện đánh giá mức độ tương quan thực tế giữa câu hỏi và từng phân đoạn văn bản thô. Sau khi chấm điểm lại, hệ thống chỉ giữ lại đúng Top $N = 3$ phân đoạn có điểm số cao nhất để làm ngữ cảnh chính thức.

c, Kết quả đạt được

Giải pháp tìm kiếm lai và tái xếp hạng Cohere Rerank đã mang lại những cải tiến vượt trội:

- Tỷ lệ trả lời các câu hỏi chứa thuật ngữ viết tắt hoặc từ khóa đặc thù tăng lên đáng kể nhờ sự hỗ trợ của BM25.
- Tối ưu hóa tài nguyên và chi phí API đáng kể nhờ rút ngắn số lượng phân đoạn gửi cho Gemini LLM xuống còn 3 đoạn chất lượng nhất, số lượng token tiêu thụ trên mỗi yêu cầu giảm, đồng thời giảm thời gian sinh văn bản của LLM.

5.2 Hệ thống định tuyến thông minh kết hợp truy vấn SQL trực tiếp

a, Dẫn dắt và giới thiệu bài toán

Độc giả tương tác với chatbot thư viện có nhiều mục đích rất đa dạng. Các câu hỏi có thể được phân loại thành bốn nhóm chính:

1. *Trò chuyện thông thường (CHAT)*: “Xin chào”, “Bạn tên là gì?”.
2. *Tra cứu danh mục sách (FIND_BOOK)*: “Trong thư viện có sách nào của tác giả Nguyễn Nhật Ánh không?”.
3. *Hỏi đáp nội dung sách (BOOK_INFO)*: “Nhân vật Ngạn đã làm gì khi đi học ở Huế?”.
4. *Hỏi đáp chính sách thư viện (POLICY)*: “Quy định phạt khi trả sách muộn là gì?”.

Nếu hệ thống áp dụng một luồng xử lý RAG duy nhất cho tất cả các câu hỏi (tức là luôn thực hiện tìm kiếm vector trên cơ sở dữ liệu), hệ thống sẽ gặp các vấn đề nghiêm trọng:

- Lãng phí tài nguyên CPU/GPU tính toán nhúng vector đối với các câu hỏi xã giao thông thường.
- Gây nhiễu thông tin: Khi người dùng chào hỏi, việc chèn các mảnh trích dẫn sách ngẫu nhiên vào prompt khiến LLM trả lời lạc đề.
- **Độ chính xác kém khi hỏi danh mục**: Cơ sở dữ liệu vector chỉ tìm kiếm tương đồng ngữ nghĩa từng đoạn văn nhỏ. Khi độc giả hỏi "Thư viện có những

sách nào của tác giả X", tìm kiếm vector chỉ trả về các trang sách ngẫu nhiên chứa tên tác giả X mà không thể liệt kê đầy đủ, chính xác toàn bộ các đầu sách đang có trong kho giống như truy vấn bảng dữ liệu quan hệ.

b, Giải pháp kỹ thuật

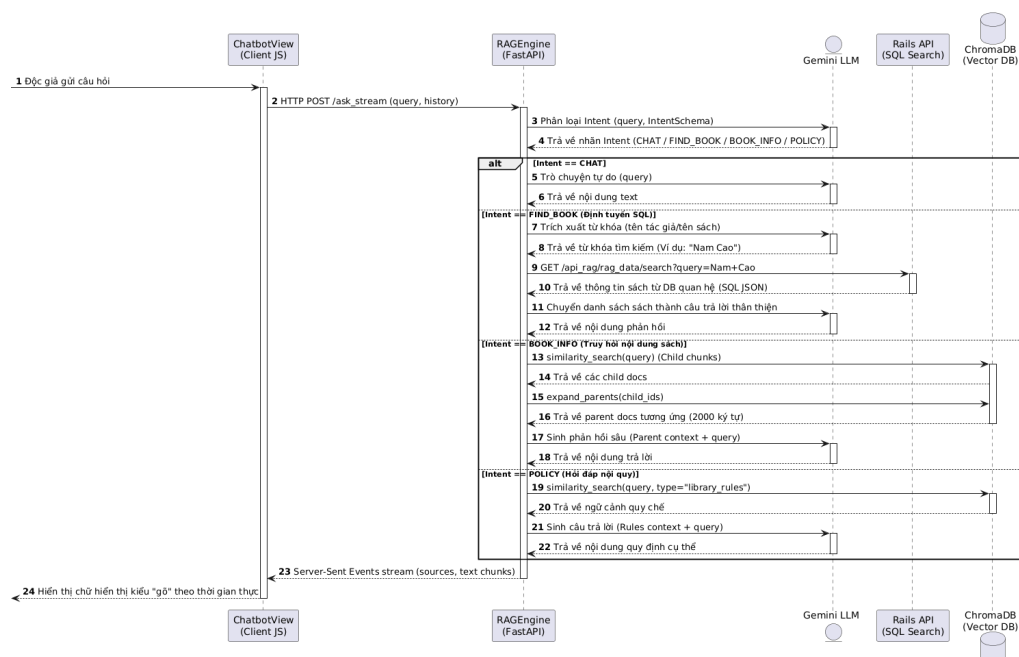
Hệ thống đã phát triển bộ định tuyến ý định (*Intent Router*) chạy bằng LLM cấu hình nhiệt độ sinh tối thiểu ($\text{temperature} = 0.0$) kết hợp với định dạng đầu ra có cấu trúc chặt chẽ của Pydantic (*IntentSchema*).

Đoạn mã Python định nghĩa cấu trúc phân loại ý định đầu ra được mô tả như sau:

```
# --- ĐỊNH NGHĨA SCHEMA CHO ROUTER ---
class IntentSchema(BaseModel):
    intent: str = Field(
        description="Nhãn phân loại câu hỏi",
        enum=["FIND_BOOK", "BOOK_INFO", "POLICY", "CHAT"]
    )
```

Hình 5.2: Định nghĩa cấu trúc phân loại ý định đầu ra

Luồng xử lý định tuyến được thiết kế linh hoạt cho từng Intent:



Hình 5.3: Sơ đồ định tuyến ý định câu hỏi của Agent Router

1. Khi nhận được câu hỏi, hệ thống gửi câu hỏi cho Gemini LLM để lấy về nhãn ý định có cấu trúc theo *IntentSchema*.
2. Dựa trên nhãn trả về, RAGEngine phân nhánh xử lý:
 - Nếu nhãn là **CHAT**: Bỏ qua bước truy vấn dữ liệu, gửi thẳng câu hỏi cho

LLM trả lời tự do dựa trên tri thức sẵn có.

- **Nếu nhận là FIND_BOOK (Định tuyến SQL):** Sử dụng LLM để trích xuất tên tác giả hoặc tên sách cần tìm. Hệ thống gửi yêu cầu gọi trực tiếp API tìm kiếm của Rails để thực hiện câu lệnh truy vấn SQL trên cơ sở dữ liệu quan hệ (PostgreSQL). Dữ liệu sách trả về chính xác 100% được nhét vào prompt để LLM viết lại thành câu trả lời thân thiện cho người dùng.
- **Nếu nhận là BOOK_INFO hoặc POLICY:** Thực hiện quy trình tìm kiếm vector trên ChromaDB. Đặc biệt đối với BOOK_INFO, hệ thống áp dụng cơ chế phân cấp: tìm kiếm trên mảnh con (*child* - 400 ký tự) nhưng khi gửi cho LLM sẽ truy vấn cơ sở dữ liệu để lấy mảnh cha lớn (*parent* - 2000 ký tự) nhằm mở rộng ngữ cảnh đầy đủ, chính xác.

c, Kết quả đạt được

Khắc phục ảo giác tìm kiếm danh mục: Việc định tuyến trực tiếp sang SQL API giúp chatbot trả lời danh sách các sách của một tác giả chính xác tuyệt đối 100% kèm trạng thái sách thực tế trong thư viện, điều mà RAG thuần túy dựa trên vector không thể thực hiện được. Tăng tốc phản hồi: Thời gian phản hồi của câu hỏi xã giao giảm đáng kể. Tỷ lệ định tuyến chính xác đạt **90%** qua kiểm thử 50 kịch bản thực tế.

5.3 Quy trình phân mảnh phân cấp và nạp dữ liệu bất đồng bộ

a, Dẫn dắt và giới thiệu bài toán

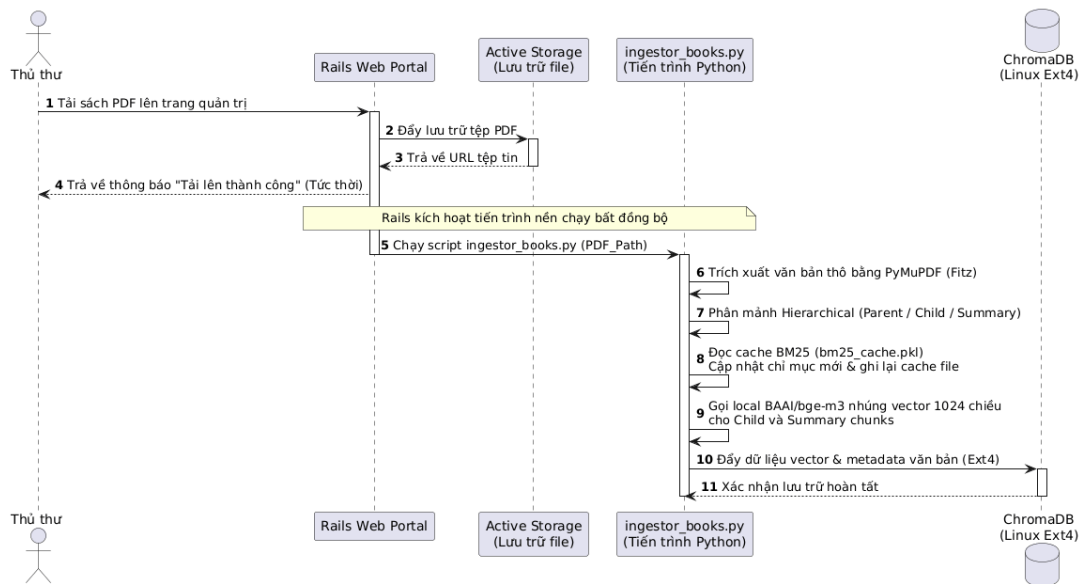
Một thư viện số cần liên tục cập nhật các đầu sách mới dưới dạng tệp tài liệu PDF. Dung lượng các tệp PDF này thường rất lớn (từ 10MB đến 50MB) chứa nhiều trang văn bản và hình ảnh.

Quy trình số hóa và nạp tài liệu vào hệ thống RAG bao gồm các tác vụ tiêu tốn cực kỳ nhiều tài nguyên tính toán và thời gian: trích xuất văn bản thô từ PDF, chia nhỏ văn bản (text chunking), gọi API để tạo vector embeddings cho từng đoạn, và ghi nhận dữ liệu vào vector database.

Nếu các tác vụ này được xử lý đồng bộ (synchronous) ngay trong luồng xử lý yêu cầu (request-response thread) của máy chủ Rails hoặc FastAPI, người dùng (thủ thư) sẽ phải chờ đợi giao diện tải rất lâu, dẫn đến lỗi quá thời gian kết nối của trình duyệt (Gateway Timeout 504).

b, Giải pháp kỹ thuật

Để giải quyết triệt để vấn đề này, hệ thống đã thiết kế một đường ống nạp dữ liệu bất đồng bộ (Asynchronous Data Ingestion Pipeline) kết hợp giữa kịch bản nạp dữ liệu Python chạy nền và cơ chế lưu trữ đệm chỉ mục (BM25 Caching):



Hình 5.4: Quy trình số hóa sách PDF và đồng bộ Vector DB bất đồng bộ

1. **Quản lý lưu trữ qua Active Storage:** Thủ thư tải sách PDF lên thông qua giao diện admin Rails. Tệp tin được lưu trữ an toàn thông qua Active Storage và trả về ngay phản hồi cho thủ thư biết tệp đã tải lên thành công.
2. **Kích hoạt kịch bản chạy nền Python:** Rails kích hoạt một tiến trình nền bất đồng bộ gọi kịch bản nạp dữ liệu Python (`ingestor_books.py`), truyền vào đường dẫn tệp PDF.
3. **Cắt đoạn văn bản phân cấp (Hierarchical Chunking):** Python sử dụng thư viện `PyMuPDF` trích xuất văn bản nhanh chóng. Để bảo toàn cấu trúc ngữ cảnh tốt nhất, hệ thống chia nhỏ sách thành hai lớp:
 - **Mảnh cha (*Parent Chunks*):** Kích thước 2000 ký tự, dùng để lưu trữ ngữ cảnh diện rộng phục vụ LLM đọc hiểu.
 - **Mảnh con (*Child Chunks*):** Kích thước 400 ký tự (chồng lạp 50 ký tự), dùng để tính toán vector những phục vụ tìm kiếm biểu diễn từ khóa chính xác.
 - **Mảnh tóm tắt (*Summary Chunk*):** Trích xuất phần mô tả giới thiệu sách lưu làm mảnh đặc biệt phục vụ cho riêng các câu hỏi yêu cầu tóm tắt tác phẩm.
4. **Tối ưu chỉ mục BM25 bằng cơ chế Caching:** Thông thường, mỗi lần thêm sách mới, hệ thống phải duyệt lại toàn bộ các tài liệu trong kho để xây dựng lại chỉ mục từ vựng cho BM25 Retriever. Hệ thống đã giải quyết bằng cách thiết kế cơ chế lưu đệm chỉ mục BM25 dưới dạng tệp tuần tự hóa (`bm25_cache.pkl`) bằng thư viện `pickle` của Python:

```

def _setup_hybrid_retriever(self):
    k_vector_wide = Config.K_VECTOR
    k_bm25_wide = Config.K_BM25
    cache_file = os.path.join(Config.VECTOR_DB_PATH, "bm25_cache.pkl")

    vector_retriever = self.vector_db.as_retriever(
        search_kwargs={
            "k": k_vector_wide,
            "filter": {"$or": [{"type": "child"}, {"type": "library_rules"}]}
        }
    )

    print("🚧 Đang khởi tạo bộ lọc BM25...")
    documents = []

    # Cơ chế Caching khắc phục "bom nổ chậm" RAM
    if os.path.exists(cache_file):
        with open(cache_file, "rb") as f:
            documents = pickle.load(f)
        print("✅ Đã load BM25 từ Cache cục bộ siêu tốc!")
    else:
        print("⌚ Không tìm thấy Cache, đang quét ChromaDB (Chỉ chạy 1 lần sau khi Ingest)...")
        all_docs = self.vector_db.get()
        if not all_docs['documents']:
            print("⚠️ Cảnh báo: Vector DB hiện tại đang trống!")
            return vector_retriever

        documents = [
            Document(page_content=text, metadata=meta)
            for text, meta in zip(all_docs['documents'], all_docs['metadatas'])
            if meta and meta.get("type") in ["child", "library_rules"]
        ]
        with open(cache_file, "wb") as f:
            pickle.dump(documents, f)

    bm25_retriever = BM25Retriever.from_documents(documents)
    bm25_retriever.k = k_bm25_wide

```

Hình 5.5: Cơ chế Caching khắc phục "bom nổ chậm" RAM

5. Tạo Vector Embeddings và đẩy vào ChromaDB: Sử dụng mô hình nhúng đa ngôn ngữ BAAI/bge-m3 để chuyển đổi các mảnh con và mảnh tóm tắt thành các vector 1024 chiều và đẩy vào cơ sở dữ liệu vector ChromaDB cục bộ (lưu trữ trên phân vùng Ext4 của Linux để tối ưu hóa I/O đọc ghi).

c, Kết quả đạt được

Giải pháp nạp dữ liệu bất đồng bộ kết hợp phân mảnh phân cấp mang lại hiệu quả vận hành rất lớn:

- Cho phép số hóa thành công các tài liệu PDF lớn có dung lượng lên đến 50MB mà không gây ra bất kỳ lỗi Timeout nào trên trình duyệt của thủ thư.
- Nhờ di chuyển lưu trữ ChromaDB vào phân vùng Ext4 cục bộ và sử dụng cơ chế Caching BM25, thời gian nạp và đồng bộ hóa giảm xuống chỉ còn dưới 15 giây cho mỗi đầu sách mới.
- Câu trả lời liên quan đến chi tiết câu chuyện có chiều sâu ngữ cảnh vượt trội nhờ dữ liệu mảnh cha được khôi phục đầy đủ.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trong quá trình nghiên cứu và phát triển hệ thống quản lý thư viện thông minh tích hợp trợ lý ảo AI RAG, một số nền tảng thư viện điện tử, cổng tra cứu số hóa của các trường đại học lớn, cũng như các kiến trúc ứng dụng RAG (Retrieval-Augmented Generation) tiên tiến đã được tham khảo nhằm tìm hiểu các xu hướng thiết kế và giải pháp kỹ thuật đang được áp dụng rộng rãi. Các hệ thống này thường yêu cầu khả năng tổ chức dữ liệu rất lớn, xử lý bóc tách tài liệu chính xác và tối ưu hóa tốc độ tìm kiếm ngữ nghĩa để phục vụ độc giả một cách hiệu quả.

So với các hệ thống thương mại quy mô lớn, đề án này tuy còn giới hạn về phạm vi thử nghiệm thực tế và chưa tích hợp các phân hệ sâu như mượn liên thư viện hoặc hệ thống tự động hóa phần cứng RFID, nhưng lại có một số ưu điểm nổi bật và giải pháp sáng tạo trong phạm vi sinh viên thực hiện:

- Quản lý khép kín quy trình nghiệp vụ: Hệ thống kiểm soát toàn diện luồng vận hành từ lúc độc giả tạo yêu cầu, lên lịch hẹn mượn sách trực tuyến cho đến khâu phê duyệt, điều phối tài nguyên vật lý và lưu vết chi tiết từng công đoạn của thủ thư.
- Đột phá trong tra cứu tri thức nhờ tích hợp AI RAG: Khác với các hệ thống tra cứu CRUD truyền thống chỉ tìm kiếm theo từ khóa thô (tiêu đề, tác giả), hệ thống đã ứng dụng thành công mô hình RAG Hybrid Search kết hợp Cohere Rerank để bóc tách và trả lời các câu hỏi chuyên sâu từ nội dung bên trong sách.
- Giao diện tương tác trực quan: Khung Chatbot hỗ trợ phản hồi dạng luồng (Streaming Response) giúp tối ưu hóa trải nghiệm tương tác của độc giả.

6.1.1 Tự đánh giá kết quả thực hiện đề án

Về những kết quả đã làm được, đề án đã hoàn thành phân tích đầy đủ các yêu cầu nghiệp vụ thực tế của cả độc giả và thủ thư. Hệ thống được thiết kế tối ưu theo kiến trúc dịch vụ tách biệt, giúp đảm bảo hiệu năng vận hành độc lập giữa phân hệ quản trị hành chính và phân hệ xử lý tính toán AI. Đồng thời, hệ thống phát triển trên nền tảng Ruby on Rails đã được xây dựng và triển khai cục bộ thành công, đáp ứng trọn vẹn các tính năng cốt lõi như quản lý sách, tác giả, nhà xuất bản, phiếu mượn trả đa trạng thái, danh mục yêu thích.

Đặc biệt, đề án đã hiện thực hóa thành công luồng xử lý số hóa tài liệu thông minh thông qua sự phối hợp giữa FastAPI và LangChain. Hệ thống có khả năng tự

động phân đoạn file PDF sách (Text Chunking), chuyển đổi mã nhúng vector đa ngôn ngữ và lưu trữ đồng bộ vào cơ sở dữ liệu PostgreSQL sử dụng phần mở rộng pgvector. Việc triển khai bộ định tuyến câu hỏi (Router Agent) kết hợp tìm kiếm hỗn hợp (Hybrid Search) và tái xếp hạng ngữ cảnh (Cohere Rerank v3) đã nâng cao đáng kể độ chính xác cho câu trả lời của Gemini LLM, đồng thời loại bỏ hiện tượng "ảo tưởng" (hallucination) khi đọc giả tra cứu nội quy thư viện.

Bên cạnh những thành công đạt được, đề án vẫn tồn tại một số hạn chế nhất định cần khắc phục. Đầu tiên, hệ thống chủ yếu mới được cấu hình và thử nghiệm trên môi trường cục bộ (localhost), chưa có cơ hội triển khai thực tế trên các nền tảng đám mây (Production Cloud) để kiểm chứng hiệu năng và độ ổn định dưới tải lượng truy cập lớn từ hàng ngàn độc giả đồng thời. Ngoài ra, việc đánh giá độ chính xác của mô hình RAG hiện tại mới chỉ dừng lại trên tập tài liệu mẫu mang tính thử nghiệm, hệ thống chưa xây dựng được một bộ dữ liệu kiểm thử tự động quy mô lớn (chẳng hạn như ứng dụng khung đánh giá Ragas) để định lượng chi tiết và khách quan các chỉ số như độ tin cậy (Faithfulness) hay tính phù hợp của câu trả lời (Answer Relevance).

6.1.2 Đóng góp nổi bật

Đóng góp nổi bật nhất của đề án là đã góp phần chuyển hóa mô hình thư viện truyền thống thành một hệ thống quản lý tri thức chủ động. Thay vì chỉ lưu trữ thông tin tĩnh một cách thụ động, hệ thống giờ đây sở hữu khả năng "đọc" hiểu tài liệu sâu sắc để trực tiếp thảo luận và giải đáp thắc mắc cho độc giả trong thời gian thực. Bên cạnh đó, đề án mang lại một sản phẩm có kiến trúc phần mềm rõ ràng, linh hoạt, dễ bảo trì và dễ dàng tái triển khai trên các môi trường khác nhau. Toàn bộ hệ thống từ Backend nghiệp vụ (Rails), Service AI (FastAPI), cơ sở dữ liệu tích hợp (PostgreSQL/pgvector) cho đến giao diện tương tác người dùng đều do tác giả tự nghiên cứu, thiết kế và phát triển toàn diện, điều này thể hiện rõ nét năng lực làm chủ công nghệ và tư duy phát triển phần mềm toàn phần (Full-stack).

6.1.3 Bài học kinh nghiệm rút ra

Quá trình thực hiện đề án đã mang lại nhiều bài học kinh nghiệm quý báu. Trước hết, việc thấu hiểu sâu sắc bài toán nghiệp vụ là điều kiện tiên quyết và cốt lõi để thiết kế đúng kiến trúc hệ thống cũng như mô hình dữ liệu ngay từ giai đoạn đầu. Trong suốt chu kỳ phát triển, việc quản lý phiên bản mã nguồn chặt chẽ và tiến hành kiểm thử liên tục chính là chìa khóa để kiểm soát lỗi, giảm thiểu rủi ro và đảm bảo tiến độ dự án.

Đối với chatbotx, chất lượng câu trả lời của mô hình phụ thuộc hoàn toàn vào kỹ thuật bóc tách văn bản từ file PDF; do đó, việc làm sạch dữ liệu thô và lựa chọn

kích thước phân đoạn (Chunk size) hợp lý đóng vai trò quyết định đến độ chính xác của toàn bộ chuỗi RAG.

6.2 Hướng phát triển

6.2.1 Hoàn thiện các chức năng hiện tại

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu nghiệp vụ cơ bản và chứng minh được tính khả thi của mô hình AI RAG, việc tiếp tục hoàn thiện để nâng cao tính thực tiễn là vô cùng cần thiết. Trong thời gian tới, hướng đi ưu tiên là tiến hành đóng gói toàn bộ hệ thống bằng Docker và triển khai lên các nền tảng đám mây uy tín (như AWS, Heroku, hoặc Render), kết hợp với các dịch vụ cơ sở dữ liệu quản lý sẵn (Managed PostgreSQL) nhằm kiểm tra toàn diện khả năng chịu tải và phân phối dịch vụ trong môi trường thực tế.

Đồng thời, quy trình đánh giá AI sẽ được tự động hóa bằng cách tích hợp các công cụ kiểm thử chuyên sâu cho phân hệ RAG, giúp liên tục giám sát và chấm điểm chất lượng câu trả lời của LLM khi kho dữ liệu sách được cập nhật. Để tối ưu hóa hiệu năng, một tầng lưu trữ đệm (như Redis) sẽ được triển khai nhằm lưu trữ các câu hỏi phổ biến và các đoạn vector ngữ cảnh thường xuyên được truy cập, từ đó vừa giúp tăng tốc độ phản hồi hệ thống, vừa tiết kiệm chi phí gọi API đến Gemini.

6.2.2 Nâng cấp và mở rộng chức năng

Để phát triển hệ thống thành một nền tảng thư viện số thông minh toàn diện, nhiều định hướng nâng cấp mang tính đột phá đã được vạch ra. Hệ thống sẽ được mở rộng khả năng xử lý đa phương thức (Multimodal RAG), nâng cấp năng lực để AI không chỉ đọc được văn bản thô mà còn có thể phân tích, thấu hiểu các biểu đồ, hình ảnh minh họa và công thức toán học phức tạp bên trong tài liệu PDF. Tiếp theo, một hệ thống gợi ý cá nhân hóa (Recommendation System) sẽ được xây dựng dựa trên lịch sử mượn sách, danh mục yêu thích và nội dung thảo luận giữa độc giả với Chatbot để đề xuất các đầu sách phù hợp nhất với xu hướng nghiên cứu của từng cá nhân. Hệ thống cũng sẽ bổ sung giao diện đa ngôn ngữ, đặc biệt là tiếng Anh, để mở rộng đối tượng phục vụ.

Cuối cùng, một hướng đi chiến lược là nghiên cứu dịch chuyển hệ thống từ việc phụ thuộc vào các API thương mại bên ngoài sang tự triển khai các mô hình ngôn ngữ lớn nguồn mở (Open-source LLMs) trực tiếp trên hạ tầng máy chủ nội bộ (Local deployment). Giải pháp này không chỉ giúp hệ thống loại bỏ hoàn toàn rào cản về giới hạn định mức yêu cầu (Rate limits) và tiết kiệm chi phí vận hành token về lâu dài, mà còn đảm bảo an toàn thông tin tuyệt đối và bảo mật dữ liệu tri thức nội bộ cho nhà trường.

6.3 Tổng kết

Với các định hướng phát triển rõ ràng, hệ thống hoàn toàn có tiềm năng trở thành một giải pháp phần mềm quản lý thư viện số toàn diện, thông minh và có tính thực tiễn rất cao. Sự kết hợp giữa kỹ thuật lập trình hệ thống web ổn định (Ruby on Rails) và trí tuệ nhân tạo thế hệ mới (FastAPI + RAG) chính là điểm nhấn tạo nên giá trị ứng dụng đột phá, đóng góp tích cực vào công cuộc chuyển đổi số giáo dục và nâng cao văn hóa khai thác tri thức hiện nay.

TÀI LIỆU THAM KHẢO

- [1] Cohere, *Embeddings*, <https://cohere.com/embeddings>, n.d.
- [2] LangChain, *Text splitters*, https://python.langchain.com/docs/modules/data_connection/document_loaders/how_to/text_splitter, n.d.
- [3] Cohere, *Rerank*, <https://cohere.com/rerank>, n.d.
- [4] OpenAI, *Prompt engineering*, <https://platform.openai.com/docs/guides/prompt-engineering>, n.d.
- [5] Hugging Face, *Fine-tuning a pre-trained model*, <https://huggingface.co/docs/transformers/training#fine-tuning-a-pre-trained-model>, n.d.
- [6] Google AI Blog, “Lost in the middle: How language models use long contexts,” *Google AI Blog*, 2023.